Welcome to API Programming Help

Introduction

Object model overview

COM availability

In the left hand pane are three tabs: **Contents**, **Index** and **Search**. Select one of these to navigate around the help system effectively.

- Use **Contents** to learn about a new topic. The expanding books will always show your location within the help system.
- Use **Index** to look up details on a specific feature.
- If these methods do not offer what you're looking for, use **Search** to conduct a text search through the entire system.

Introduction

An Application Programming Interface (API) is a functional interface exposed by a software application as a means for other software applications to interact with it. In other words, an API is a set of routines provided by an application in such a way that other applications can use them.

The API in the software is important for the following reasons:

It allows you to customise the software and add specific functionality to meet your particular business requirements

It promotes automation of repetitive tasks, which may help your productivity

It allows integration with other systems, for example, sharing of data with resource planning systems

The API can be called from another application that is written in a language that supports COM (eg. Visual Basic, VB.NET, Visual C++, C#, Delphi, Perl, etc). The API can also be called from application macros and plug-ins written in any .NET language (eg. VB.NET, C#, etc). Use of the API assumes that you are familiar with one or more of these programming languages.

A simple example of programming with the API using VBA is given here:

Application object example

A simple example of programming with the API using C# is given here:

Creating .NET plug-ins

Object model overview

The API is exposed through a set of **objects**. An object has events, methods and properties that you can use to both query and edit the object.

To start customising the software using the API, you should first learn about the structure of the API. This structure is often referred to as the **object model**. An overview of the object model is given here. Detail of each object, its events, methods and properties can be found in subsequent sections of the online help.

At the top of the object model hierarchy is the **Application** object. All other objects in the model are located below the Application object, and can be accessed via this top-level object. Detailed knowledge of the object model hierarchy will mean that you can readily traverse through the hierarchy to gain access to a particular type of object.

A large number of objects in the API can only be accessed indirectly. In such cases you must reference the object from another object higher in the object hierarchy. For example, the **MacArguments** object can only be accessed indirectly. It cannot exist on its own, in fact the MacArguments object exists only as a member of the **MacCommand** object. Therefore access to the MacArguments object is possible only from the MacCommand object.

Another concept vital to the API is the collection object. A collection object is a special type of object, allowing iteration over an existing set of objects. All collection objects support the Count property and the Item method. Count returns the number of items in the collection while Item returns the indicated item in the collection. Further properties and methods are defined according to requirements for each collection object. An example of a collection object is the **MacCommands** collection. In addition to the Count property and the Item method, the MacCommands object also supports the Add method which allows creation of new objects into the collection.

Note: When using the API from VBA, you can view all of the available objects using the **Object Browser** in the VBA editor. This provides a quick and easy point of reference for details of the API.

COM availability

The objects in the API are used via the Component Object Model (COM), an open software architecture allowing interoperation between this and other software programs. The COM interface in this program is generally available for use by other programs, but there are occasions when it is temporarily disabled.

If an external client (i.e. some other program) attempts to make a COM connection to this program at a time when this program's COM interface is not available then the COM call will fail with the error code APC_E_CALL_REJECTED, which is defined as 0x80040529 or -2147220183 and typically produces an error message such as "Call rejected by callee."

This program itself will not be able to make COM calls internally at all while its COM interface is disabled.

The COM interface is disabled when the program is in any of the following states:

In verifier mode within the nest editor state.

In polygon window mode.

Showing any "modal" dialog (i.e. a pop-up dialog box which prevents you from doing anything else with the program until that dialog box is dismissed, e.g. the File | Open dialog box).

Waiting for user input from a keystroke command or MAC macro.

Waiting for long calculations to complete, e.g. running the nester or drawing hatching.

Moving clamps, pickers and chutes.

Objects

Application object

Document object

Mac object

MacArgument object

MacArguments collection

MacCommand object

MacCommands collection

MacFile object

MacFiles collection

PluginManager object

SystemDataFiles collection

SystemDataFile object

SystemDataItem object

Application object

The Application object is the top-level object. It provides a convenient point of access to the rest of the object model.

The Application object has the following methods and properties:

Methods

Method	Description
Help As Boolean	Displays online help. **Return Value** Returns **True** if the online help was displayed successfully; otherwise **False**.
NewDrawing(• *DiscardChanges* As Boolean)	Starts a new, empty drawing after closing any open drawing or symbol. **Parameters** DiscardChanges Discard any changes made to the currently open drawing or symbol when starting the new drawing. **Return Value** Nothing - if a new drawing can't be created the method will cause an error.
NewSymbol(• *DiscardChanges* As Boolean)	Starts a new, empty symbol in the part editor after closing any open symbol. **Parameters** DiscardChanges Discard any changes made to the currently open symbol when starting the new symbol. **Return Value** Nothing - if a new symbol can't be created the method will cause an error.
OpenDrawing(• *FilePath* As String • *DiscardChanges* As Boolean • *OptionsFilePath* As String)	Opens the given drawing file after closing any open drawing or symbol. This method can also open DXF, DWG and IGES files. **Parameters** FilePath The path of the drawing to open. DiscardChanges Discard any changes made to the currently open drawing or symbol when opening the given drawing. OptionsFilePath The import options for DXF, DWG and IGES files, given either as the path of an options file or (for DXF and DWG files only) as the name of a template.

Return Value

Nothing - if the drawing can't be opened, the method will cause an error.

Opens the given raster image file as a sheet remnant drawing after closing any open drawing or symbol.

Parameters

OpenSheetRemnantDrawingFromRasterImage
(

- *FilePath* As String
- *DiscardChanges* As Boolean
- *OptionsFilePath* As String
- *SheetX* As Double

)

FilePath
: The path of the raster image to open.

DiscardChanges
: Discard any changes made to the currently open drawing or symbol when opening the given raster image.

OptionsFilePath
: The raster-to-vector conversion options, given as the path of an options file.

SheetX
: The sheet size in X. Overrides the setting in the options file (if given).

Return Value

Nothing - if the raster image can't be opened, the method will cause an error.

Opens the given symbol file in the part editor after closing any open symbol. This method can also open DXF and DWG files.

Parameters

OpenSymbol(

- *FilePath* As String
- *DiscardChanges* As Boolean
- *OptionsFilePath* As String

)

FilePath
: The path of the symbol to open.

DiscardChanges
: Discard any changes made to the currently open symbol when opening the given symbol.

OptionsFilePath
: The import options for DXF and DWG files, given either as the path of an options file or as the name of a template.

Return Value

Nothing - if the symbol can't be opened, the method will cause an error.

Opens the given raster image file as a symbol in the part editor after closing any open symbol.

Parameters

OpenSymbolFromRasterImage(

- *FilePath* As String
- *DiscardChanges* As Boolean
- *OptionsFilePath* As String

)

FilePath
: The path of the raster image to open.

DiscardChanges
: Discard any changes made to the currently open symbol when opening the given raster image.

OptionsFilePath
: The raster-to-vector conversion options, given as the path of an options file.

Return Value

	Nothing - if the raster image can't be opened, the method will cause an error.
Quit As Boolean	Shuts down the application. If this method is called from a plug-in it must be the last statement to be executed. In addition, the plug-in or macro should dismiss all of its visible windows (dialogs, forms, etc) before calling Quit.

Return Value

Nothing.

Properties

Property	Description
ActiveDocument As Document	Gets the currently active Document object.
Application As Application	Gets the Application object itself.
DatPath As String	Gets the folder path to the software program's configuration files.
FullName As String	Gets the pathed file specification for the software program.
GUIState As RadGUIState	Gets the GUI state that the software is currently in.
GUISubState As RadGUISubState	Gets the GUI substate that the software is currently in.
Interactive As Boolean	Gets/sets whether the software program is interactive while running COM calls.
Language As RadLanguage	Gets the language in which the software is currently running.
Mac As Mac	Gets the Mac object that provides direct support for many MAC language functions and variables.
MacFiles As MacFiles	Gets the MacFiles collection object.
Name As String	Gets the name of the software program.
Parent As Application	Gets the Application object itself.
Path As String	Gets the folder path to the software program's exe file.
PluginManager As PluginManager	Gets the PluginManager object.
ProcessID As Long	Gets the process ID for this instance of the application.
SystemDataFiles As SystemDataFiles	Gets the SystemDataFiles collection.
SoftwareVersion As String	Gets the current software level.
Visible As Boolean	Gets/sets whether the software program is visible.

If the software program is set running from an external client, Microsoft Word for example, the application will not be visible by default but will be interactive while running COM calls.

A simple example (from an external client)

The following VBA code demonstrates a few of the concepts you need to understand in order to use both the API and VBA effectively.

```
Public Sub sample()
    On Error Resume Next

    Dim app As Radraft.Application
                    ' Try to connect to a running instance
    Set app = GetObject(, "Radraft.Application")

    If (Err) Then
        Err.Clear
                    ' Try to start up a new instance
        Set app = CreateObject("Radraft.Application")

        If (Err) Then
            MsgBox "Unable to connect to the software application"
            Exit Sub
        End If
                    ' Make the software application visible
        app.Visible = True
```

```
    End If
                    ' Display the software application's name
    MsgBox app.Name
End Sub
```

Explanation

This is a VBA macro. It is a Sub procedure with no arguments.

A variable of type **Radraft.Application** is declared. The "Radraft." part of this variable declaration indicates that the Application object we want is the one in the type library named "Radraft". Although it is not strictly necessary to specify this, doing so avoids conflicts with other libraries that also contain an Application object.

The call to GetObject() is an attempt to connect to a running instance of the software, setting a reference to it in the variable app.

If the software program is not already running, the call to GetObject() will fail. In that case, the call to CreateObject() attempts to start up the software program. If this attempt is successful, the software will now be running, but in a hidden state. By setting the Visible property to True, the software will become visible.

Finally, once the software is running visibly, the API can be used further. In this example, we simply show a message box to display the Name of the software program.

Note that this example is written for use from an external client. It uses the GetObject() and CreateObject() function calls to obtain a reference to an Application object. To achieve the same result from within this software program, the example can be greatly simplified. The top-level Application object is already available, making the GetObject() and CreateObject() function calls unnecessary. The app variable is redundant and the methods and properties of the object model should be accessed directly through the top-level Application object.

Document object

The Document object represents a drawing or a symbol, and, in fact, is actually either a DrawingDocument or a SymbolDocument object, both of which classes inherit from Document.

The base-class Document object has the following methods and properties:

Methods

Method	Description
Close(• *DiscardChanges* As Boolean)	Closes this document (closing a symbol document returns to the drawing document, closing a drawing document starts a new one). **Parameters** DiscardChanges Discard any changes made to the currently open document when closing it. **Return Value** Nothing.
Save()	Saves this document to the current folder and name using the native file format. **Return Value** Nothing - if the document can't be saved, it will cause an error.
	Saves this document to the given file path using the native file format (the document is renamed if different from the given file name).

SaveAs(

- *FilePath* As String

)

Parameters

FilePath
The path to save the document to.

Return Value

Nothing - if the document can't be saved, it will cause an error.

Saves a copy of this document to the given file path using any supported format (the document is not renamed if different from the given file name). Drawings can be saved as the native drawing file format or as DXF or IGES files. Symbols can be saved as the native symbol file format or as DXF files.

SaveCopyAs(

- *FilePath* As String
- *OptionsFilePath* As String

)

Parameters

FilePath
The path to save the document to.
OptionsFilePath
The export options for DXF files, given either as the path of an options file or as the name of a template.

Return Value

Nothing - if the document can't be saved, it will cause an error.

Properties

Property	Description
Application As Application	Gets the Application object.
Dirty As Boolean	Gets whether this document has been changed since it was last saved.
Parent As Application	Gets the parent object from which this object can be logically reached.
Type As RadDocumentType	Gets the document type.

Mac object

The Mac object provides direct support for many MAC language functions and variables. Mac object method names, method arguments, and property names are exactly the same as those used in the corresponding MAC language functions and variables.

To see a full alphabetical list of all Mac object properties and methods, look up 'Mac object' in the help index. Mac object properties and methods are also organised into the following categories:

Common API properties

Licensing

User interface

Keystroke commands and variables

Keystroke command syntax using VBA

Keystroke command syntax using C#

Scanning and finding 2D features

Editing 2D features

2D geometry analysis

2D geometry utilities

Attributes

2D export

3D import

3D export

3D unfolding

Scanning 3D entities

3D utilities

Automatic tooling

Part Editor tooling

Automatically adding embedded part removal

Single part nesting

Parts nesting

Nest projects

Finish nesting

Sheet scrapping and offcuts

Order mode

Block compiling

Program verification

Machine configuration

Tool library

Standard tool loading

Standard sheets

Utility methods

MAC language access

Mac object: Common API properties

The Mac object supports the following properties that are common to all API objects:

Common API properties

Property	Description
Application As Application	Gets the Application object.
Parent As Application	Gets the parent object from which this object can be logically reached.

Mac object: Licensing

The Mac object contains a number of methods for querying and requesting licences.

Licences are referred to by a string name.

Licensing methods

Mac object: User interface

The Mac object contains a small number of methods that allow you to directly control the main application user interface. Currently, these methods are limited to allowing you to set text in the Instruction bar and the Information bar (see The screen layout).

The Mac object also contains a number of methods for displaying open and save file browsers. These are the same browsers that are used in the main application user interface, and include thumbnail previews for drawings and symbols, and access to the Most Recently Used lists. The Mac object also contains methods for managing the Most Recently Used lists, and a number of properties that identify various folders used by the application (e.g. PATH_DAT).

User interface methods

Method	Description
lic_available(• ByVal *name* As String) As Boolean	Return true if the given licence is available. **Parameters** [in] name Licence name. **Return Value** Return true if the given licence is available.
lic_confirm(• ByVal *name* As String) As Boolean	Return true if the given licence is held by this client. **Parameters** [in] name Licence name. **Return Value** Return true if the given licence is held by this client.
lic_get_holder() As String	Return the licence holder. **Return Value** Return the licence holder.
lic_get_servercode() As String	Return the licence server code. **Return Value** Return the licence server code.
	Request the named licence.

lic_request(

- ByVal *name* As String

) As Boolean

Parameters

[in] name
Licence name.

Return Value

Return true for success, false for failure.

Method	Description
uim_error(• ByVal *message* As String) As Boolean	Display the given message in the UI error message area. **Parameters** [in] message Message to be displayed. **Return Value** Return true for success, false for failure.
uim_info(• ByVal *message* As String) As Boolean	Display the given message in the UI information message area. **Parameters** [in] message Message to be displayed. **Return Value** Return true for success, false for failure.
uim_prompt(• ByVal *message* As String) As Boolean	Display the given message in the UI prompt message area. **Parameters** [in] message Message to be displayed. **Return Value** Return true for success, false for failure.

File browser methods

Method	Description
fla_browse_for_file_to_open(• ByVal *allowMultiSelect* As	Display the Open File browser. **Parameters** [in] allowMultiSelect Allow multiple file selection. [in] title Dialog title.

Boolean
- ByVal *title* As String
- ByVal *filter* As String
- ByRef *filePath* As String

) As Boolean

[in] filter
Filter string. Filter strings are separated by |, and must be terminated by ||. For example, 'DXF Files (*.dxf)|*.dxf|IGES Files (*.igs)|*.igs;*.iges|All Files (*.*)|*.*||'.
[in,out] filePath
In: Initial path (or empty string). Out: Returned path or paths. If more than one file is returned, the return string is a | separated list of files. For example,'C:\tmp\junk.txt|C:\tmp\fred.txt|c:\tmp\foo.txt'.

Return Value

Return the number of files selected.

fla_browse_for_file_to_save(

- ByVal *reserved* As Boolean
- ByVal *title* As String
- ByVal *filter* As String
- ByRef *filePath* As String

) As Boolean

Display the Save File browser.

Parameters

[in] reserved
Reserved - set to false.
[in] title
Dialog title.
[in] filter
Filter string. Filter strings are separated by |, and must be terminated by ||. For example, 'DXF Files (*.dxf)|*.dxf|IGES Files (*.igs)|*.igs;*.iges|All Files (*.*)|*.*||'.
[in,out] filePath
In: Initial path (or empty string). Out: Returned path.

Return Value

Return the number of files selected.

Most Recently Used list methods

Method	Description
reg_most_recent_list_add(• ByVal *recentKey* As String • ByVal *value* As String) As Boolean	Add a new value to a Most Recently Used list.

Parameters

[in] recentKey
Recent list key (e.g. 'dxf').
[in] value
Value.

Return Value

Return true for success, false for failure.

reg_most_recent_list_get_at(

- ByVal *recentKey* As String
- ByVal *index* As Long

) As String

Get an entry from a Most Recently Used list.

Parameters

[in] recentKey
Recent list key (e.g. 'dxf').
[in] index
Index into recent list (0 based).

Return Value

Return the value, or an empty string on failure.
Get the number of entries in a Most Recently Used list with the given key.

reg_most_recent_list_get_count(

- ByVal *recentKey* As String

) As Long

Parameters

[in] recentKey
Recent list key (e.g. 'dxf').

Return Value

Return the number of entries in the list.
Convert a Most Recently Used list type to key string.

reg_most_recent_list_type_to_key(

- ByVal *type* As Long

) As String

Parameters

[in] type
Recent list type (e.g. RECENT_DRAWING_FILE, RECENT_SYMBOL_FILE, etc).

Return Value

Recent list key (e.g. 'dxf').

Application paths

Property	Description
PATH_ATTR_DEFN As String	Get: Attribute definition folder.
PATH_BIN As String	Get: bin folder.
PATH_DAT As String	Get: dat folder.
PATH_DTM As String	Get: dtm folder.
PATH_DXF As String	Get: dxf folder.
PATH_ETC As String	Get: etc folder.
PATH_GPPDATA As String	Get: gppdata folder.
PATH_GUI As String	Get: gui folder.
PATH_GVFDATA As String	Get: gvfdata folder.
PATH_ICON As String	Get: icon folder.
PATH_INC As String	Get: include folder.
PATH_MMG As String	Get: mmg folder.
PATH_MODEL As String	Get: model folder.
PATH_MSG As String	Get: msg folder.
PATH_PPP As String	Get: ppp folder.
PATH_RADAN As String	Get: RADAN folder.
PATH_RADATA As String	Get: radata folder.
PATH_SXF As String	Get: sxf folder.
PATH_SYM_DIR As String	Get: Symbol folder.
PATH_USER As String	Get: user folder.
TMP As String	Get: Folder for temporary files.

Most Recently Used list types

Property	Description
RECENT_DRAWING_FILE As Long	Get: Most Recently Used list type.
RECENT_DTM_FILE As Long	Get: Most Recently Used list type.
RECENT_FUNCTION_KEY_MACRO_NAME As Long	Get: Most Recently Used list type.
RECENT_POPUP_MACRO_NAME As Long	Get: Most Recently Used list type.
RECENT_SYMBOL_FILE As Long	Get: Most Recently Used list type.
RECENT_UNKNOWN As Long	Get: Most Recently Used list type.
RECENT_USER_KEY As Long	Get: Most Recently Used list type.

Mac object: Keystroke commands and variables

The Mac object contains a number of methods for executing keystroke commands and various other methods commonly used with these commands. A large number of properties are set after each keystroke command is executed. These properties contain information about the current state of the application, and about any found points and features.

For a full list of drafting and tooling mode keystroke commands, see Keystroke Reference. For Order mode keystroke commands, see Order mode keystroke reference. You can find many other useful help pages about keystroke commands by searching the help for 'keystrokes'.

The syntax of the keystroke command strings passed to fmac2(), mac2() and rfmac() is described in Keystroke command syntax using VBA and Keystroke command syntax using C#.

Keystroke command methods

Method	Description
fmac2(• ByVal *command* As String) As Long	End any current feature scan, and then run the given keystroke command. Does not update current state values. **Parameters** [in] command The keystroke command to execute. **Return Value** Return a positive value for success, zero for failure and negative if the command has initiated application termination (in which case, do not call any more API methods/properties).
mac2(• ByVal *command* As String) As Long	End any current feature scan, then run the given keystroke command, and then update the current state values (e.g. S1X, S1Y, etc). **Parameters** [in] command The keystroke command to execute. **Return Value** Return a positive value for success, zero for failure and negative if the command has initiated application termination (in which case, do not call any more API methods/properties).

Run the given keystroke command. Does not end current feature scan. Does not update current state values.

Note: You cannot use rfmac() if any of the called keystroke commands perform a scan of their own. The following list of keystroke commands do not currently do scans, and so they can safely be used with rfmac(): s c ~ d e x (but not for hatching or window delete) m (but not symbols or window move) P a l . f t 1 < > , "

rfmac(

- ByVal *command* As String

) As Long

Parameters

[in] command
The keystroke command to execute.

Return Value

Return a positive value for success, zero for failure and negative if the command has initiated application termination (in which case, do not call any more API methods/properties).

Switching between NC and drafting modes

Method	Description
silent_exit_nc_mode() As Long	Silently exit NC mode to drafting mode (e.g. so you can execute a drafting keystroke command). Should be matched with a call to silent_nc_mode(). Return zero for success or a non-zero error code.

Return Value

Return zero for success or a non-zero error code.

silent_nc_mode() As Long	Silently enter to NC mode from drafting mode (e.g. so you can execute a tooling keystroke command). Should be matched with a call to silent_exit_nc_mode(). Return zero for success or a non-zero error code.

Return Value

Return zero for success or a non-zero error code.

Managing the undo list

Method	Description
d_dump() As Boolean	Undoes the last frame of modifications back to the last d_fix(). Scraps any undo list entries after the resulting undo list position. To allow several calls to d_dump(), it also cancels the effect of the last d_fix().

Return Value

Return true for success, false for failure.

d_fix() As Boolean	Prepares to start a new undo list entry as soon as the next modification is made to the drawing/part.

Return Value

Return true for success, false for failure.

Keyboard and mouse input methods

Method	Description

Returns a mask containing the state of the keyboard modifier keys at the time of the last key or mouse button event. Bit 1: 0 - shift up, 1 - shift down. Bit 2: 0 - control up, 1 - control down.

get_modifier_state() As Long

Return Value

Returns a mask containing the state of the keyboard modifier keys at the time of the last key or mouse button event. Bit 1: 0 - shift up, 1 - shift down. Bit 2: 0 - control up, 1 - control down.

Wait for the user to press a key or mouse button in the graphics area, and then return the key value. Mode: 0 - rubber band, 1 - rubber window box, 2 - rubber band and preview latching (if enabled).

Note: The gin() function was designed to be run from macros executed from function keys or menus. It does not work when run from dialogs (because of various reasons to do with keyboard focus and keyboard events). If you want to run gin() from a dialog (e.g. from a dialog button command handler), then you must hide the dialog before you call gin() and then (if required) re-show the dialog again after gin() has returned.

gin(

- ByVal *mode* As Long

) As Long

Parameters

[in] mode
: Input mode: 0 - rubber band, 1 - rubber window box, 2 - rubber band and preview latching (if enabled).

Return Value

Return the key value.

Application environment

Property	Description
LANG As Long	Get: Current language in use, e.g. 0 - English, 2 - French, etc.
RADRAFT_ARGS As String	Get: Command line arguments.
UID As String	Get: User name.

Application mode

Property	Description
NCD As Boolean	Get: NC drafting mode.
ORD As Boolean	Get: Order mode.
RUNNING_UNLOADER As Boolean	Get: Unloader is running.
SALES_QUOTE_MODE As Boolean	Get: The system is in sales quote mode.
VFY As Boolean	Get: Verifier mode.
VFY_TYPE As Long	Get: Verifier type (0 - undefined, 1 - C verifier, 2 - generic verifier).

Drawing properties

Property	Description
C_DWG As String	Get: Name of current drawing.
C_DWGDIR As String	Get: Name of current drawing directory.
DRN As String	Get: Drawing name.
DRS As Double	Get: Drawing scale.

DRU As Double	Get: Drawing units.
DWG_SOURCE As Long	Get: Current drawing source (0 - new, 1 - security save, 2 - recalled).
PSI As Long	Get: Paper size number.
PSX As Double	Get: Paper size.
PSY As Double	Get: Paper size.

Drawing edit state

Property	Description
DWG_EDITED As Boolean	Get: Drawing has been edited.
DWG_REAL_EDITED_COUNT As Long	Get: Total number of edits made during the current session.
REDOS As Long	Get: Number of redos available.
UNDOS As Long	Get: Number of undos available.

Patterns

Property	Description
COP As String	Get: Currently open pattern.
COV As Long	Get: Currently open pattern visible (positive - visible, zero - not visible, negative - no pattern).
CPS As Double	Get: Currently open pattern scale.
CUP As String	Get: Current pattern path.
CUV As Long	Get: Current pattern visible (positive - visible, zero - not visible, negative - no pattern).
PS0 As Double	Get: Root pattern scale.
PZ1 As Double	Get: Z1 value of current pattern (CUP).
PZ2 As Double	Get: Z2 value of current pattern (CUP).

Reserved patterns

Property	Description
PART_PATTERN As String	Get: The reserved drawing pattern used for the Part Editor.
PCC_PATTERN_LAYOUT As String	Get: The reserved drawing pattern that contains the nest parts.
PCC_PATTERN_LAYOUT_OFFCUTS As String	Get: The reserved drawing pattern that contains nest offcut tooling.
PCC_PATTERN_LAYOUT_SCRAP As String	Get: The reserved drawing pattern that contains nest scrap tooling.
PCC_PATTERN_LAYOUT_SHEET As String	Get: The reserved drawing pattern that contains the nest sheet.
PCC_PATTERN_LAYOUT_TEXT As String	Get: The reserved drawing pattern that contains nest annotation text.

Cursor objects properties

Property	Description
COE As Boolean	Get: Cursor object is an ellipse.
COT As String	Get: Cursor Object type: + or - then one of: l - line, a - arc, p - pattern/window, s - symbol, t - text, ? - other.
XOR As Double	Get: Cursor object orientation.
XSC As Double	Get: Cursor object scale.
XSH As Double	Get: Cursor object shear.

Current keystroke mode

Property	Description
EDT As Boolean	Get: Edit mode.
HEL As Boolean	Get: Help mode.
PAT As Boolean	Get: Pattern mode.
PON As Boolean	Get: Line/arc profile select mode.
WIN As Boolean	Get: Window mode.

Location

Property	Description
CX As Double	Get: Current location.
CY As Double	Get: Current location.
SX As Double	Get: Start location.
SY As Double	Get: Start location.
UX As Double	Set/get: Current position.
UY As Double	Set/get: Current position.

Drafting aids

Property	Description
BON As Boolean	Get: Rubber band status.
DON As Boolean	Get: Datum status.
DX As Double	Get: Datum location (0,0 if no datum).
DY As Double	Get: Datum location (0,0 if no datum).
GON As Boolean	Get: Grid status.
RON As Long	Get: Snap type (0 - none, 1 - cartesian, 2 - polar).

Properties used when adding features

Property	Description
CET As Long	Get: Current line end terminator.
CETS As Double	Get: Current end terminator size.
CLT As Long	Get: Current line type.
CON As Boolean	Get: Dimension centralising on/off.
CST As Long	Get: Current line start terminator.
CSTS As Double	Get: Current start terminator size.

Keyboard state

Property	Description
KEY As Long	Get: Last key pressed - ASCII value.

Window state

Property	Description
UW As Long	Set/get: Current window number (1 - main, 2 - supplementary).
W1X As Double	Get: Current zoom window (bottom left corner).
W1Y As Double	Get: Current zoom window (bottom left corner).
W2X As Double	Get: Current zoom window (top right corner).
W2Y As Double	Get: Current zoom window (top right corner).
WPX As Long	Get: Window size in pixels.
WPY As Long	Get: Window size in pixels.
WS1X As Double	Get: Supplementary window: Current zoom window (bottom left corner).

WS1Y As Double	Get: Supplementary window: Current zoom window (bottom left corner).
WS2X As Double	Get: Supplementary window: Current zoom window (bottom left corner).
WS2Y As Double	Get: Supplementary window: Current zoom window (bottom left corner).
ZMF As Double	Get: Current zoom factor.
ZMS As Double	Get: Supplementary window: Current zoom factor.

Graphics window prompt

Property	Description
MES As Boolean	Set/get: Prompt area message visibility.
PRC As Long	Get: Prompt row/column position.
PRR As Long	Get: Prompt row/column position.
PRS As String	Set/get: Prompt string.

Mac language macros

Property	Description
M_ARGMAC As String	Get: Command line Mac language macro.
M_EXIT As String	Get: Current exit macro name.
M_FUN As String	Get: Current function key macro name.
M_POP As String	Get: Current popup-menu macro name.
M_START As String	Get: Start-up Mac language macro.

Current machine

Property	Description
MC_TRUE_TYPE As Long	Get: True current machine type (PCC_PUNCHING, PCC_COMBINATION, PCC_LASER, PCC_NONE).
MC_TYPE As Long	Get: Current machine type (PCC_PUNCHING, PCC_COMBINATION, PCC_LASER, PCC_NONE).

Machine types

Property	Description
PCC_COMBINATION As Long	Get: Combination machine type.
PCC_LASER As Long	Get: Laser machine type.
PCC_NONE As Long	Get: Undefined machine type.
PCC_PUNCHING As Long	Get: Punch machine type.

Mac object: Keystroke command syntax using VBA

Getting the Mac object

The Mac object is a property of the Application object. In VBA macros and plug-ins, the Mac object can be obtained as follows:

```
Dim mac As Radraft.mac
Set mac = Radraft.mac
```

To get the Application object in an external client, see Application object example.

Executing a keystroke command

The methods for executing keystroke commands (mac2(), fmac2() and rfmac()) all take a single command argument. The command argument is a string containing the keystroke command. For example:

```
' Find the nearest feature to the current cursor position.
mac.mac2 "f"
```

```
' Delete the last found feature.
mac.mac2 "x"
```

Multiple keystroke commands

The command argument can contain multiple keystroke commands. For example:

```
' Find the nearest feature to the current cursor position and delete it.
mac.mac2 "fx"
```

Using multiple commands can be slightly more efficient. However, error checking becomes more difficult (as you cannot check the success of each command), and it becomes more difficult to read your code.

Keystroke command arguments

Some keystroke commands require arguments. Keystroke commands that take arguments must be surrounded by **\?** and **?**. Arguments are separated by a comma. For example:

```
' Add a fillet (keystroke 5), with radius 1.5, between the last two found features.
mac.mac2 "\?5,1.5?"
```

Variable arguments

It is possible to use the names of MAC variables for keystroke command arguments. To set and get MAC variables, you need to use SetString(), SetNumber(), GetString() and GetNumber(). For example:

```
Dim filletRadius As double
filletRadius = 20.5

' Create a MAC variable called 'radius' and set its value.
mac.SetNumber "radius", filletRadius

' Add a fillet between the last two found lines using the given radius.
mac.mac2 "\?5,radius?"
```

Note: Using variables (with meaningful names) makes it much easier to understand keystroke command strings.

Text arguments

Some keystroke commands require text arguments. Text arguments must be surrounded by **""**. For example:

```
' Set cursor object text to: My text
mac.mac2 "\?T,""My text""?"
```

If you want to include a double-quote within the text, you must use **\""**. For example:

```
' Set cursor object text to: My text with "double-quote" characters
mac.mac2 "\?T,""My text with \""double-quote\""characters""?"
```

If you want to include a backslash character within the text, you must use \\\\. For example:

```
' Set cursor object text to: Save in C:\Documents
mac.mac2 "\?T,""Save in C:\\\\Documents""?"
```

Note: Not all application fonts support the backslash character.

Your code will usually be easier to read if you use MAC variables for text keystroke command arguments. For example:

```
' Set cursor object text to: Double-quote " and backslash \ characters
Dim myText As String
myText = "Double-quote "" and backslash \\ characters"
mac.SetString "text", myText
mac.mac2 "\?T,text?"
```

The draw rectangle command

The keystroke command used to draw a rectangle is normally just one double-quote, i.e. **"**. However, in VBA you must use two double-quotes, i.e. **""**. For example:

```
' Set the position of the first corner
mac.UX = 100
mac.UY = 200
mac.mac2 "s"

' Set the position of the opposite corner
mac.UX = mac.UX + 300
mac.UY = mac.UY + 400

' Draw the rectangle
mac.mac2 """"
```

Default arguments using "def"

Keystroke commands allow you to use default arguments. The default value for an argument is usually the previously used value for that argument. To indicate that you want to use a default value for an argument, you must use **""def""**. For example:

```
' Add a fillet, with the default radius, between the last two found features.
mac.mac2 "\?5,""def""?"
```

User supplied arguments using "usr"

Keystroke commands allow you to prompt the user to enter a value in the graphics area. To indicate that you want the user to enter a value for an argument, you must use **""usr""**. For example:

```
' Prompt the user for the fillet radius, and then add a fillet between the last two found features.
mac.mac2 "\?5,""usr""?"
```

Note: If the user presses the Esc key when prompted for a value, the command will be cancelled and will return false.

Using the Ctrl key with keystroke commands

Some keystroke commands use the Ctrl key together with another key, e.g. Ctrl+q is used to display the current drawing details (name, etc). In a keystroke command argument string, use \ to represent the Ctrl key. For example:

```
' Display drawing name etc in the graphics area.
mac.mac2 "\q"

' Draw only the features drawn in pen 2.
mac.mac2 "\?\r,2?"
```

Note: Ctrl keystroke command keys are case insensitive. For example, \q is the same as \Q.

Using the Esc key with keystroke commands

The Esc keystroke command is used for cancelling the cursor object and for cancelling the current keystroke mode (e.g. pattern mode). In a keystroke command argument string, use **\!** to represent the Esc key. For example:

```
' Cancel the cursor object.
mac.mac2 "\!"
```

Setting the cursor position using UX and UY

Many keystroke commands need to use the current cursor position (e.g. the find feature command 'f'). You can set the position of the cursor that will be used by the keystroke commands, by setting **UX** and **UY**. For example:

```
' Find the feature closest to (200,300)
mac.UX = 200
mac.UY = 300
mac.mac2 "f"
```

If you do not set UX and UY, then they will automatically be set to the current real cursor position.

Controlling keystroke command messages using MES and PRS

Many keystroke commands display messages in the graphics area (e.g. a failure message). To stop these messages being shown, you can set the **MES** flag to false. The text for these messages is stored in the **PRS** string (even when MES is false), which you may want to record or display in some other way, e.g. in a message box. For example:

```
' Clear command message.
mac.PRS = ""

' Remember the original value of MES.
Dim oldMes As Boolean
oldMes = mac.MES

' Switch messages off.
mac.MES = False

' Find 2 features.
mac.mac2 "F"

' Restore MES to original value.
mac.MES = oldMes

' Display message (if any) from the last keystroke command in message box.
If Not mac.PRS = "" Then
    MsgBox mac.PRS
End If
```

User data input in the graphics area

You can prompt the user to enter a value in the graphics area using the keystroke command functions. The value will be stored in a MAC variable. To do this, surround a prompt and variable with **\:** and **:**. The prompt also has to be surrounded by **""**. To set and get MAC variables, you need to use SetString(), SetNumber(), GetString() and GetNumber(). For example:

```
Dim value As String
value = "old string"
mac.SetString "stringVar", value
mac.mac2 "\:""Enter string"",stringVar:"
value = mac.GetString("stringVar")
```

User key input in the graphics area

You can use the keystroke command functions to prompt the user to enter a single key or mouse button press in the graphics area. To do this, surround your prompt text with **\""** and **""**. For example:

```
mac.mac2 "\""Press any key to continue"""
```

Once the prompt is displayed, the system will wait for the user to press a key or mouse button in the graphics area before continuing.

Note: Often it is more useful to use gin() to get key and mouse button press events from the user.

Keeping keystrokes synchronised with the current GUI state

Generally, the keystroke commands are divided into three groups, i.e. Drafting, Tooling and Order. So, for example, to execute a Tooling keystroke command, you need to be in the keystroke tooling mode. Once your code has finished executing keystroke commands, it is recommended that you reset the keystroke mode so that it is synchronised with the current GUI state. If this is not done, then the user interface may not work as expected. One way of doing this is shown in the following example:

```
            ' Note: Keystroke commands cannot be executed in 3D mode
If Not Radraft.GUIState = Radraft.rad3D Then

            ' If we are executing Order mode keystroke commands, but the
            ' GUI is not in Order mode, switch out of this keystroke mode
    If mac.ORD And Not Radraft.GUISubState = Radraft.radOrder Then
        mac.mac2 "O"
    End If

            ' If we are executing Tooling mode keystroke commands, but the
```

```
            ' GUI is not in Tooling mode, switch out of this keystroke mode
    If mac.NCD And Not Radraft.GUISubState = Radraft.radTooling Then
        mac.mac2 "N"
    End If
End If
```

Pattern keystroke command example

The following example shows how to use various pattern related keystroke commands. The syntax for these commands can sometimes be quite complicated, so it is useful to have some examples.

```
'
' Check whether the given pattern exists.
'
Public Function PatternExists(pattern As String) As Boolean
    Dim mac As Radraft.mac
    Set mac = Application.mac

                ' Update all MAC variables by executing an empty command
    mac.mac2 ""

                ' Record the currently open pattern
    Dim oldOpenPattern As String
    oldOpenPattern = mac.COP

                ' Try to open the pattern we want to check. If the
                ' pattern can be opened, then it exists. Patterns
                ' that do not exist cannot be opened.
    mac.SetString "stringVal", pattern
    PatternExists = mac.mac2("\?\P,stringVal?o")

                ' Re-open the original open pattern.
    mac.SetString "stringVal", oldOpenPattern
    mac.mac2 "\?\P,stringVal?o"
End Function

'
' Open the given pattern. It is assumed that the pattern already exists.
'
Public Function PatternOpen(pattern As String) As Boolean
    Dim mac As Radraft.mac
    Set mac = Application.mac

                ' Note: You can only open a pattern if it exists
    If PatternExists(pattern) Then
        mac.SetString "stringVal", pattern
        PatternOpen = mac.mac2("\?\P,stringVal?o")
    Else
        PatternOpen = False
    End If
End Function

'
' Delete the given pattern.
'
Public Function PatternDelete(pattern As String) As Boolean
    Dim mac As Radraft.mac
    Set mac = Application.mac

                ' Note: You can only delete a pattern if it exists
    If PatternExists(pattern) Then
        mac.SetString "stringVal", pattern
        PatternDelete = mac.mac2("\?\P,stringVal?x")
    Else
        PatternDelete = False
    End If
End Function

'
' Create the given pattern. If the pattern already exists, it will re-created.
'
Public Function PatternNew(pattern As String) As Boolean
    Dim mac As Radraft.mac
```

```
    Set mac = Application.mac

                ' If the pattern already exists, delete it
                ' so that we can re-create a new pattern.
    If PatternExists(pattern) Then
        PatternDelete pattern
    End If

                ' Create the pattern. This also makes the
                ' pattern the currently open pattern.
    mac.SetString "stringVal", pattern
    PatternNew = mac.mac2("\?\P,stringVal,""def""?")
End Function

'
' Set the visibility of the given pattern.
'
Public Function PatternSetVisible(pattern As String, visible As Boolean) As Boolean
    Dim mac As Radraft.mac
    Set mac = Application.mac

                ' Note: You can only change the pattern visibility if it exists
    If PatternExists(pattern) Then
        mac.SetString "stringVal", pattern
        If visible Then
            mac.SetNumber "numVal", 1
        Else
            mac.SetNumber "numVal", 0
        End If
        PatternSetVisible = mac.mac2("\?\P,stringVal?\?v,numVal?")
    Else
        PatternSetVisible = False
    End If
End Function

'
' An example that calls various pattern functions.
'
Public Sub PatternExample()

                ' Create a new pattern under the root pattern
    PatternNew "/first"

                ' Create two sub-patterns
    PatternNew "/first/left"
    PatternNew "/first/right"

                ' Re-open the first pattern
    PatternOpen "/first"

                ' Delete one of the sub-patterns
    PatternDelete "/first/left"

                ' Make the other sub-pattern invisible
    PatternSetVisible "/first/right", False
End Sub
```

Mac object: Keystroke command syntax using C#

Getting the Mac object

The Mac object is a property of the Application object. In .NET language plug-ins, a handle to the Application object is provided to your OnConnectToApplication() method (see Creating .NET plug-ins). Assuming you have stored the Application handle as member data (e.g. as 'm_app'), then the Mac object can be obtained as follows:

```
Radraft.Interop.Mac mac = m_app.Mac;
```

Executing a keystroke command

The methods for executing keystroke commands (mac2(), fmac2() and rfmac()) all take a single command argument. The command argument is a string containing the keystroke command. For example:

```
// Find the nearest feature to the current cursor position.
mac.mac2("f");

// Delete the last found feature.
mac.mac2("x");
```

Multiple keystroke commands

The command argument can contain multiple keystroke commands. For example:

```
// Find the nearest feature to the current cursor position and delete it.
mac.mac2("fx");
```

Using multiple commands can be slightly more efficient. However, error checking becomes more difficult (as you cannot check the success of each command), and it becomes more difficult to read your code.

Keystroke command arguments

Some keystroke commands require arguments. Keystroke commands that take arguments must be surrounded by **\\?** and **?**. Arguments are separated by a comma. For example:

```
// Add a fillet (keystroke 5), with radius 1.5, between the last two found features.
mac.mac2("\\?5,1.5?");
```

Variable arguments

It is possible to use the names of MAC variables for keystroke command arguments. To set and get MAC variables, you need to use SetString(), SetNumber(), GetString() and GetNumber(). For example:

```
double filletRadius = 20.5;

// Create a MAC variable called 'radius' and set its value.
mac.SetNumber("radius", filletRadius);

// Add a fillet between the last two found lines using the given radius.
mac.mac2("\\?5,radius?");
```

Note: Using variables (with meaningful names) makes it much easier to understand keystroke command strings.

Text arguments

Some keystroke commands require text arguments. Text arguments must be surrounded by **\"**. For example:

```
// Set cursor object text to: My text
mac.mac2("\\?T,\"My text\"?");
```

If you want to include a double-quote within the text, you must use **\\\"**. For example:

```
// Set cursor object text to: My text with "double-quote" characters
mac.mac2("\\?T,\"My text with \\\"double-quote\\\" characters\"?");
```

If you want to include a backslash character within the text, you must use ****. For example:

```
// Set cursor object text to: Save in C:\Documents
mac.mac2("\\?T,\"Save in C:\\\\\\\\temp\"?");
```

Note: Not all application fonts support the backslash character.

Your code will usually be easier to read if you use MAC variables for text keystroke command arguments. For example:

```
// Set cursor object text to: Double-quote " and backslash \ characters
String myText = "Double-quote \" and backslash \\\\ characters";
mac.SetString("text", myText);
mac.mac2("\\?T,text?");
```

The draw rectangle command

The keystroke command used to draw a rectangle is normally just one double-quote, i.e. **"**. However, in C# you must use **\"**. For example:

```
// Set the position of the first corner
mac.UX = 100;
mac.UY = 200;
mac.mac2("s");

// Set the position of the opposite corner
mac.UX = mac.UX + 300;
mac.UY = mac.UY + 400;

// Draw the rectangle
mac.mac2("\"");
```

Default arguments using "def"

Keystroke commands allow you to use default arguments. The default value for an argument is usually the previously used value for that argument. To indicate that you want to use a default value for an argument, you must use **\"def\"**. For example:

```
// Add a fillet, with the default radius, between the last two found features.
mac.mac2("\\?5,\"def\"?");
```

User supplied arguments using "usr"

Keystroke commands allow you to prompt the user to enter a value in the graphics area. To indicate that you want the user to enter a value for an argument, you must use **\"usr\"**. For example:

```
// Prompt the user for the fillet radius, and then add a fillet between the last two found features.
mac.mac2("\\?5,\"usr\"?");
```

Note: If the user presses the Esc key when prompted for a value, the command will be cancelled and will return false.

Using the Ctrl key with keystroke commands

Some keystroke commands use the Ctrl key together with another key, e.g. Ctrl+q is used to display the current drawing details (name, etc). In a keystroke command argument string, use \\ to represent the Ctrl key. For example:

```
// Display drawing name etc in the graphics area.
mac.mac2("\\q");

// Draw only the features drawn in pen 2.
mac.mac2("\\?\\r,2?");
```

Note: Ctrl keystroke command keys are case insensitive. For example, \\q is the same as \\Q.

Using the Esc key with keystroke commands

The Esc keystroke command is used for cancelling the cursor object and for cancelling the current keystroke mode (e.g. pattern mode). In a keystroke command argument string, use **\\!** to represent the Esc key. For example:

```
// Cancel the cursor object.
mac.mac2("\\!");
```

Setting the cursor position using UX and UY

Many keystroke commands need to use the current cursor position (e.g. the find feature command 'f'). You can set the position of the cursor that will be used by the keystroke commands, by setting **UX** and **UY**. For example:

```
// Find the feature closest to (200,300)
mac.UX = 200;
mac.UY = 300;
mac.mac2("f");
```

If you do not set UX and UY, then they will automatically be set to the current real cursor position.

Controlling keystroke command messages using MES and PRS

Many keystroke commands display messages in the graphics area (e.g. a failure message). To stop these messages being shown, you can set the **MES** flag to false. The text for these messages is stored in the **PRS** string (even when MES is false), which you may want to record or display in some other way, e.g. in a message box. For example:

```
// Clear command message.
mac.PRS = "";

// Remember the original value of MES.
bool oldMes = mac.MES;

// Switch messages off.
mac.MES = false;

// Find 2 features.
mac.mac2("F")

// Restore MES to original value.
mac.MES = oldMes;

// Display message (if any) from the last keystroke command in message box.
if (!string.IsNullOrEmpty(mac.PRS))
    MessageBox.Show(mac.PRS);
```

User data input in the graphics area

You can prompt the user to enter a value in the graphics area using the keystroke command functions. The value will be stored in a MAC variable. To do this, surround a prompt and variable with **\\:** and **:**. The prompt also has to be surrounded by **\"**. To set and get MAC variables, you need to use SetString(), SetNumber(), GetString() and GetNumber(). For example:

```
String value = "old string";
mac.SetString("stringVar", value);
mac.mac2("\\:\"Enter string\",stringVar:");
value = mac.GetString("stringVar");
```

User key input in the graphics area

You can use the keystroke command functions to prompt the user to enter a single key or mouse button press in the graphics area. To do this, surround your prompt text with **\\\"** and **\"**. For example:

```
mac.mac2("\\\"Press any key to continue\"");
```

Once the prompt is displayed, the system will wait for the user to press a key or mouse button in the graphics area before continuing.

Note: Often it is more useful to use gin() to get key and mouse button press events from the user.

Keeping keystrokes synchronised with the current GUI state

Generally, the keystroke commands are divided into three groups, i.e. Drafting, Tooling and Order. So, for example, to execute a Tooling keystroke command, you need to be in the keystroke tooling mode. Once your code has finished executing keystroke commands, it is recommended that you reset the keystroke mode so that it is synchronised with the current GUI state. If this is not done, then the user interface may not work as expected. One way of doing this is shown in the following example:

```
// Note: Keystroke commands cannot be executed in 3D mode
if (m_app.GUIState != Radraft.Interop.RadGUIState.rad3D)
{
    // If we are executing keystroke Order mode commands, but the
    // GUI is not in Order mode, switch out of this keystroke mode
    if (mac.ORD && m_app.GUISubState != Radraft.Interop.RadGUISubState.radOrder)
        mac.mac2("O");

    // If we are executing keystroke Tooling mode commands, but the
    // GUI is not in Tooling mode, switch out of this keystroke mode
    if (mac.NCD && m_app.GUISubState != Radraft.Interop.RadGUISubState.radTooling)
        mac.mac2("N");
}
```

Pattern keystroke command example

The following example shows how to use various pattern related keystroke commands. The syntax for these commands can sometimes be quite complicated, so it is useful to have some examples.

```
//
// Check whether the given pattern exists.
//
private bool PatternExists(string pattern)
{
    Radraft.Interop.IMac mac = m_app.Mac;

    // Update all MAC variables by executing an empty command
    mac.mac2("");

    // Record the currently open pattern
    string oldOpenPattern = mac.COP;

    // Try to open the pattern we want to check. If the
    // pattern can be opened, then it exists. Patterns
    // that do not exist cannot be opened.
    mac.SetString("stringVal", pattern);
    int result = mac.mac2("\\?\\P,stringVal?o");

    // Re-open the original open pattern.
    mac.SetString("stringVal", oldOpenPattern);
    mac.mac2("\\?\\P,stringVal?o");

    return result != 0;
}

//
// Open the given pattern. It is assumed that the pattern already exists.
//
private bool PatternOpen(string pattern)
{
    Radraft.Interop.IMac mac = m_app.Mac;

    // Note: You can only open a pattern if it exists
    int result = 0;
    if (PatternExists(pattern))
    {
        mac.SetString("stringVal", pattern);
        result = mac.mac2("\\?\\P,stringVal?o");
    }

    return result != 0;
}

//
// Delete the given pattern.
//
private bool PatternDelete(string pattern)
{
    Radraft.Interop.IMac mac = m_app.Mac;

    // Note: You can only delete a pattern if it exists
    int result = 0;
    if (PatternExists(pattern))
    {
        mac.SetString("stringVal", pattern);
        result = mac.mac2("\\?\\P,stringVal?x");
    }

    return result != 0;
}

//
// Create the given pattern. If the pattern already exists, it will re-created.
//
private bool PatternNew(string pattern)
{
    Radraft.Interop.IMac mac = m_app.Mac;
```

```
    // If the pattern already exists, delete it
    // so that we can re-create a new pattern.
    if (PatternExists(pattern))
    {
        PatternDelete(pattern);
    }

    // Create the pattern. This also makes the
    // pattern the currently open pattern.
    mac.SetString("stringVal", pattern);
    int result = mac.mac2("\\?\\P,stringVal,\"def\"?");

    return result != 0;
}

//
// Set the visibility of the given pattern.
//
private bool PatternSetVisible(string pattern, bool visible)
{
    Radraft.Interop.IMac mac = m_app.Mac;

    // Note: You can only change the pattern visibility if it exists
    int result = 0;
    if (PatternExists(pattern))
    {
        mac.SetString("stringVal", pattern);
        if (visible)
            mac.SetNumber("numVal", 1);
        else
            mac.SetNumber("numVal", 0);

        result = mac.mac2("\\?\\P,stringVal?\\?v,numVal?");
    }

    return result != 0;
}

//
// An example that calls various pattern functions.
//
private void OnPatternExample()
{

    // Create a new pattern under the root pattern
    PatternNew("/first");

    // Create two sub-patterns
    PatternNew("/first/left");
    PatternNew("/first/right");

    // Re-open the first pattern
    PatternOpen("/first");

    // Delete one of the sub-patterns
    PatternDelete("/first/left");

    // Make the other sub-pattern invisible
    PatternSetVisible("/first/right", false);
}
```

Mac object: Scanning and finding 2D features

The Mac object contains a number of methods for scanning and finding 2D drawing features, and various other related methods. A large number of properties are set during a scan and when a feature is found. These properties contain information about the the scanned feature, the last and last but one found feature, and about found points on features.

To use the scan functions effectively, you need to understand the concept of patterns. For details about patterns, see Introduction to patterns. Useful information relating to finding points and features can be found in Using locations in drafting, Finding and selecting features and Using keystrokes to find features.

Scanning for features

Method	Description
	Set the scan values (e.g. S0X, S0Y, etc) with details of the last added feature.
added_to_scan() As Boolean	**Return Value** Return true for success, false for failure.
	End the current feature scan early (i.e. without going to the end of the scan).
end_scan() As Boolean	**Return Value** Return true for success, false for failure.
	Get the next feature in the current feature scan. Update all of the scan values (e.g. S0X, S0Y, etc).
next() As Long	**Return Value** The number of the feature in the scan (0 - failure, 1 - first feature, 2 - second feature, etc).
	Ignore the remaining features in the currently scanned pattern or symbol.
rewind() As Boolean	**Return Value** Return true for success, false for failure.
	Start a feature scan. filter: Any features from 'alchHsdtpSCRg', plus optionally, x - all instances in step and repeat symbols, i - include invisible patterns, D - drafting features, N - drafting and tooling features, O - drafting, tooling and order features.
scan(• ByVal *path* As String • ByVal *filter* As String • ByVal *number* As Long) As Boolean	**Parameters** [in] path Starting path. [in] filter Filter of features to scan: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout, g - general feature. Additional filters: x - all instances in step and repeat symbols, i - include invisible patterns, D - drafting mode features, N - drafting and tooling mode features, O - drafting, tooling and order mode features. [in] number Feature number to start from. 0 to start from first feature.
	Return Value Return true for success, false for failure.
	Set the symbol level for feature scans.
scan_level(• ByVal *level* As Long) As Long	**Parameters** [in] level The depth of scans into symbols.
	Return Value

The previously set symbol scan level.

Finding points and features

Method	Description
find() As Boolean	Make the current feature in the current feature scan the 'last found' feature. Update the 'last found' and 'last but one' found values (e.g. S1X, S2X, etc). # Return Value Return true for success, false for failure.
find2(• ByVal *x* As Double • ByVal *y* As Double) As Boolean	Make the current feature in the current feature scan the 'last found' feature and set the 'found point'. Update the 'last found' and 'last but one' found values (e.g. S1X, S2X, etc). # Parameters [in] x Found point X. [in] y Found point Y. # Return Value Return true for success, false for failure.
find_identifier(• ByVal *identifier* As String • ByVal *point* As String • ByVal *region* As String) As Boolean	Find the feature indicated by the identifier. Update the 'last found' and 'last but one' found values (e.g. S1X, S2X, etc). # Parameters [in] identifier Identifier to find. [in] point Point on feature (s - start, e - end, t - top, b - bottom, l - left, r - right, m - middle, c - centre, d - datum). [in] region Region code (empty - all, a - above, b - below, l - left, r - right, i - inside, o - outside). # Return Value Return true for success, false for failure.
find_xy_identifier(• ByVal *identifier* As String • ByVal *x* As Double • ByVal *y* As Double) As Boolean	Find the identified feature and use the given coordinates to determine the found point on the feature. Update the 'last found' and 'last but one' found values (e.g. S1X, S2X, etc). # Parameters [in] identifier Identifier to find. [in] x Find the given feature as though the cursor was at this X. [in] y Find the given feature as though the cursor was at this Y.

Return Value

Return true for success, false for failure.

Set up the drafting or tooling mode feature find mask (any combination of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout).

fla_feature_mask(

- ByVal *setToolingModeMask* As Boolean
- ByVal *mask* As String

) As Boolean

Parameters

[in] setToolingModeMask
: True - set up mask for tooling mode. False - set up mask for drafting mode.

[in] mask
: Feature mask. Any combination of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout.

Return Value

Return true for success, false for failure.

Allow single feature keystroke query to use the filter set up using fla_feature_mask (use: 0 - use default filter, 1 - use fla_feature_mask with restricted searches, 2 - use fla_feature_mask with unrestricted searches).

fla_filter_f(

- ByVal *use* As Long

) As Boolean

Parameters

[in] use
: Use option: 0 - use default filter, 1 - use fla_feature_mask with restricted searches, 2 - use fla_feature_mask with unrestricted searches.

Return Value

Return true for success, false for failure.

Return the current latch point mask (any of: p - geometry points, b - bisections, i - intersections, f - feature points, c - points on imaginary horizontal/vertical lines, a - points on imaginary 30/45/60 degree angle lines, t - annotation points).

fla_get_latch_mask() As String

Return Value

Return the current latch point mask (any combination of: p - standard points on geometry, b - bisections, i - intersections, f - general points on features, c - points on imaginary cartesian (horizontal and vertical) guide lines, a - points on imaginary 30, 45 and 60 degree angle lines, t - standard points on annotation).

Jump the region containing the given point onto the cursor.

fla_jump_region(

- ByVal *x* As Double
- ByVal *y* As Double
- ByVal *anchorX* As Double
- ByVal *anchorY* As Double

) As Boolean

Parameters

[in] x
: Selected point's X coordinate.

[in] y
: Selected point's Y coordinate.

[in] anchorX
: X coordinate of the point to anchor to the cursor.

[in] anchorY
: Y coordinate of the point to anchor to the cursor.

Return Value

Returns true for success, false for failure.

Return the current smart select feature find mask (any combination of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout).

get_latch_feature_mask() As String

Return Value

Return the current smart select feature find mask (any combination of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout).

Retrieves information on any currently smart selected feature. Also sets: CI, CW, CD. Returns: 0 (failure), LATCH_NONE, LATCH_POINT, LATCH_FEATURE.

get_latched(

- ByVal *reserved* As Double
- ByRef *featureId* As String
- ByRef *x* As Double
- ByRef *y* As Double

) As Long

Parameters

[in] reserved
: Reserved - set to 0.

[out] featureId
: Feature id of selected feature.

[out] x
: Selected point X.

[out] y
: Selected point Y.

Return Value

Returns: 0 (failure), LATCH_NONE, LATCH_POINT, LATCH_FEATURE.

Constrain smart select (also known as preview latching) to the horizontal or vertical (-1 - no constraint, 0 - horizontal, 1 - vertical). Return the previous constraint.

latch_constrain(

- ByVal *constrain* As Long

) As Long

Parameters

[in] constrain
: Contrain type (-1 - no constraint, 0 - horizontal, 1 - vertical).

Return Value

Return the previous constraint.

Set up the smart select feature find mask (any of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout). Return previous mask.

latch_feature_mask(

- ByVal *mask* As String

) As String

Parameters

[in] mask
: Feature mask. Any combination of: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout.

Return Value

Return the previous mask.

Set up latch point mask: p - geometry points, b - bisections, i - intersections, f - feature points, c - points on imaginary horizontal/vertical lines, a - points on imaginary 30/45/60 degree angle lines, t - annotation points. Return previous mask.

latch_mask(

- ByVal *mask* As String

) As String

Parameters

[in] mask
: Point mask. Any combination of: p - standard points on geometry, b - bisections, i - intersections, f - general points on features, c - points on imaginary cartesian (horizontal and vertical) guide lines, a - points on imaginary 30, 45 and 60 degree angle lines, t - standard points on annotation.

Return Value

Return the previous mask.

Set the latch point radius (as a percentage of the graphics window width). Return the previous radius.

latch_radius(

- ByVal *radius* As Double

) As Double

Parameters

[in] radius
: Latch point radius (as a percentage of the graphics window width).

Return Value

Return the previous radius.

Set the current 'found point'.

pnt(

- ByVal *x* As Double
- ByVal *y* As Double

) As Boolean

Parameters

[in] x
: Found point X.

[in] y
: Found point Y.

Return Value

Return true for success, false for failure.

Set the current 'found point' on the feature with the given identifier at the given point (s - start, e - end, t - top, b - bottom, l - left, r - right, m - middle, c - centre, d - datum).

pnt_identifier(

- ByVal *identifier* As String
- ByVal *point* As String

) As Boolean

Parameters

[in] identifier
: Feature identifier.

[in] point
: Point on feature (s - start, e - end, t - top, b - bottom, l - left, r - right, m - middle, c - centre, d - datum).

Return Value

Return true for success, false for failure.

preview_latch_enable(• ByVal *enable* As Boolean) As Boolean	Enable/disable smart select (also known as preview latching). Return the previous enabled state. **Parameters** [in] enable True - enable smart select. False - disable smart select. **Return Value** Return the previous enabled state.

Managing general features

Method	Description
gadd(• ByVal *type* As Long • ByVal *data* As String) As Boolean	Add a general feature of the given type. **Parameters** [in] type General feature type. [in] data General feature data. **Return Value** Return true for success, false for failure.
gdel() As Boolean	Delete the currently scanned general feature. **Return Value** Return true for success, false for failure.
grep(• ByVal *type* As Long • ByVal *data* As String) As Boolean	Replace the currently scanned general feature with one of the given type. **Parameters** [in] type General feature type. [in] data General feature data. **Return Value** Return true for success, false for failure.

Utilities

Method	Description
isa_datum(	Return true if the given name or symbol path is for a datum symbol (e.g. odatum, mdatum, ldatum, etc). **Parameters**

- ByVal *path* As String

) As Boolean

[in] path
Pattern of symbol (or just the symbol name).

Return Value

Return true if the given name or symbol path is for a datum symbol.

Return true if the supplied path is a tool symbol name or a path to a tool symbol instance, otherwise return false. Multi-tools are always ignored (i.e. return false if the supplied path is for a multi-tool symbol). See also mtm_isa_tool_sym().

isa_tool_sym(

- ByVal *path* As String

) As Boolean

Parameters

[in] path
Pattern of symbol (or just the symbol name).

Return Value

Return true if given pattern path is a path to a tool symbol.

Calculate the bounds around the features specified by the given filter in the given path. Return zero for success or a non-zero error code.

Parameters

pattern_bounds2(

- ByVal *path* As String
- ByVal *filter* As String
- ByRef *xMin* As Double
- ByRef *yMin* As Double
- ByRef *xMax* As Double
- ByRef *yMax* As Double
- ByVal *descend* As Boolean
- ByVal *fullLinesOnly* As Boolean
- ByVal *pensToIgnore* As String

) As Long

[in] path
Pattern path to scan.
[in] filter
Filter of features to scan: a - arc, l - line, c - centre-line-cross, h - hatching, H - smart hatching, s - symbol, d - dimension, t - text, p - pattern, S - linear slot, C - arc slot, R - rectangular cutout. Additional filters: x - all instances in step and repeat symbols, i - include invisible patterns, D - drafting mode features, N - drafting and tooling mode features, O - drafting, tooling and order mode features.
[out] xMin
Min extent in X.
[out] yMin
Min extent in Y.
[out] xMax
Max extent in X.
[out] yMax
Max extent in Y.
[in] descend
Descend into sub-patterns.
[in] fullLinesOnly
Ignore non-full line types.
[in] pensToIgnore
Ignore features in these pens (comma separated list of pen numbers).

Return Value

Return zero for success or a non-zero error code.

Feature properties (0 - scanned feature, 1 - last found feature, 2 - last but one found feature)

Property	Description
AR0, AR1, AR2 As String	Get: Arc/ellipse/symbol rotation (", 'CW', 'CCW', 'RCW' or 'RCCW'), where the 'R' applies to rotated step and repeats.
AS0, AS1, AS2 As Double	Get: Text aspect ratio.

C0X, C1X, C2X As Double	Get: Centre point X.
C0Y, C1Y, C2Y As Double	Get: Centre point Y.
CH0, CH1, CH2 As Double	Get: Cusp height.
CS0, CS1, CS2 As Boolean	Get: Common cut slot.
D0X, D1X, D2X As Double	Get: Feature datum point X.
D0Y, D1Y, D2Y As Double	Get: Feature datum point Y.
DIMA0, DIMA1, DIMA2 As Double	Get: Dimension: angle of supplementary line.
DIMC0, DIMC1, DIMC2 As Long	Get: Dimension: form.
DIMEP0, DIMEP1, DIMEP2 As Long	Get: Dimension: terminator inside/outside end witness.
DIMES0, DIMES1, DIMES2 As Double	Get: Dimension: end witness standoff length.
DIMEW0, DIMEW1, DIMEW2 As Boolean	Get: Dimension: end witness line visible.
DIMEX0, DIMEX1, DIMEX2 As Double	Get: Dimension: end witness extension length.
DIMK0, DIMK1, DIMK2 As Boolean	Get: Dimension: kink is on.
DIMKA0, DIMKA1, DIMKA2 As Double	Get: Dimension: kink angle.
DIMKH0, DIMKH1, DIMKH2 As Double	Get: Dimension: kink height.
DIML0, DIML1, DIML2 As Long	Get: Dimension: dimension line automatic(0)/off(1)/on(2).
DIMP0, DIMP1, DIMP2 As Boolean	Get: Dimension: parametric.
DIMS0, DIMS1, DIMS2 As Long	Get: Dimension: style.
DIMSP0, DIMSP1, DIMSP2 As Long	Get: Dimension: terminator inside/outside start witness.
DIMSS0, DIMSS1, DIMSS2 As Double	Get: Dimension: start witness standoff length.
DIMSW0, DIMSW1, DIMSW2 As Boolean	Get: Dimension: start witness line visible.
DIMSX0, DIMSX1, DIMSX2 As Double	Get: Dimension: start witness extension length.
DIMT0, DIMT1, DIMT2 As Long	Get: Dimension: text position (above/in/below line).
DIMTH0, DIMTH1, DIMTH2 As Long	Get: Dimension: text horizontal or at angle.
DIMTO0, DIMTO1, DIMTO2 As Long	Get: Dimension: text angle (0 - 0, 1 - 90, 2 - 180, 3 - 270).
DIMV0, DIMV1, DIMV2 As Double	Get: Dimension: value.
DP0, DP1, DP2 As String	Get: Dimension precision string.
DT0, DT1, DT2 As String	Get: Expanded dimension text.
E0X, E1X, E2X As Double	Get: End point X.
E0Y, E1Y, E2Y As Double	Get: End point Y.
EO0, EO1, EO2 As Double	Get: End overlap.
ET0, ET1, ET2 As Long	Get: End terminator.
ETS0, ETS1, ETS2 As Double	Get: End terminator size.
FA0, FA1, FA2 As Double	Get: Angle (degrees), orientation of symbol/text, untransformed swept angle for ellipse.
FF0, FF1, FF2 As Long	Get: Text font.
FI0, FI1, FI2 As String	Get: Feature identifier.
FL0, FL1, FL2 As Double	Get: Length of Features, circumference for arcs/ellipse, total width for text string.
FP0, FP1, FP2 As Long	Get: Pen.
FR0, FR1, FR2 As Double	Get: Arc radius, total height for text string, geometric mean of semi-major and semi-minor axes for ellipse.
FS0, FS1, FS2 As Double	Get: Pattern scale.
FT0, FT1, FT2 As String	Get: Dimension text, pattern/symbol name, text feature text.
FTR_INF0, FTR_INF1, FTR_INF2 As String	Get: Additional feature information.
FX0, FX1, FX2 As Double	Get: X length component, symbol X scale, major axis for ellipse, character width for text.
FY0, FY1, FY2 As Double	Get: Y length component, symbol Y scale, minor axis for ellipse, character height for text.
F_BEND_ALLOWANCE0, F_BEND_ALLOWANCE1, F_BEND_ALLOWANCE2 As Double	Get: Allowance value corresponding to the allowance type (i.e. neutral axis or setback).

F_BEND_ALLOWANCE_TYPE0, F_BEND_ALLOWANCE_TYPE1, F_BEND_ALLOWANCE_TYPE2 As Long	Get: Bend allowance type (0 - neutral axis, 1 - setback).
F_BEND_ANGLE0, F_BEND_ANGLE1, F_BEND_ANGLE2 As Double	Get: Bend angle (degrees).
F_BEND_RADIUS0, F_BEND_RADIUS1, F_BEND_RADIUS2 As Double	Get: Bend radius (in the current drawing units).
F_BEND_ID0, F_BEND_ID1, F_BEND_ID2 As String	Get: Bend identifier.
F_BEV_BOT0, F_BEV_BOT1, F_BEV_BOT2 As Boolean	Get: Bevel: Bottom bevelled.
F_BEV_BOT_A0, F_BEV_BOT_A1, F_BEV_BOT_A2 As Double	Get: Bevel: Bottom start angle.
F_BEV_BOT_D0, F_BEV_BOT_D1, F_BEV_BOT_D2 As Double	Get: Bevel: Bottom start depth.
F_BEV_BOT_FULL0, F_BEV_BOT_FULL1, F_BEV_BOT_FULL2 As Boolean	Get: Bevel: Full depth.
F_BEV_TOP0, F_BEV_TOP1, F_BEV_TOP2 As Boolean	Get: Bevel: Top bevelled.
F_BEV_TOP_A0, F_BEV_TOP_A1, F_BEV_TOP_A2 As Double	Get: Bevel: Top start angle.
F_BEV_TOP_D0, F_BEV_TOP_D1, F_BEV_TOP_D2 As Double	Get: Bevel: Top start depth.
F_BEV_TOP_FULL0, F_BEV_TOP_FULL1, F_BEV_TOP_FULL2 As Boolean	Get: Bevel: Full depth.
F_UNDERCUT_LEAD_IN_REQUIRED0, F_UNDERCUT_LEAD_IN_REQUIRED1, F_UNDERCUT_LEAD_IN_REQUIRED2 As Boolean	Get: Feature/profile should be tooled with an undercut lead-in.
LL0, LL1, LL2 As Long	Get: Level of parent symbol (1 = top Level).
LN0, LN1, LN2 As String	Get: Name of parent symbol.
LT0, LT1, LT2 As ULong	Get: Line type, general feature type.
LX0, LX1, LX2 As Double	Get: X coordinate of parent symbol.
LY0, LY1, LY2 As Double	Get: Y coordinate of parent symbol.
MK0, MK1, MK2 As Long	Get: Mark number.
MN0, MN1, MN2 As Long	Get: Feature extension count.
MT0, MT1, MT2 As Double	Get: Master tool number.
PA0, PA1, PA2 As String	Get: Feature path.
PARAM0, PARAM1, PARAM2 As String	Get: Pattern parametric type (N - none, P - parametric, R - rigid).
PC0, PC1, PC2 As String	Get: Path name component of the feature identifier.
PN0, PN1, PN2 As String	Get: Punch or nibble ('P' or 'N').
PRF_BRIDGE_ORDER0, PRF_BRIDGE_ORDER1, PRF_BRIDGE_ORDER2 As Long	Get: Profile number of the bridged profile.
PRF_CLOSED0, PRF_CLOSED1, PRF_CLOSED2 As Long	Get: Closed profile feature (TRI_TRUE, TRI_FALSE or TRI_INDETERMINATE).
PRF_CORNER_LOOP0, PRF_CORNER_LOOP1, PRF_CORNER_LOOP2 As Boolean	Get: Corner loop feature.
PRF_COMMON_GRID_CUT0, PRF_COMMON_GRID_CUT1, PRF_COMMON_GRID_CUT2 As Boolean	Get: Common grid cut feature.

PRF_CUT_FIRST0, PRF_CUT_FIRST1, PRF_CUT_FIRST2 As Boolean	Get: Cut this feature first if the order is ambiguous.
PRF_DUPLICATE0, PRF_DUPLICATE1, PRF_DUPLICATE2 As Boolean	Get: Duplicate feature.
PRF_ENDBURN0, PRF_ENDBURN1, PRF_ENDBURN2 As Boolean	Get: Endburn feature.
PRF_EPT_MJ0, PRF_EPT_MJ1, PRF_EPT_MJ2 As Boolean	Get: Micro-joint terminator on feature end point.
PRF_EPT_TERM0, PRF_EPT_TERM1, PRF_EPT_TERM2 As Long	Get: Profile terminator type on feature end point (PCC_NO_TERM, PCC_TERM_START, PCC_TERM_END, PCC_TERM).
PRF_INTERNAL0, PRF_INTERNAL1, PRF_INTERNAL2 As Long	Get: Internal profile feature (TRI_TRUE, TRI_FALSE or TRI_INDETERMINATE).
PRF_OFFCUT0, PRF_OFFCUT1, PRF_OFFCUT2 As Boolean	Get: Offcut feature.
PRF_ORDER0, PRF_ORDER1, PRF_ORDER2 As Long	Get: Profile number.
PRF_OVERBURN0, PRF_OVERBURN1, PRF_OVERBURN2 As Boolean	Get: Overburn feature.
PRF_PLASMA_BOLT_HOLE0, PRF_PLASMA_BOLT_HOLE1, PRF_PLASMA_BOLT_HOLE2 As Boolean	Get: Plasma bolt hole feature.
PRF_REVERSE0, PRF_REVERSE1, PRF_REVERSE2 As Boolean	Get: Reverse profile feature.
PRF_SCRAP_CUT0, PRF_SCRAP_CUT1, PRF_SCRAP_CUT2 As Boolean	Get: Scrap cut feature.
PRF_SMART_ORDER_STATUS0, PRF_SMART_ORDER_STATUS1, PRF_SMART_ORDER_STATUS2 As Long	Get: Smart order status (DDM_SMART_ORDER_UNDEFINED, DDM_SMART_ORDER, DDM_NON_SMART_ORDER).
PRF_SPT_MJ0, PRF_SPT_MJ1, PRF_SPT_MJ2 As Boolean	Get: Micro-joint terminator on feature start point.
PRF_SPT_TERM0, PRF_SPT_TERM1, PRF_SPT_TERM2 As Long	Get: Profile terminator type on feature start point (PCC_NO_TERM, PCC_TERM_START, PCC_TERM_END, PCC_TERM).
PRF_TYPE0, PRF_TYPE1, PRF_TYPE2 As Long	Get: Profile feature type (PCC_MID_PROFILE, PCC_LEAD_IN, PCC_LEAD_OUT, PCC_OPEN_LEAD_IN).
PRF_UNDERCUT0, PRF_UNDERCUT1, PRF_UNDERCUT2 As Boolean	Get: Undercut feature.
PV0, PV1, PV2 As Boolean	Get: Pattern visibility.
S0X, S1X, S2X As Double	Get: Start point X.
S0Y, S1Y, S2Y As Double	Get: Start point Y.
SA0, SA1, SA2 As Double	Get: Angle of shear-invariant axis for ellipse.
SC0, SC1, SC2 As Long	Get: Scrap code.
SD0, SD1, SD2 As String	Get: Symbol file path.
SL0, SL1, SL2 As Double	Get: Text slant.
SM0, SM1, SM2 As Long	Get: Symmetry (0 - none, 1 - reflect, 2 - rotate).
SO0, SO1, SO2 As Double	Get: Start overlap, angle of major axis to horizontal for ellipse.
SR0, SR1, SR2 As Long	Get: Symbol step and repeat number.
ST0, ST1, ST2 As Long	Get: Start terminator.
STS0, STS1, STS2 As Double	Get: Start terminator size.
TA0, TA1, TA2 As String	Get: Tool angle ('X' or 'Y').
TE0X, TE1X, TE2X As Double	Get: True end point X (tooling feature).
TE0Y, TE1Y, TE2Y As Double	Get: True end point Y (tooling feature).
TF0, TF1, TF2 As String	Get: Feature type, 'a', 'l', etc.

TO0, TO1, TO2 As String	Get: Tool offset ('I' or 'O' or 'C').
TS0X, TS1X, TS2X As Double	Get: True start point X (tooling feature).
TS0Y, TS1Y, TS2Y As Double	Get: True start point Y (tooling feature).
TX0, TX1, TX2 As Double	Get: Tool size in X.
TY0, TY1, TY2 As Double	Get: Tool size in X.
Z10, Z11, Z12 As Double	Get: 'Z1' values of feature.
Z20, Z21, Z22 As Double	Get: 'Z2' values of feature.

Find results (1 - last found point/feature, 2 - last but one found point/feature)

Property	Description
CD As Long	Get: Selected smart hatching datum point (for smart select).
CI As String	Get: Found point identifier.
CW As String	Get: Found point type (s - start, e - end, c - centre, d - datum).
P1X, P2X As Double	Get: Found point X.
P1Y, P2Y As Double	Get: Found point Y.
PD1, PD2 As Long	Get: Found datum point number.
PI1, PI2 As String	Get: Found feature point is on the feature with this identifier.
PW1, PW2 As String	Get: Where the found feature point is on the feature.

Profile feature type

Property	Description
PCC_LEAD_IN As Long	Get: Profile feature type.
PCC_LEAD_OUT As Long	Get: Profile feature type.
PCC_OPEN_LEAD_IN As Long	Get: Profile feature type.
PCC_MID_PROFILE As Long	Get: Profile feature type.

Profile feature terminator type

Property	Description
PCC_NO_TERM As Long	Get: Profile feature terminator type.
PCC_TERM As Long	Get: Profile feature terminator type.
PCC_TERM_END As Long	Get: Profile feature terminator type.
PCC_TERM_START As Long	Get: Profile feature terminator type.

General feature types for unload data

Property	Description
ORD_UNL_GTYPE_BIN_INFO As Long	Get: General feature type for unload bin info.
ORD_UNL_GTYPE_PICK_INFO As Long	Get: General feature type for unload pick info.
ORD_UNL_GTYPE_PTN_SCALE As Long	Get: General feature type for unload pattern scale.

Feature info constants

Property	Description
PCC_TEXT_FAST As String	Get: Feature info 'fast' value.
PCC_TEXT_PIERCE_DATA As String	Get: Feature info pierce data tag.
PCC_TEXT_PROF_MARKING_DATA As String	Get: Feature info profile marking data tag.

Smart order status types

Property	Description
DDM_NON_SMART_ORDER As Long	Get: Smart order status (for symbols, linear and arc slots).
DDM_SMART_ORDER As Long	Get: Smart order status (for symbols, linear and arc slots).
DDM_SMART_ORDER_UNDEFINED As Long	Get: Smart order status (for symbols, linear and arc slots).

Smart select latch types

Property	Description
LATCH_FEATURE As Long	Get: Smart select latch type: Latched to feature.
LATCH_NONE As Long	Get: Smart select latch type: Not latched.
LATCH_POINT As Long	Get: Smart select latch type: Latched to point.

Tri-state Boolean types for profile types

Property	Description
TRI_TRUE As Long	Get: True.
TRI_FALSE As Long	Get: False.
TRI_INDETERMINATE As Long	Get: Indeterminate.

Mac object: Editing 2D features

The Mac object contains a set of methods that can be used for editing some of the properties of 2D drafting and tooling features. Usually, properties that cannot be edited using these methods can be edited using keystroke commands, see Keystroke commands and variables.

Feature editing methods

fed_batch_begin() As Boolean	Feature Editor: Begin a batch of edits. Do not update the application state after each edit until a call to fed_batch_end(). The use of fed_batch_begin/end() is not required. However, if you are editing multiple features or profiles, it is recommended. # Return Value Return true for success, false for failure.
fed_batch_end() As Boolean	Feature Editor: End a batch of edits. Update the state of the application to reflect the edits made since the call to fed_batch_begin(). Matching pairs of fed_batch_begin/end() calls can be nested. The application will not be updated until the last call of fed_batch_end(). # Return Value Return true for success, false for failure.
fed_clear_properties() As Boolean	Feature Editor: Clear (i.e. set to be undefined) all current FED property values (e.g. FED_BEND_ANGLE, etc). See the table below for a list of all FED properties. # Return Value

fed_edit_feature(

- ByVal *featureId* As String

) As Boolean

Return true for success, false for failure.

Feature Editor: Use the current FED property values (e.g. FED_BEND_ANGLE, etc) to edit the feature with the given feature identifier. If this fails, return false and set FED_ERROR_STRING with an error message. Otherwise return true.

See the table below for details of the FED properties which act as parameters to this method.

Parameters

[in] featureId
Feature identifier of the feature to edit.

Return Value

Return true for success, false for failure (and set FED_ERROR_STRING with an error message).

fed_edit_profile_features(

- ByVal *featureId* As String

) As Boolean

Feature Editor: Use the current FED property values to edit each feature in the profile that contains the feature with the given feature identifier. If this fails, return false and set FED_ERROR_STRING with an error message. Otherwise return true.

See the table below for details of the FED properties which act as parameters to this method.

Parameters

[in] featureId
Feature identifier of a feature in the profile.

Return Value

Return true for success, false for failure (and set FED_ERROR_STRING with an error message).

Error properties

FED_ERROR_STRING As Double	Get: Feature Editor: Set with a suitable error message when one of the Feature Editor functions (e.g. fed_edit_feature()) fails, and cleared if the function succeeds.

Feature properties

FED_BEND_ALLOWANCE As Double	Set/get: Feature Editor property: Allowance value corresponding to the allowance type (i.e. neutral axis or setback).
FED_BEND_ALLOWANCE_TYPE As Long	Set/get: Feature Editor property: Bend allowance type (0 - neutral axis, 1 - setback).
FED_BEND_ANGLE As Double	Set/get: Feature Editor property: Bend angle (-180 to 180 degrees).
FED_BEND_RADIUS As Double	Set/get: Feature Editor property: Bend radius (in current drawing units).
FED_UNDERCUT_LEAD_IN_REQUIRED As Boolean	Set/get: Feature Editor property: An undercut lead-in is required on the profile.

Example using C#

The following C# example shows how the Feature Editor functions and properties can be used to edit features during a scan.

```
// Get the handle to the Mac object (see Creating .NET plug-ins and Keystroke command syntax using C#).
Radraft.Interop.IMac mac = m_app.Mac;

// Scan for all lines and arcs (see Scanning and finding 2D features).
if (mac.scan("/", "al", 0))
{
    // Since we are going to make several edits, it is
```

```
    // more efficient to batch them together (we do not
    // need the application to be updated after every edit).
    mac.fed_batch_begin();

    while (mac.next() != 0)
    {
        // Change all 90 degree bends to 45 degree down bends.
        if (mac.F_BEND_ANGLE0 == 90.0)
        {
            // Clear all current property values.
            mac.fed_clear_properties();

            // Set the bend angle and radius properties.
            mac.FED_BEND_ANGLE = 45.0; // Degrees
            mac.FED_BEND_RADIUS = 3.0;

            // Edit the bend angle and radius properties
            // of the currently scanned feature.
            if (!mac.fed_edit_feature(mac.FI0))
                MessageBox.Show(mac.FED_ERROR_STRING);
        }

        // Set the undercut lead-in required flag on all profiles drawn in pen 3.
        if (mac.FP0 == 3 && !mac.F_UNDERCUT_LEAD_IN_REQUIRED0)
        {
            // Clear all current property values (in particular,
            // the bend properties we have just used).
            mac.fed_clear_properties();

            // Set the undercut lead-in property.
            mac.FED_UNDERCUT_LEAD_IN_REQUIRED = true;

            // Edit the undercut lead-in required property on all of the
            // features in the same profile as the currently scanned feature.
            if (!mac.fed_edit_profile_features(mac.FI0))
                MessageBox.Show(mac.FED_ERROR_STRING);
        }
    }

    // End the batch of edits. The state of the application will
    // now be updated to reflect the changes made by the edits.
    mac.fed_batch_end();
}
```

Mac object: 2D geometry analysis

The Mac object contains a number of methods for analysing 2D geometry. In particular, it contains a set of methods for efficiently sorting lines and arcs into loops and performing geometrical operations on two-dimensional shapes. These methods are collectively known as the elf library (elf - Efficent Loops Functions).

The elf functions deal with **lines and arcs** only. Arcs may be elliptical. Other drafting features like text and hatching, as well as tooling features, are ignored by the elf if they are present in the data (but see below for the handling of symbols).

By default, only lines and arcs with Full linetype are relevant: dashed lines and arcs are ignored if present. By default, the elf functions also take account of the 'pens to ignore in nesting' system data (item u15 in the 'psys_user' file): if pen 2 is set to be ignored in nesting, for example, then all lines and arcs in pen 2 will be ignored by the elf. You can change this behaviour by using the `elf_set_option()` function.

When using an elf function in Mac, you must generally give it the path of a **pattern** containing all the lines and arcs that you want to analyse. If your macro is working with a drawing then this might be the root pattern; in the Part Editor, you would normally use `PART_PATTERN` as the source pattern. For most operations you can use a memory buffer called the **cache** as an alternative to using a pattern.

Except for `elf_copy_geometry()`, the elf functions ignore lines and arcs in sub-patterns of the source pattern.

If the source pattern contains **symbols** then lines and arcs within the symbols are included in the analysis. Tool symbols are an exception: lines and arcs within tool symbols are ignored by the elf (even if the symbol was fixed down in drafting mode). It follows that tool symbols must first be 'realised' if you want them to be included in the analysis. Note that the elf functions do not work with symbols as such, but with lines and arcs copied from symbols; the result pattern created by an elf function will not

contain any symbol instances.

Two features are said to be **connected** if they join end-to-end. The elf tolerates (in fact it automatically 'heals') very small gaps between features (less than about 0.01 millimetre), but if a gap is larger than this the features are *not* treated as connected. If your data is likely to contain significant gaps, or duplicated and overlapping features, then you should clean up the geometry before using the elf functions on it (e.g. using profile_healing()).

A sequence of connected lines and arcs is called a **loop** (or sometimes a 'profile', but that word usually refers to tooling). If a number of features are connected in a complete circuit they form a **closed loop**, otherwise they form an **open loop**. A circle (or ellipse) is a simple case of a closed loop, consisting of just one arc.

A pattern consists of **closed geometry** if all relevant lines and arcs are in closed loops and if, in addition, none of the loops intersect with or touch each other. Remember that sub-patterns are ignored, so a pattern can contain closed geometry even if its sub-patterns do not. Closed geometry can be considered to be a well-formed **shape**, such as the shape of a sheet-metal part. The 'Check geometry' button in the Part Editor and in the Patterns list tests whether geometry is closed.

A closed loop around the outside of a shape is called an **external loop** and a closed loop that defines a hole in the shape is called an **internal loop**. In a sheet metal context, a 'part' is normally a shape with one external loop and perhaps a number of internal loops, but in general the elf allows a shape to contain any number of external loops as well as any number of internal loops. A shape with several external loops could, for example, be a set of parts arranged together in a miniature nest and saved together as one symbol.

The elf treats a loop inside an internal loop as an external loop. If there is another loop inside that one it is treated as an internal loop, and so on. An external loop inside an internal loop is like a small part nested in an aperture in a large part.

Certain operations use the idea of a **convex loop**. A loop is convex if it is closed and if a walk around the loop in the anticlockwise direction does not involve turning right at any point. Less formally, a loop is convex if there are no indentations in its shape.

elf methods

Method	Description
elf_arcs_to_lines(• ByVal *sourcePattern* As String • ByVal *targetPattern* As String • ByVal *outsideShape* As Boolean • ByVal *chordTol* As Double • ByVal *reserved* As Double) As Boolean	Copy geometry from the source pattern (or ELF_CACHE_ID) to the target pattern, replacing arcs with sequences of lines. **Parameters** [in] sourcePattern Pattern to convert (or ELF_CACHE_ID). [in] targetPattern Pattern to receive results. [in] outsideShape Lines on outside of shape, else on the inside. [in] chordTol Chord tolerance. [in] reserved Reserved - set to 0. **Return Value** Return true for success, false for no geometry or failure.
elf_bounds(	Return the lower left corner and the upper right corner of the bounding box of the geometry in the pattern (or ELF_CACHE_ID). This only takes notice of lines and arcs that count as geometry. **Parameters** [in] sourcePattern Pattern to analyse (or ELF_CACHE_ID).

- ByVal *sourcePattern* As String
- ByVal *reserved* As String
- ByRef *left* As Double
- ByRef *bottom* As Double
- ByRef *right* As Double
- ByRef *top* As Double

) As Boolean

[in] reserved
Reserved - set to empty string.
[out] left
Left.
[out] bottom
Bottom.
[out] right
Right.
[out] top
Top.

Return Value

Return true for success, false for no geometry or failure.

Extract geometry from the given pattern, analyse it, and store the results in the cache for later use.

elf_cache(

- ByVal *pattern* As String

) As Boolean

Parameters

[in] pattern
Extract geometry from this pattern.

Return Value

Return true for success, false for failure.

Return true if the pattern contains closed geometry, false if it does not.

elf_closed(

- ByVal *pattern* As String
- ByVal *graphicsMode* As Long

) As Boolean

Parameters

[in] pattern
Pattern to analyse (or ELF_CACHE_ID).
[in] graphicsMode
Kind of graphics wanted.

Return Value

Return true if the pattern contains closed geometry, false if it does not.

Construct the convex hull of the geometry in the source pattern (or ELF_CACHE_ID). Put the results in the target pattern.

elf_convex_hull(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *chordTol* As Double

) As Boolean

Parameters

[in] sourcePattern
Pattern to get hull of (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] chordTol
Chord tolerance.

Return Value

Return true for success, false for failure.

Copy geometry from a source pattern (or ELF_CACHE_ID) to a target pattern.

elf_copy_geometry(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *includeSubPatterns* As Boolean

) As Boolean

Parameters

[in] sourcePattern
Pattern to copy from (or ELF_CACHE_ID).
[in] targetPattern
Pattern to copy to.
[in] includeSubPatterns
Copy from sub-patterns too.

Return Value

Return true for success, false for failure.

Free the memory used by the cache.

elf_free_cache() As Boolean

Return Value

Return true for success, false for failure.

Compute the intersection of two shapes (source patterns can be ELF_CACHE_ID).

elf_intersection(

- ByVal *sourcePattern1* As String
- ByVal *sourcePattern2* As String
- ByVal *targetPattern* As String

) As Boolean

Parameters

[in] sourcePattern1
Pattern containing first shape (or ELF_CACHE_ID).
[in] sourcePattern2
Pattern containing second shape (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.

Return Value

Return true for success, false for failure.

Copy geometry from one pattern (or ELF_CACHE_ID) to another, translating it by a vector.

elf_move(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *x* As Double
- ByVal *y* As Double
- ByVal *reserved* As Double

) As Boolean

Parameters

[in] sourcePattern
Pattern to move (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] x
X displacement.
[in] y
Y displacement.
[in] reserved
Reserved - set to 0.

Return Value

Return true for success, false for no geometry or failure.

Set (*nx, *ny) to the nearest point to (x, y) that lies on a line or arc in the given pattern (or ELF_CACHE_ID). Also set *distance to the distance from (x, y) to (*nx, *ny).

elf_nearest_point_on_edge(

- ByVal *sourcePattern* As String
- ByVal *x* As Double
- ByVal *y* As Double
- ByRef *nx* As Double
- ByRef *ny* As Double
- ByRef *distance* As Double

) As Boolean

Parameters

[in] sourcePattern
Pattern containing shape (or ELF_CACHE_ID).
[in] x
X coord of point.
[in] y
Y coord of point.
[out] nx
X coord of nearest point on edge.
[out] ny
Y coord of nearest point on edge.
[out] distance
Distance to nearest point.

Return Value

Return true for success, false for no geometry or failure.

Construct the parallel profile of the geometry in the source pattern (or ELF_CACHE_ID). Put the results in the target pattern. (Note: minFilletAngle is in degrees).

elf_parallel_profile(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *offset* As Double
- ByVal *minFilletAngle* As Double
- ByVal *reserved* As Double

) As Boolean

Parameters

[in] sourcePattern
Pattern to profile (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] offset
Parallel distance.
[in] minFilletAngle
Minimum angle to fillet (degrees).
[in] reserved
Reserved - set to 0.

Return Value

Return true for success, false for failure.

Construct a partition of the geometry in the source pattern (or ELF_CACHE_ID). Create sub-patterns in the target pattern, each containing one (anticlockwise) loop of the partition. Return the number of pieces in the partition or 0 for failure.

elf_partition(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *partitionType* As Long
- ByVal *dashedDiagonals* As Boolean
- ByVal *chordTol* As Double

) As Long

Parameters

[in] sourcePattern
Pattern to get partition of (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] partitionType
Type of partition.
[in] dashedDiagonals
Use dashed lines for diagonals.
[in] chordTol
Chord tolerance for linearisation.

Return Value

Return 0 for failure or the number of pieces in the partition for success.

Return 0 if point is outside the shape in the given pattern (or ELF_CACHE_ID), 1 if inside, or 2 if on the edge of the shape.

Parameters

elf_point_in_shape(

- ByVal *sourcePattern* As String
- ByVal *x* As Double
- ByVal *y* As Double

) As Long

[in] sourcePattern
Pattern containing shape (or ELF_CACHE_ID).
[in] x
X coord of point.
[in] y
Y coord of point.

Return Value

Return 0 if point is outside the shape in the given pattern (or ELF_CACHE_ID), 1 if inside, or 2 if on the edge of the shape.

Set the object properties giving the area (PAR), centroid (PCX, PCY), and other properties of the given pattern (or ELF_CACHE_ID).

Parameters

elf_properties(

- ByVal *pattern* As String

) As Boolean

[in] pattern
Pattern to analyse (or ELF_CACHE_ID).

Return Value

Return true for success, false for no geometry or failure.

Copy geometry from one pattern (or ELF_CACHE_ID) to another, rotating it by angle (in radians) around a point.

Parameters

elf_rotate(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *x* As Double
- ByVal *y* As Double
- ByVal *angle* As Double

) As Boolean

[in] sourcePattern
Pattern to rotate (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] x
X coord of rotation centre.
[in] y
Y coord of rotation centre.
[in] angle
Rotation angle (radians).

Return Value

Return true for success, false for no geomtry or failure.

Copy geometry from one pattern (or ELF_CACHE_ID) to another, scaling it from a point.

Parameters

elf_scale(

[in] sourcePattern

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *x* As Double
- ByVal *y* As Double
- ByVal *scale* As Double

) As Boolean

Pattern to scale (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] x
X coord of scale centre.
[in] y
Y coord of scale centre.
[in] scale
Scale factor.

Return Value

Return true for success, false for no geometry or failure.

Set an option (e.g. 'all_linetypes', 'all_pens') to influence the working of the elf macro functions. Return the previous value of the changed option, or -1 if error.

elf_set_option(

- ByVal *option* As String
- ByVal *value* As Double

) As Double

Parameters

[in] option
Name of option to set (e.g. 'all_linetypes', 'all_pens').
[in] value
New value of option.

Return Value

Return the previous value of the changed option, or -1 if error.

Return true if the shape in the first pattern (or ELF_CACHE_ID) is inside the shape in the second (or ELF_CACHE_ID).

elf_shape_in_shape(

- ByVal *containedPattern* As String
- ByVal *containerPattern* As String

) As Boolean

Parameters

[in] containedPattern
Pattern (or ELF_CACHE_ID).
[in] containerPattern
Pattern (or ELF_CACHE_ID).

Return Value

Return true if the shape in the first pattern (or ELF_CACHE_ID) is inside the shape in the second (or ELF_CACHE_ID).

Find a short path between the two given points in the source pattern (or ELF_CACHE_ID), putting the result into the target pattern.

Parameters

elf_short_path_in_shape(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByVal *x1* As Double
- ByVal *y1* As Double
- ByVal *x2* As Double
- ByVal *y2* As Double
- ByVal *chordTol* As Double

[in] sourcePattern
Pattern containing shape (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[in] x1
Start point X.
[in] y1
Start point Y.
[in] x2
End point X.

) As Boolean

[in] y2
End point Y.
[in] chordTol
Chord tolerance for linearisation.

Return Value

Return true for success, false for failure.

Create sub-patterns of the target pattern, each containing one loop of lines and arcs copied from the source pattern (or ELF_CACHE_ID). The features in each sub-pattern are sorted into order.

elf_sort(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String
- ByRef *totalLoopsCount* As Long
- ByRef *externalLoopsCount* As Long
- ByRef *internalLoopsCount* As Long
- ByRef *openLoopsCount* As Long

) As Boolean

Parameters

[in] sourcePattern
Pattern to analyse (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.
[out] totalLoopsCount
Number of loops.
[out] externalLoopsCount
Number of external closed loops.
[out] internalLoopsCount
Number of internal closed loops.
[out] openLoopsCount
Number of open loops.

Return Value

Return true for success, false for failure.

Create a hierarchy of patterns in the target pattern containing lines/arcs copied from the source pattern (or ELF_CACHE_ID). Each pattern contains the lines/arcs of one loop (sorted into order), with sub-patterns for the loops contained within that loop.

elf_sort_tree(

- ByVal *sourcePattern* As String
- ByVal *targetPattern* As String

) As Boolean

Parameters

[in] sourcePattern
Pattern to analyse (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.

Return Value

Return true for success, false for failure.

Compute the shape that results from subtracting sourcePattern2 from sourcePattern1 (source patterns can be ELF_CACHE_ID).

elf_subtraction(

- ByVal *sourcePattern1* As String
- ByVal *sourcePattern2* As String
- ByVal *targetPattern* As String

Parameters

[in] sourcePattern1
Pattern containing first shape (or ELF_CACHE_ID).
[in] sourcePattern2
Pattern containing second shape (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.

) As Boolean

Return Value

Return true for success, false for failure.

Compute the union of two shapes. Source patterns can be ELF_CACHE_ID.

elf_union(

- ByVal *sourcePattern1* As String
- ByVal *sourcePattern2* As String
- ByVal *targetPattern* As String

) As Boolean

Parameters

[in] sourcePattern1
Pattern containing first shape (or ELF_CACHE_ID).
[in] sourcePattern2
Pattern containing second shape (or ELF_CACHE_ID).
[in] targetPattern
Pattern to receive results.

Return Value

Return true for success, false for failure.

Pattern properties results

Property	Description
PAH As Double	Get: Pattern properties: Area including holes.
PAR As Double	Get: Pattern properties: Area.
PCX As Double	Get: Pattern properties: Centroid X.
PCY As Double	Get: Pattern properties: Centroid Y.
PNF As Long	Get: Pattern properties: Feature count.
PNL As Long	Get: Pattern properties: Closed loop count.
PPA As Double	Get: Pattern properties: Angle of principal axis.
PPE As Double	Get: Pattern properties: Perimeter.
PPO As Double	Get: Pattern properties: Outer perimeter.
PUU As Double	Get: Pattern properties: Min moment of inertia.
PVV As Double	Get: Pattern properties: Max moment of inertia.
PXX As Double	Get: Pattern properties: Second moment about X axis.
PXY As Double	Get: Pattern properties: Product of inertia XY.
PYY As Double	Get: Pattern properties: Second moment about Y axis.
PZZ As Double	Get: Pattern properties: Second moment about normal axis.

Mac object: 2D geometry utilities

The Mac object contains a number of 2D geometry utility methods for profile extraction and healing. Generally, these perform the same operations that are available in the Geometry Utilities dialog.

2D geometry utilities

Method	Description
profile_extraction(• ByVal *pattern* As String • ByVal *includeSubPatterns* As	Do profile extraction in the same way as the Geometry Utilities dialog without showing the dialog. If this fails, PRS is set to an error message.

Parameters

[in] pattern
Path of pattern to process.
[in] includeSubPatterns

Boolean
- ByVal *deleteByPen* As Boolean
- ByVal *penMask* As Long
- ByVal *linesArcsOnly* As Boolean
- ByVal *fullLinetypeOnly* As Boolean

) As Boolean

Apply to nested patterns.
[in] deleteByPen
Delete-by-pen flag.
[in] penMask
Mask of pens to keep (bits 0 to 15).
[in] linesArcsOnly
Keep lines/arcs only.
[in] fullLinetypeOnly
Keep full lines/arcs only.

Return Value

Return true for success, false for failure (and set PRS to an error message).

Do profile healing in the same way as the Geometry Utilities dialog without showing the dialog. If this fails, PRS is set to an error message.

Parameters

profile_healing(

- ByVal *pattern* As String
- ByVal *includeSubPatterns* As Boolean
- ByVal *tolerance* As Double
- ByVal *realiseEllipses* As Boolean
- ByVal *removeSmallFtres* As Boolean
- ByVal *closeSmallGaps* As Boolean
- ByVal *mergeOverlaps* As Boolean
- ByVal *simplifyData* As Boolean

) As Boolean

[in] pattern
Path of pattern to process.
[in] includeSubPatterns
Apply to nested patterns.
[in] tolerance
Tolerance for healing (in drawing units)
[in] realiseEllipses
Realise ellipses.
[in] removeSmallFtres
Remove small isolated ftres.
[in] closeSmallGaps
Close small gaps.
[in] mergeOverlaps
Merge overlapping ftres.
[in] simplifyData
Carry out data smoothing.

Return Value

Return true for success, false for failure (and set PRS to an error message).

Do profile healing with a timeout (in seconds) in the same way as the Geometry Utilities dialog without showing the dialog. Returns 1 for success, 0 for failure, or -1 for an early abort of the task. If failure, set PRS to an error message.

Parameters

profile_healing_with_timeout(

- ByVal *pattern* As String
- ByVal *includeSubPatterns* As Boolean
- ByVal *tolerance* As Double
- ByVal *realiseEllipses* As Boolean
- ByVal *removeSmallFtres* As Boolean
- ByVal *closeSmallGaps* As Boolean
- ByVal *mergeOverlaps* As Boolean
- ByVal *simplifyData* As

[in] pattern
Path of pattern to process.
[in] includeSubPatterns
Apply to nested patterns.
[in] tolerance
Tolerance for healing (in drawing units)
[in] realiseEllipses
Realise ellipses.
[in] removeSmallFtres
Remove small isolated ftres.
[in] closeSmallGaps
Close small gaps.
[in] mergeOverlaps
Merge overlapping ftres.

Boolean
- ByVal *timeLimit* As Double

) As Long

[in] simplifyData
Carry out data smoothing.
[in] timeLimit
Time limit to apply (in seconds).

Return Value

Returns 1 for success, 0 for failure, or -1 for an early abort of the task. If failure, set PRS to an error message.

Mac object: Attributes

The Mac object contains a number of methods for managing drawing, symbol and 3D assembly attributes. With these methods it is possible to set and get attributes in the current drawing, the Part Editor or in files on disk. For more details about attributes, see Attributes and The list of manufacturing attributes.

Attributes are referred to by an attribute number. For a full list of valid attribute numbers, see The attribute list.

Attribute methods

Method	Description
att_current_dwg() As Long	Returns a handle to the attributes in the current drawing.
att_expand_str(• ByVal *handle* As Long • ByVal *inString* As String • ByRef *outString* As String) As Boolean	Replace attribute control codes (Sn) in the string with the value of attribute n, using the values in the given attribute table.
att_free(• ByVal *handle* As Long) As Boolean	Free the attribute table.

Return Value

Returns a handle to the attributes in the current drawing.

Parameters

[in] handle
Handle of attribute table.
[in] inString
String to process.
[out] outString
Variable to receive result.

Return Value

Return true for success, false for failure.

Parameters

[in] handle
Handle of attribute table.

Return Value

Return true for success, false for failure.

Get the value of a current drawing attribute.

att_get_dwg_attr(

- ByVal *number* As Long
- ByRef *value* As String

) As Boolean

Parameters

[in] number
Attribute number.
[out] value
Attribute value.

Return Value

Return true for success, false for failure.

Get various properties of the given group.

att_get_group_props(

- ByVal *handle* As Long
- ByVal *groupNumber* As Long
- ByRef *groupName* As String
- ByRef *groupDescription* As String
- ByRef *reserved1* As String
- ByRef *reserved2* As String
- ByRef *reserved3* As String
- ByRef *systemGroup* As Boolean
- ByRef *groupOrder* As Long
- ByRef *memberCount* As Long
- ByRef *reserved4* As Double
- ByRef *reserved5* As Double

) As Boolean

Parameters

[in] handle
Handle of attribute table.
[in] groupNumber
Group number to query.
[out] groupName
Group name.
[out] groupDescription
Group description.
[out] reserved1
Reserved.
[out] reserved2
Reserved.
[out] reserved3
Reserved.
[out] systemGroup
System group.
[out] groupOrder
Display order number.
[out] memberCount
Number of attributes in group.
[out] reserved4
Reserved.
[out] reserved5
Reserved.

Return Value

Return true for success, false for failure.

Get various properties of the given attribute.

att_get_props(

- ByVal *handle* As Long
- ByVal *number* As Long
- ByRef *name* As String
- ByRef *description* As String
- ByRef *reserved1* As String

Parameters

[in] handle
Handle of attribute table.
[in] number
Attribute number.
[out] name
Attribute name.
[out] description
Attribute description.
[out] reserved1
Reserved.
[out] reserved2

- ByRef *reserved2* As String
- ByRef *reserved3* As String
- ByRef *type* As Long
- ByRef *system* As Boolean
- ByRef *groupNumber* As Long
- ByRef *order* As Long
- ByRef *reserved4* As Double

) As Boolean

Reserved.
[out] reserved3
Reserved.
[out] type
Attribute type (0 - undefined, 1 - string, 2 - integer, 3 - real, 4 - time).
[out] system
System attribute.
[out] groupNumber
Group number.
[out] order
Display order number.
[out] reserved4
Reserved.

Return Value

Return true for success, false for failure.

Get various properties of the given attribute.

Parameters

att_get_valid_props(

- ByVal *handle* As Long
- ByVal *number* As Long
- ByRef *validString* As String
- ByRef *minValue* As String
- ByRef *maxValue* As String
- ByRef *defaultValue* As String
- ByRef *dummy* As String
- ByRef *switchedOn* As Boolean
- ByRef *permission* As Long
- ByRef *minLength* As Long
- ByRef *maxLength* As Long
- ByRef *mandatory* As Boolean

) As Boolean

[in] handle
Handle of attribute table.
[in] number
Attribute number.
[out] validString
Validation string.
[out] minValue
Minimum value.
[out] maxValue
Maximum value.
[out] defaultValue
Default value.
[out] dummy
[Unused.]
[out] switchedOn
Switched-on flag.
[out] permission
Permission.
[out] minLength
Minimum length.
[out] maxLength
Maximum length.
[out] mandatory
Mandatory flag.

Return Value

Return true for success, false for failure.

Gets the value of an attribute.

Parameters

att_get_value(

- ByVal *handle* As Long
- ByVal *number* As Long
- ByRef *value* As String

[in] handle
Handle of attribute table.
[in] number
Attribute number.
[out] value

) As Boolean

Attribute value (as a string).

Return Value

Return true for success, false for failure.

Is the attribute table open.

att_is_open(

- ByVal *handle* As Long

) As Boolean

Parameters

[in] handle
Handle of attribute table.

Return Value

Return true if the given attribute table is open, otherwise false.

Is the attribute present in the attribute table.

att_is_present(

- ByVal *handle* As Long
- ByVal *number* As Long

) As Boolean

Parameters

[in] handle
Handle of attribute table.
[in] number
Attribute number.

Return Value

Return true if the given attribute is present in the given attribute table, otherwise false.

Does the value of the given attribute meet the attribute's validation criteria.

att_is_valid(

- ByVal *handle* As Long
- ByVal *number* As Long

) As Boolean

Parameters

[in] handle
Handle of attribute table.
[in] number
Attribute number.

Return Value

Return true if the current value of the given attribute is valid, otherwise false.

Return the handle to a new attribute table filled with the attributes of the given drawing or symbol file.

att_load(

- ByVal *file* As String
- ByVal *useCachedCopy* As Boolean

) As Long

Parameters

[in] file
Drawing/symbol file path.
[in] useCachedCopy
If true, get from cache if possible, otherwise read from file. If false, always read from file.

Return Value

Return the handle to a new attribute table filled with the attributes of the given drawing or symbol file.

Create and initialise a table of attributes for a new drawing or symbol.

att_new(

- ByVal *file* As String

) As Long

Parameters

[in] file
File that the attributes will be saved to, or just a file extension to identify the type (e.g. 'drg' or 'sym').

Return Value

Return a handle to the attributes in the given file.

Get the number of the next attribute in the iteration.

att_next(

- ByVal *handle* As Long
- ByRef *number* As Long

) As Boolean

Parameters

[in] handle
Handle of attribute table.
[out] number
Attribute number.

Return Value

Return true for success, false for failure.

Initialise an iteration through the attributes in the given table.

att_rewind(

- ByVal *handle* As Long
- ByVal *order* As Long

) As Long

Parameters

[in] handle
Handle of attribute table.
[in] order
Order of the sequence (0 - any, 1 - grouped, 2 - numeric, 3 - alphabetic).

Return Value

Return the number of attributes in the sequence.

Set the value of the given attribute of the current drawing.

att_set_dwg_attr(

- ByVal *number* As Long
- ByVal *value* As String

) As Boolean

Parameters

[in] number
Attribute number.
[in] value
Attribute value.

Return Value

Return true for success, false for failure.

Set the value of the given attribute.

Parameters

att_set_value(

- ByVal *handle* As Long
- ByVal *number* As Long
- ByVal *value* As String

) As Boolean

[in] handle
Handle of attribute table.
[in] number
Attribute number.
[in] value
Attribute value.

Return Value

Return true for success, false for failure.

Make the value of the given attribute undefined.

Parameters

att_set_value_undef(

- ByVal *handle* As Long
- ByVal *number* As Long

) As Boolean

[in] handle
Handle of attribute table.
[in] number
Attribute number.

Return Value

Return true for success, false for failure.

Update the attribute table of the given drawing/symbol file with the attribute table referred to by the given handle.

Parameters

att_update_file(

- ByVal *file* As String
- ByVal *handle* As Long
- ByVal *merge* As Boolean

) As Boolean

[in] file
File to modify.
[in] handle
Handle of attribute table.
[in] merge
Whether to merge (true) or replace (false).

Return Value

Return true for success, false for failure.

Part Editor attribute methods

Method	Description
ped_attrs_handle() As Long	Get a handle for the attributes of the part being edited in the Part Editor.

Return Value

Handle of attribute table.

Get the material and orientation attributes of the part in the Part Editor. Units: 0 - mm, 1 - in, 2 - swg. Orientation: 1 - 0, 2 - 90, 3 - 0/90, 5 - 0/180, 6 - 90/270, 7 - 0/90/180/270, 8 - any.

Parameters

[out] name

ped_get_attrs2(

- ByRef *name* As String
- ByRef *material* As String
- ByRef *strategy* As String
- ByRef *thickness* As Double
- ByRef *units* As Long
- ByRef *orientation* As Long

) As Boolean

Name attribute.
[out] material
Material attribute.
[out] strategy
Strategy attribute.
[out] thickness
Thickness attribute.
[out] units
Thickness units attribute (0 - mm, 1 - in, 2 - swg).
[out] orientation
Allowed nest orientation attribute (1 - 0, 2 - 90, 3 - 0/90, 5 - 0/180, 6 - 90/270, 7 - 0/90/180/270, 8 - any).

Return Value

Return true for success, false for failure.

Set the material and orientation attributes of the part in the Part Editor. Units: 0 - mm, 1 - in, 2 - swg. Orientation: 1 - 0, 2 - 90, 3 - 0/90, 5 - 0/180, 6 - 90/270, 7 - 0/90/180/270, 8 - any.

Parameters

ped_set_attrs2(

- ByVal *name* As String
- ByVal *material* As String
- ByVal *strategy* As String
- ByVal *thickness* As Double
- ByVal *units* As Long
- ByVal *orientation* As Long

) As Boolean

[in] name
Name attribute.
[in] material
Material attribute.
[in] strategy
Strategy attribute.
[in] thickness
Thickness attribute.
[in] units
Thickness units attribute (0 - mm, 1 - in, 2 - swg).
[in] orientation
Allowed nest orientation attribute (1 - 0, 2 - 90, 3 - 0/90, 5 - 0/180, 6 - 90/270, 7 - 0/90/180/270, 8 - any).

Return Value

Return true for success, false for failure.

Mac object: 2D export

The Mac object contains methods that allow you to export 2D drawings and parts.

2D export methods

Method	Description
fla_thumbnail(	Export the current drawing or part (when in Part Editor mode) as an image to a BMP or PNG file. The image will be created using the given size (in pixels). Return true if the file is saved successfully, otherwise false. Notes: 1. Images will be created with a white background if **Make thumbnail backgrounds white** is selected on the Themes tab of your Colours system data. Otherwise, images will be created with the background colour of the drawing. 2. Nest annotation will not be included in images of nest drawings. 3. fla_thumbnail() can be called from 3D mode, but the image will only contain what

- ByVal *file* As String
- ByVal *width* As Long
- ByVal *height* As Long

) As Boolean

is shown in 2D mode.

Parameters

[in] file
Full path of BMP or PNG file in which to save the image.
[in] width
The required image width (in pixels).
[in] height
The required image height (in pixels).

Return Value

Return true if the file is saved successfully, otherwise false.

Mac object: 3D import

The Mac object contains a number of methods that allow you import 3D models that are stored using various different formats (e.g. SAT, Parasolid, etc). For more details about importing 3D models, see Introduction to importing files.

3D import methods

Method	Description
mfl_read_model(• ByVal *fileformat* As String • ByVal *heal* As Boolean • ByVal *regularise* As Boolean • ByVal *reverse* As Boolean • ByVal *combine* As Boolean • ByVal *allowComplexGeom* As Boolean • ByVal *scale* As Double • ByVal *thickness* As Double • ByVal *material* As String • ByVal *filename* As String) As Boolean	Read in a 3D model with the given format (i.e. 'sat', 'solidworks', 'catia v4', 'catia v5', 'parasolid', 'iges', 'step', 'inventor', 'proe', 'solid edge', 'unigraphics'). # Parameters [in] fileformat format of input file format (i.e. 'sat', 'solidworks', 'catia v4', 'catia v5', 'parasolid', 'iges', 'step', 'inventor', 'proe', 'solid edge', 'unigraphics'). [in] heal Heal flag. [in] regularise Regularise flag. [in] reverse Reverse flag. [in] combine Combine flag. [in] allowComplexGeom Allow complex geometry flag. [in] scale Scale. [in] thickness Thickness in mm. [in] material Material. [in] filename File to load. # Return Value Return true for success, false for failure. Read in a 3D model with the given format using an alternate method (i.e. 'inventor'). # Parameters

mfl_read_model_alternate(

- ByVal *fileformat* As String
- ByVal *heal* As Boolean
- ByVal *regularise* As Boolean
- ByVal *reverse* As Boolean
- ByVal *combine* As Boolean
- ByVal *allowComplexGeom* As Boolean
- ByVal *scale* As Double
- ByVal *thickness* As Double
- ByVal *material* As String
- ByVal *filename* As String

) As Boolean

[in] fileformat
format of input file format (i.e. 'inventor').
[in] heal
Heal flag.
[in] regularise
Regularise flag.
[in] reverse
Reverse flag.
[in] combine
Combine flag.
[in] allowComplexGeom
Allow complex geometry flag.
[in] scale
Scale.
[in] thickness
Thickness in mm.
[in] material
Material.
[in] filename
File to load.

Return Value

Return true for success, false for failure.

Read in a 3D model with the given format using default options (i.e. 'sat', 'solidworks', 'catia v4', 'catia v5', 'parasolid', 'iges', 'step', 'inventor', 'proe', 'solid edge', 'unigraphics').

Parameters

mfl_read_model_default(

- ByVal *fileformat* As String
- ByVal *filename* As String
- ByVal *material* As String
- ByVal *thickness* As Double
- ByVal *scale* As Double

) As Boolean

[in] fileformat
format of input file format (i.e. 'sat', 'solidworks', 'catia v4', 'catia v5', 'parasolid', 'iges', 'step', 'inventor', 'proe', 'solid edge', 'unigraphics').
[in] filename
File to load.
[in] material
Material.
[in] thickness
Thickness in mm.
[in] scale
Scale.

Return Value

Return true for success, false for failure.

Import a SAT file.

mfl_read_sat(

- ByVal *file* As String
- ByVal *thickness* As Double
- ByVal *material* As String
- ByVal *scale* As Double

) As Boolean

Parameters

[in] file
File to load.
[in] thickness
Thickness in mm.
[in] material
Material.
[in] scale
Scale.

Return Value

Return true for success, false for failure.
Import an SMX file (and centre the view).

mfl_read_smx(

- ByVal *file* As String

) As Boolean

Parameters

[in] file
File to load.

Return Value

Return true for success, false for failure.
Import an SMX file (do not centre the view).

mfl_read_smx_no_centre_view(

- ByVal *file* As String

) As Boolean

Parameters

[in] file
File to load.

Return Value

Return true for success, false for failure.
Read in a SolidWorks file using the given configuration file.

mfl_read_solidworks_configuration (

- ByVal *configurationName* As String
- ByVal *heal* As Boolean
- ByVal *regularise* As Boolean
- ByVal *reverse* As Boolean
- ByVal *combine* As Boolean
- ByVal *allowComplexGeom* As Boolean
- ByVal *scale* As Double
- ByVal *thickness* As Double
- ByVal *material* As String
- ByVal *filename* As String

) As Boolean

Parameters

[in] configurationName
Configuration name.
[in] heal
Heal flag.
[in] regularise
Regularise flag.
[in] reverse
Reverse flag.
[in] combine
Combine flag.
[in] allowComplexGeom
Allow complex geometry flag.
[in] scale
Scale.
[in] thickness
Thickness in mm.
[in] material
Material.
[in] filename
File to load.

Return Value

Return true for success, false for failure.

Mac object: 3D export

The Mac object contains a number of methods that allow you to export 3D models using various formats (e.g. SAT, STL, etc). For more details about exporting 3D models, see Introduction to exporting files.

3D export methods

Method	Description
mfl_thumbnail(• ByVal *file* As String • ByVal *width* As Long) As Boolean	Export the current view (when in 3D mode) as an image to a BMP or PNG file. The image will be created using the given width (in pixels) and the same aspect ratio as the current view. Return true if the file is saved successfully, otherwise false. # Parameters [in] file Full path of BMP or PNG file in which to save the image. [in] width The required image width (in pixels). # Return Value Return true if the file is saved successfully, otherwise false.
mfl_write_model(• ByVal *format* As String • ByVal *file* As String) As Boolean	Write a 3D model with the given format (i.e. 'iges', 'step'). # Parameters [in] format Format of output file format (i.e. 'iges', 'step'). [in] file Write to this file. # Return Value Return true for success, false for failure.
mfl_write_sat(• ByVal *file* As String • ByVal *majorVersion* As Long • ByVal *minorVersion* As Long) As Boolean	Export the current 3D model to a SAT file using the given SAT version. # Parameters [in] file Write to this file. [in] majorVersion SAT major version. [in] minorVersion SAT minor version. # Return Value Return true for success, false for failure.
mfl_write_smx(• ByVal *file* As String • ByVal *majorVersion* As Long	Export the current 3D model to an SMX file using the given SMX version. # Parameters [in] file Generic file name. [in] majorVersion SMX major version.

- ByVal *minorVersion* As Long

) As Boolean

[in] minorVersion
SMX minor version.

Return Value

Return true for success, false for failure.

Export the current 3D model to an SMX file, and choose whether to create multiple SMX files (1 per part).

Parameters

mfl_write_smx2(

- ByVal *file* As String
- ByVal *majorVersion* As Long
- ByVal *minorVersion* As Long
- ByVal *writeNewFilePerPart* As Boolean

) As Boolean

[in] file
Generic file name.
[in] majorVersion
SMX major version.
[in] minorVersion
SMX minor version.
[in] writeNewFilePerPart
Whether to create a new SMX file for each part.

Return Value

Return true for success, false for failure.

Export the current 3D model to an STL file (mesh density: 0 - low, 1 - medium, 2 - high).

Parameters

mfl_write_stl(

- ByVal *file* As String
- ByVal *meshDensity* As Long

) As Boolean

[in] file
Write to this file.
[in] meshDensity
Mesh density (0 - low, 1 - medium, 2 - high).

Return Value

Return true for success, false for failure.

Export the current 3D model to a VRML file (mesh density: 0 - low, 1 - medium, 2 - high).

Parameters

mfl_write_vrml(

- ByVal *file* As String
- ByVal *meshDensity* As Long
- ByVal *significantFigures* As Long

) As Boolean

[in] file
Write to this file.
[in] meshDensity
Mesh density (0 - low, 1 - medium, 2 - high).
[in] significantFigures
Number of significant figures required.

Return Value

Return true for success, false for failure.

Mac object: 3D unfolding

The Mac object contains a number of properties and methods that allow you unfold 3D sheet objects. For more details about

unfolding 3D sheet objects, see Introduction to the developer.

Unfold methods

Method	Description
mfl_auto_unfold(• ByVal *ident* As String) As Boolean	Unfold the 3D object with the given identifier. The unfolding operation can be influenced by setting variables such as MFL_U_PART_EDITOR_SPEC, and it sets other variables such as MFL_U_ERROR_RES to give information about the result. **Parameters** [in] ident Identifier of object to unfold. **Return Value** Return true for success, false for failure.
mfl_ray_dirn(• ByVal *x* As Double • ByVal *y* As Double • ByVal *z* As Double) As Boolean	Set the 3D direction vector of a ray (used with mfl_unfold). **Parameters** [in] x Ray direction vector X. [in] y Ray direction vector Y. [in] z Ray direction vector Z. **Return Value** Return true for success, false for failure.
mfl_ray_start(• ByVal *x* As Double • ByVal *y* As Double • ByVal *z* As Double) As Boolean	Set the 3D start point of a ray (used with mfl_unfold). **Parameters** [in] x Ray start X in current drawing units. [in] y Ray start Y in current drawing units. [in] z Ray start Z in current drawing units. **Return Value** Return true for success, false for failure.
mfl_unfold() As Boolean	Unfold a 3D object. This function does not use the variables that mfl_auto_unfold uses. It is recommended to use mfl_auto_unfold instead of mfl_unfold. **Return Value** Return true for success, false for failure.

Auto Unfold: Configuration

Property	Description
MFL_U_PART_EDITOR_SPEC As Boolean	Set/get: Auto unfold: User option: True to unfold directly to the Part Editor, false to unfold to a drawing pattern.
MFL_U_DEV_NAME_SPEC As String	Set/get: Auto unfold: User option: Development pattern name.
MFL_U_INFO_DF_AREA_SPEC As Double	Set/get: Auto unfold: User option: Report bend paths within this % area from best only.
MFL_U_INFO_DF_COUNT_SPEC As Long	Set/get: Auto unfold: User option: Report bend paths within this # dfs from best only.
MFL_U_MATERIAL_SPEC As String	Set/get: Auto unfold: User option: Sheet metal material.
MFL_U_PATH_SPEC As Long	Set/get: Auto unfold: User option: Which Bend Path number to unfold.
MFL_U_SMOOTHING As Boolean	Set/get: Auto unfold: User smoothing option: true - smoothing on, false - smoothing off.
MFL_U_SMOOTHING_CIRCULAR As Boolean	Set/get: Auto unfold: User smoothing option: true - smoothing on, false - smoothing off.
MFL_U_SMOOTHING_RADIUS As Double	Set/get: Auto unfold: User smoothing option: Circular relief radius.
MFL_U_SMOOTHING_SIZE As Double	Set/get: Auto unfold: User smoothing option: Square relief size.
MFL_U_SMOOTHING_SQUARE As Boolean	Set/get: Auto unfold: User smoothing option: true - smoothing on, false - smoothing off.
MFL_U_THICKNESS_SPEC As Double	Set/get: Auto unfold: User option: Thickness of sheet.
MFL_U_TOP_SURFACE_SPEC As Boolean	Set/get: Auto unfold: User option: True to unfold top surface, false to unfold other surface (if it exists, else do top surface again).
MFL_U_TYPE_SPEC As Long	Set/get: Auto unfold: User option: Type of auto analysis (MFL_U_TYPE_DEFAULT, MFL_U_TYPE_THICKNESS, MFL_U_TYPE_PATH, MFL_U_TYPE_INFO).

Auto Unfold: Results

Property	Description
MFL_U_DEV_PATTERN_RES As String	Get: Auto unfold result: Development pattern name.
MFL_U_DF_AREA_RES As Double	Get: Auto unfold result: Area (3D) of developed surface.
MFL_U_DF_COUNT_RES As Long	Get: Auto unfold result: Count of (3D) faces developed.
MFL_U_ERROR_RES As String	Get: Auto unfold result: Error string.
MFL_U_INVALID_FORM As Boolean	Get: Auto unfold result: Development is only partial.
MFL_U_NUM_DEVS_RES As Long	Get: Auto unfold result: One, or number of developments (info only).
MFL_U_THICKNESS_RES As Double	Get: Auto unfold result: Thickness of sheet or 0.0 for skin.
MFL_U_TYPE_RES As Long	Get: Auto unfold result: Type of development.
MFL_U_WARNINGS_COUNT As Long	Get: Auto unfold result: Count of Warnings.

Auto Unfold: Auto analysis types

Property	Description
MFL_U_TYPE_DEFAULT As Long	Get: Auto unfold: 'Default' auto analysis type.

MFL_U_TYPE_INFO As Long	Get: Auto unfold: 'Info' auto analysis type.
MFL_U_TYPE_PATH As Long	Get: Auto unfold: 'Path' auto analysis type.
MFL_U_TYPE_THICKNESS As Long	Get: Auto unfold: 'Thickness' auto analysis type.

Auto Unfold: Unfold types

Property	Description
MFL_U_TYPE_SHEET As Long	Get: Auto unfold: 'Sheet' type.
MFL_U_TYPE_SKIN As Long	Get: Auto unfold: 'Skin' type.
MFL_U_TYPE_UNKNOWN As Long	Get: Auto unfold: Unknown type.

Mac object: Scanning 3D entities

The Mac object contains a number of properties and methods that allow you to scan 3D entities. For more details about unfolding 3D sheet objects, see Introduction to the developer.

3D scan methods

Method	Description
mfl_info_scan_next() As String	Return the identifier of the next entity in the current 3D model scan (and set ND_IDENT1 and ND_SCANLEVEL1). **Return Value** Return the identifier of the next entity in the current 3D model scan (and set ND_IDENT1 and ND_SCANLEVEL1).
mfl_info_scan_start() As Boolean	Start a scan of all entities in the 3D model. **Return Value** Return true for success, false for failure.
mfl_object_info(• ByVal *ident* As String • ByVal *massPropertiesRequired* As Boolean) As Boolean	Return the details (using ND_NAME1, ND_VIS1, etc) of the 3D model entity with the given identifier. **Parameters** [in] ident Entity identifier. [in] massPropertiesRequired Calculate and return (using ND_VOLUME1, ND_MASS1, etc) the mass properties. **Return Value** Return true for success, false for failure.

3D scan results

Property	Description
ND_BOXMAXX1 As Double	Get: Maximum X of 3D entity specified by mfl_object_info.
ND_BOXMAXY1 As Double	Get: Maximum Y of 3D entity specified by mfl_object_info.
ND_BOXMAXZ1 As Double	Get: Maximum Z of 3D entity specified by mfl_object_info.

ND_BOXMINX1 As Double	Get: Minimum X of 3D entity specified by mfl_object_info.
ND_BOXMINY1 As Double	Get: Minimum Y of 3D entity specified by mfl_object_info.
ND_BOXMINZ1 As Double	Get: Minimum Z of 3D entity specified by mfl_object_info.
ND_COFGX1 As Double	Get: X centre of gravity of 3D entity specified by mfl_object_info.
ND_COFGY1 As Double	Get: Y centre of gravity of 3D entity specified by mfl_object_info.
ND_COFGZ1 As Double	Get: Z centre of gravity of 3D entity specified by mfl_object_info.
ND_COLBLUE1 As Double	Get: Blue component (0.0 - 1.0) of the colour of the 3D entity specified by mfl_object_info.
ND_COLGREEN1 As Double	Get: Green component (0.0 - 1.0) of the colour of the 3D entity specified by mfl_object_info.
ND_COLRED1 As Double	Get: Red component (0.0 - 1.0) of the colour of the 3D entity specified by mfl_object_info.
ND_DENSITY1 As Double	Get: Density of 3D entity specified by mfl_object_info.
ND_IDENT1 As String	Get: Identifier of 3D scanned entity or entity specified by mfl_object_info.
ND_MASS1 As Double	Get: Mass of 3D entity specified by mfl_object_info.
ND_MATERIAL1 As String	Get: Material of 3D entity specified by mfl_object_info.
ND_MATERIALSTATUS1 As Long	Get: Material status of 3D entity specified by mfl_object_info (MFL_MATERIAL_TRANSLATED, MFL_MATERIAL_IMPORT_DEFAULT, MFL_MATERIAL_SET_BY_USER, etc).
ND_NAME1 As String	Get: Name of 3D entity specified by mfl_object_info.
ND_SCANLEVEL1 As Long	Get: Scan level of 3D scanned entity.
ND_SURFAREA1 As Double	Get: Surface area of 3D entity specified by mfl_object_info.
ND_THICKNESS1 As Double	Get: Sheet thickness of 3D entity specified by mfl_object_info.
ND_THICKUNITS1 As String	Get: Sheet thickness units of 3D entity specified by mfl_object_info (mm, in, swg).
ND_THICKSTATUS1 As Long	Get: Sheet thickness status of 3D entity specified by mfl_object_info (MFL_THICKNESS_TRANSLATED, MFL_THICKNESS_IMPORT_DEFAULT, MFL_THICKNESS_CALCULATED, MFL_THICKNESS_CALCULATION_FAILED, MFL_THICKNESS_SET_BY_USER, etc).
ND_VIS1 As Boolean	Get: Visibility of 3D entity specified by mfl_object_info.
ND_VOLUME1 As Double	Get: Volume of 3D entity specified by mfl_object_info.

Material status constants

Property	Description
MFL_MATERIAL_IMPORT_DEFAULT As Long	Get: Material status: Import default.
MFL_MATERIAL_LEGACY As Long	Get: Material status: Legacy.
MFL_MATERIAL_SET_BY_USER As Long	Get: Material status: Set by user.
MFL_MATERIAL_SET_FROM_MAC As Long	Get: Material status: Set from MAC.
MFL_MATERIAL_TRANSLATED As Long	Get: Material status: Translated on import.
MFL_MATERIAL_UNKNOWN As Long	Get: Material status: Unknown.

Thickness status constants

Property	Description
MFL_THICKNESS_CALCULATED As Long	Get: Thickness status: Calculated.
MFL_THICKNESS_CALCULATION_FAILED As Long	Get: Thickness status: Calculation failed.
MFL_THICKNESS_IMPORT_DEFAULT As Long	Get: Thickness status: Import default.
MFL_THICKNESS_LEGACY As Long	Get: Thickness status: Legacy.
MFL_THICKNESS_SET_BY_USER As Long	Get: Thickness status: Set by user.

MFL_THICKNESS_SET_FROM_MAC As Long	Get: Thickness status: Set from MAC.
MFL_THICKNESS_TRANSLATED As Long	Get: Thickness status: Translated on import.
MFL_THICKNESS_UNKNOWN As Long	Get: Thickness status: Unknown.

Mac object: 3D utilities

The Mac object contains a number of 3D utility methods.

3D utility methods

Method	Description
mfl_set_material(• ByVal *ident* As String • ByVal *material* As String) As Boolean	Set the material of the 3D part with the given identifier. **Parameters** [in] ident Identifier of part. [in] material New material. **Return Value** Return true for success, false for failure.
mfl_set_thickness(• ByVal *ident* As String • ByVal *thickness* As Double • ByVal *units* As String) As Boolean	Set the thickness of the 3D part with the given identifier. **Parameters** [in] ident Identifier of part. [in] thickness New thickness value. [in] units New thickness units (mm, in, swg). **Return Value** Return true for success, false for failure.
mfl_calculate_thickness(• ByVal *ident* As String) As Double	Calculate 3D part thickness. If the calculation is successful, the thickness is set in the part properties and the value is also returned (in the current drawing units). If the calculation fails, 0.0 is returned. **Parameters** [in] ident Identifier of part. **Return Value** Return the thickness. If the calculation fails, 0 is returned.

Convert the current 3D model objects to 3D sheet objects, optionally deleting the original objects after conversion.

This looks at all the 3D objects in the current assembly and converts them whenever

possible to 3D sheet objects. It uses the planar face with the largest area in each object as the base face for an SMX output and input. SMX version 1 transfer format is used and so objects containing forms, hems, holes in bends, etc, are not fully converted although it does make as good a job as it can. SMX and file errors are reported via debug.txt.

If *deleteBodies* is true, and if an object is converted without error the original is deleted. If not, the original is renamed with the suffix "_orig". In each case the new object has the same name as the original.

If *deleteBodies* is false, no objects are deleted.

A warning box shows objects which have not been fully converted, as well as a list of any objects found which contain no planar faces. Objects which contain no planar faces are unchanged by the process and keep their original names.

mfl_convert_to_radbend_object(

- ByVal *deleteBodies* As Boolean

) As Boolean

Parameters

[in] deleteBodies
True to delete objects after conversion.

Return Value

Return true for success, false for failure.

Update a bend in a 3D part using tooling information from Radbend if available. Normally, information is available only when switching from Radbend to RADAN 3D. This function is not generally useful for applications other than Radbend.

If the bend is not updated then MFL_TFB_RESULT is set to an error code.

mfl_get_tooling_from_radbend(

- ByVal *ident* As String

) As Boolean

Parameters

[in] ident
Identifier of bend.

Return Value

Return true for success, false for failure.

Update all bends in a 3D part using tooling information from Radbend if available. Normally, information is available only when switching from Radbend to RADAN 3D. This function is not generally useful for applications other than Radbend.

If any bends are not updated then MFL_TFB_ALL_RESULT is set to an error code, and MFL_TFB_FAILED_INDEX_RES is set to the index of the first such bend.

mfl_get_all_tooling_from_radbend(

- ByVal *ident* As String

) As Boolean

Parameters

[in] ident
Identifier of part.

Return Value

Return true for success, false for failure.

Debug function to highlight a 3D face with the given identifier.

mfl_face_highlight(

- ByVal *ident* As String

) As Boolean

Parameters

[in] ident
Identifier of face to highlight.

Return Value

Return true for success, false for failure.

Print the current 3D view to the default printer.

mfl_print() As Boolean

Return Value

Return true for success, false for failure.

Do a Boolean operation with the two parts with the given identifiers.

mfl_boolean(

- ByVal *operation* As Long
- ByVal *blankIdent* As String
- ByVal *toolIdent* As String

) As Boolean

Parameters

[in] operation
Operation code (MFL_BOOLEAN_SUBTRACT, MFL_BOOLEAN_INTERSECT or MFL_BOOLEAN_UNITE).
[in] blankIdent
Identifier of blank part.
[in] toolIdent
Identifier of tool part.

Return Value

Return true for success, false for failure.

General 3D properties

Property	Description
ND_DIRECT_DEVELOPMENT_PATTERN As String	Get: 3D direct development pattern.
MFL_TFB_RESULT As Long	Get: Error code set by mfl_get_tooling_from_radbend when it returns false.
MFL_TFB_ALL_RESULT As Long	Get: Error code set by mfl_get_all_tooling_from_radbend when it returns false.
MFL_TFB_FAILED_INDEX_RES As Long	Get: Index of first bend that failed to update when mfl_get_all_tooling_from_radbend returns false.

Boolean operations

Property	Description
MFL_BOOLEAN_SUBTRACT As Long	Get: operation code to use with mfl_boolean.
MFL_BOOLEAN_INTERSECT As Long	Get: operation code to use with mfl_boolean.
MFL_BOOLEAN_UNITE As Long	Get: operation code to use with mfl_boolean.

Mac object: Automatic tooling

The Mac object contains a method that allows you to run automatic tooling. For more details about automatic tooling, see Automatic tooling.

Automatic tooling methods

Method	Description
pfl_auto_tool(• ByVal *sourcePattern* As String • ByVal *destinationPattern* As String • ByVal *reserved* As Double • ByRef *errorStatus* As Double) As Boolean	Run autotooling using system data settings for the current material, thickness and strategy. The material, thickness and strategy will be taken from attribute values in the Part Editor and from order text in the Nest Editor, if present, otherwise attribute values. Any problems that occurred during autotooling will be reported in the autotooling report file. This file can be found in the dat directory and is called autotooling_report.username.txt. It will be overwritten during the next run of autotooling. # Parameters [in] sourcePattern Source pattern (required in Nest mode only). [in] destinationPattern Target pattern (required in Nest mode only). [in] reserved Reserved - set to 0. [out] errorStatus Error return status. # Return Value Return true for success, false for failure.

Mac object: Part Editor tooling

The Mac object contains a method to estimate the time it takes to machine a tooled part in the Part Editor. To calculate this time, the verifier is used. To get an accurate estimate, various verify times and machine speeds need to be configured correctly. See The Time Estimates page of the Machine Configuration Editor and the Times and Speeds, Punch Times, Operation Times and Accelerations pages underneath it.

Part Editor tooling methods

Method	Description
pfl_get_run_time(• ByVal *reserved1* As Double • ByRef *errorMessage* As String • ByRef *runTime* As Double • ByRef *reserved2* As Double) As Boolean	Calculates the run time for the current part in the Part Editor. The calculation is based on a layout of a single instance of the part nested at its current orientation. If this fails, the calculation is based on the part nested at 90 degrees to the current orientation. A layout will only be made with the part at an allowable nest orientation. The run time is calculated based on the existing tooling. However, if the part is not tooled, the run time will be calculated based on tooling added to the part on the nest using automatic tooling with settings for the current machine, material, thickness and strategy. This function must be run from the Part Editor. It can be used in either Drafting or Tooling mode. # Parameters [in] reserved1 Reserved - set to 0. [out] errorMessage Error message string. [out] runTime

Run time (minutes).
[out] reserved2
Reserved.

Return Value

Return true for success, false for failure.

Mac object: Automatically adding embedded part removal

The Mac object contains a method that allows you to automatically add embedded part removal to parts in the Part Editor. For more details about embedded part removal, see Embedded part removal.

Automatic part removal methods

Method	Description
apr_auto_add_part_rems(• ByVal *reserved1* As String • ByVal *reserved2* As String • ByRef *showGraphics* As Boolean • ByRef *interactive* As Boolean • ByRef *reserved3* As Double • ByRef *errorStatus* As Long) As Boolean	Automatically add part removals to the part in the Part Editor.

Parameters

[in] reserved1
Reserved - set to empty string.
[in] reserved2
Reserved - set to empty string.
[in,out] showGraphics
Show part removal device graphics on the screen.
[in,out] interactive
Display dialog boxes when necessary.
[out] reserved3
Reserved - set to 0.
[out] errorStatus
Return status (set on failure).

Return Value

Return true for success, false for failure.

Mac object: Single part nesting

The Mac object contains methods and properties that allow you to use the single part nester to calculate sheet utilisation. For more details about single part nesting, see Introduction to single part nesting.

Single part nesting methods

Method	Description
lay_calculate_utilisations(• ByVal *useSystemDataClearances* As Boolean • ByVal *calculateCompSize* As Boolean	Calculate the utilisations for the single part nests specified by the various LAY_ properties (e.g. LAY_COMP_NAME, LAY_SHEET_SOURCE, etc). Access results using lay_get_utilisation(). Return number of utilisations calculated.

Parameters

[in] useSystemDataClearances
Use clearance values from system data (instead of using the LAY_ clearance property values, e.g. LAY_XYGAP, etc).
[in] calculateCompSize
Calculate the part size and area (instead of using the LAY_ size property values,

) As Long

e.g. LAY_COMP_XSIZE, etc).

Return Value

Return the number of utilisation results calculated.

lay_clear_properties() As Boolean

Clear the configuration and result properties used by single part nesting. Return properties to default values.
After this method has been called, the property values will be:

- LAY_COMP: "-"
- LAY_ORIENTATION: LAY_ORIENT_ANY
- LAY_COMP_XSIZE, LAY_COMP_YSIZE: 100x100
- LAY_COMP_AREA: 0
- LAY_MATERIAL, LAY_THICKNESS, LAY_THICK_UNITS: "-" 1mm
- LAY_STRATEGY: ""
- LAY_SHEET_SOURCE: LAY_SHEETS_STANDARD
- LAY_SHEETX, LAY_SHEETY, LAY_SHEET_UNITS: 1000x1000mm
- LAY_CLAMPSTRIP: 1
- All clearances and gaps (LAY_MIN_DAT, etc): 10
- LAY_BATCH: 100
- LAY_OVERPRODUCE: 1
- LAY_TRUE_SHAPE: 0
- All results (LAY_TOTAL, etc): 0

Return Value

Return true for success, false for failure.

lay_get_utilisation(

- ByVal *index* As Long

) As Boolean

Set the various LAY_ result properties (LAY_TOTAL, LAY_SHEET_UTIL, etc) for the utilisation with the given index (1-based). Utilisations are calculated using lay_calculate_utilisations(). Return true if results exist, otherwise false.

Parameters

[in] index
Index (1-based) into array of utilisation results.

Return Value

Return true if the result properties have been set, otherwise false.

Allowed part orientations

Property	Description
LAY_ORIENT_0 As Long	Get: Single part nesting: Allowed part orientation: 0 degrees.
LAY_ORIENT_0_180 As Long	Get: Single part nesting: Allowed part orientation: 0 and 180 degrees.
LAY_ORIENT_0_90 As Long	Get: Single part nesting: Allowed part orientation: 0 and 90 degrees.
LAY_ORIENT_0_90_180_270 As Long	Get: Single part nesting: Allowed part orientation: 0, 90, 180 and 270 degrees (only 0 and 90 degrees are used if the part is treated as rectangular).
LAY_ORIENT_0_XOR_90 As Long	Get: Single part nesting: Allowed part orientation: 0 or 90 degrees, whichever gives the best utilisation.
LAY_ORIENT_90 As Long	Get: Single part nesting: Allowed part orientation: 90 degrees.
LAY_ORIENT_90_270 As Long	Get: Single part nesting: Allowed part orientation: 90 and 270 degrees.
LAY_ORIENT_ANY As Long	Get: Single part nesting: Allowed part orientation: All orientations (only 0 and 90 degrees are used if the part is treated as rectangular).

Sheet source types

Property	Description
LAY_SHEETS_NONE As Long	Get: Single part nesting: Use this option for a single customer sheet. LAY_SHEETX, LAY_SHEETY and LAY_SHEET_UNITS must be set.
LAY_SHEETS_PRE_CUT_BLANK As Long	Get: Single part nesting: Use this option to make a pre-cut blank for the part.
LAY_SHEETS_PROJECT As Long	Get: Single part nesting: All of the sheet sizes in the Sheets list of the nest project, including those for which the remaining quantity is zero.
LAY_SHEETS_PROJECT_REMAINING As Long	Get: Single part nesting: The sheet sizes in the Sheets list of the nest project, but excluding sheets for which the remaining quantity is zero.
LAY_SHEETS_PROJECT_STANDARD As Long	Get: Single part nesting: All of the sheet sizes in the Sheets list of the nest project, and also the sheet sizes in your standard sheets file.
LAY_SHEETS_STANDARD As Long	Get: Single part nesting: The sheet sizes in your standard sheets file, as shown in the Standard Sheets Editor.

Part details

Property	Description
LAY_COMP As String	Set/get: Single part nesting: The full path of the part file (i.e. the .sym file).
LAY_MATERIAL As String	Set/get: Single part nesting: Part/sheet material.
LAY_ORIENTATION As Long	Set/get: Single part nesting: The allowed part orientation (LAY_ORIENT_0, LAY_ORIENT_0_XOR_90, etc).
LAY_STRATEGY As String	Set/get: Single part nesting: Strategy when creating a nest drawing or reading clearances from system data.
LAY_THICKNESS As Double	Set/get: Single part nesting: Part/sheet thickness.
LAY_THICK_UNITS As String	Set/get: Single part nesting: Part/sheet thickness units.

Part size (only used when not automatically calculated)

Property	Description
LAY_COMP_AREA As Double	Set/get: Single part nesting: The part area used when calculating the utilisation.
LAY_COMP_XSIZE As Double	Set/get: Single part nesting: The part size in the X direction that will be used during nesting.
LAY_COMP_YSIZE As Double	Set/get: Single part nesting: The part size in the Y direction that will be used during nesting.

Sheet details

Property	Description
LAY_SHEETX As Double	Set/get: Single part nesting: The sheet size in the X direction for a custom sheet or for the current sheet when getting utilisation results.
LAY_SHEETY As Double	Set/get: Single part nesting: The sheet size in the Y direction for a custom sheet or for the current sheet when getting utilisation results.
LAY_SHEET_SOURCE As Long	Set/get: Single part nesting: The types of sheets to use (LAY_SHEETS_STANDARD, LAY_SHEETS_PROJECT, etc).
LAY_SHEET_UNITS As String	Set/get: Single part nesting: Size units for the custom sheet or for the current sheet when getting utilisation results.

Clearances (only used when not automatically using system data values)

Property	Description
LAY_CLAMPSTRIP As Boolean	Set/get: Single part nesting: Clearances: Use clamp strip.
LAY_CLAMPWIDTH As Double	Set/get: Single part nesting: Clearances: When using a clamp strip, this is the minimum clearance between the parts and the clamped edge of the sheet.
	Set/get: Single part nesting: Clearances: The minimum clearance between the parts and

LAY_MIN_DAT As Double	the vertical edge of the sheet that is nearest the mdatum.
LAY_MIN_NONDAT As Double	Set/get: Single part nesting: Clearances: The minimum clearance between the parts and the vertical edge of the sheet that is opposite the mdatum.
LAY_MIN_UNCLAMPED As Double	Set/get: Single part nesting: Clearances: The minimum clearance between the parts and the unclamped edge of the sheet (i.e. the edge opposite the clamped edge).
LAY_OPT_UNCLAMPED As Double	Set/get: Single part nesting: Clearances: The optimum clearance at the unclamped edge of the sheet. It should be greater than the minimum unclamped clearance.
LAY_OPT_VERT As Double	Set/get: Single part nesting: Clearances: The optimum clearance at the vertical edges of the sheet. This should be greater than the minimum datum and non-datum vertical edge clearances.
LAY_XGAP As Double	Set/get: Single part nesting: Clearances: The clearance that will be left in the X direction between two parts.
LAY_XYGAP As Double	Set/get: Single part nesting: Clearances: In true shaped nesting, the XY gap is the clearance that will be left between the parts. The X gap and Y gap are not used.
LAY_YGAP As Double	Set/get: Single part nesting: Clearances: The clearance that will be left in the Y direction between two parts.

Single part nesting options

Property	Description
LAY_BATCH As Long	Set/get: Single part nesting: The total number of parts you want to make (not just the number of parts on one sheet). This number affects the LAY_BATCH_UTIL and LAY_SHEETS_NEEDED results.
LAY_OVERPRODUCE As Boolean	Set/get: Single part nesting: Select this option to completely fill each sheet with as many parts as possible. If the option is off, the number of parts nested will be no more than LAY_BATCH.
LAY_TRUE_SHAPE As Boolean	Set/get: Single part nesting: Select this option if you want to allow the part's true shape to be used for nesting. If this option is off, the part will be treated as rectangular, and will be nested at 0 and 90 only.

Utilisation result for one sheet size

Property	Description
LAY_BATCH_UTIL As Double	Set/get: Single part nesting: Results: The overall material utilisation for the sheets needed to make LAY_BATCH parts. This is the combined area of LAY_BATCH parts, divided by the combined area of the sheets.
LAY_SHEETS_NEEDED As Long	Set/get: Single part nesting: Results: The number of sheets needed to make LAY_BATCH parts. This number is influenced by whether LAY_OVERPRODUCE is on or off.
LAY_SHEET_UTIL As Double	Set/get: Single part nesting: Results: The material utilisation percentage for a single sheet. This is the combined area of the parts nested on one sheet divided by the area of the sheet.
LAY_TOTAL As Long	Set/get: Single part nesting: Results: Total number of parts nested.
LAY_XX As Long	Set/get: Single part nesting: Results: Number of parts nested at 0/180 degrees.
LAY_XY As Long	Set/get: Single part nesting: Results: Number of parts nested at 90/270 degrees.

Mac object: Parts nesting

The Mac object contains methods and properties that allow you to add parts and sheets to the current nest schedule and run the parts nester. For more details about parts nesting, see Introduction to the parts nester. For more details about adding parts and sheets to the nest schedule, see Creating a nesting schedule and Defining the sheets to use for nesting.

Parts nesting methods

Method	Description
	Run the nester. Return a status code (return zero for success or a non-zero error code).

lay_run_nest(

- ByVal *reserved* As Long

) As Long

Parameters

[in] reserved
Reserved - set to 0.

Return Value

Return zero for success or a non-zero error code.

nst_add_part() As Long

Add a part to the Nest Schedule using the current values of the NST_ properties (NST_NAME, NST_MATERIAL, etc). Return zero for success or a non-zero error code.

Return Value

Return zero for success or a non-zero error code.

nst_add_sheet() As Long

Add a sheet to the Nest Schedule using the current values of the NST_ properties (NST_SHEET_X, NST_SHEET_Y, NST_MATERIAL, etc). Return zero for success or a non-zero error code.

Return Value

Return zero for success or a non-zero error code.

nst_additional_info(

- ByVal *label* As String
- ByVal *value* As String

) As Long

Setup another Additional Information item for the next part to be added to the Nest Schedule (return zero for success or a non-zero error code).

Parameters

[in] label
Additional info label.
[in] value
Additional info value.

Return Value

Return zero for success or a non-zero error code.

nst_delete_all_parts() As Long

Delete all part entries from the Nest Schedule (return zero for success or a non-zero error code).

Return Value

Return zero for success or a non-zero error code.

nst_delete_all_sheets() As Long

Delete all multiple sheet entries from the Nest Schedule (return zero for success or a non-zero error code).

Return Value

Return zero for success or a non-zero error code.

nst_finish_adding_parts() As Long

Update the Nest Schedule after multiple additions (return zero for success or a non-zero error code).

Return Value

Return zero for success or a non-zero error code.

Set the nest folder in the Nest Schedule (return zero for success or a non-zero error code).

nst_set_path(

- ByVal *path* As String

) As Long

Parameters

[in] path
Nest folder path.

Return Value

Return zero for success or a non-zero error code.

Reads in the Nest Schedule data ready for multiple additions (return zero for success or a non-zero error code).

nst_start_adding_parts() As Long

Return Value

Return zero for success or a non-zero error code.

Adding parts and sheets: Configuration

Property	Description
NST_BIN As Long	Set/get: Part bin number.
NST_COMMON_CUT As Long	Set/get: Part common cut method (0 - none, 1 - unrestricted, 2 - same part).
NST_EXTRA As Long	Set/get: Number of extra parts allowed.
NST_KIT As String	Set/get: Full file path of kit symbol.
NST_MATERIAL As String	Set/get: Part/sheet material.
NST_MAX_COMMON_CUT As Long	Set/get: Max common cut part group size for 'same part' common cutting.
NST_MIRROR As Boolean	Set/get: Part can be mirrored.
NST_NAME As String	Set/get: Full file path of part symbol.
NST_NUMBER As Long	Set/get: Number of parts required or sheets available.
NST_ORIENT As Long	Set/get: Allowed part orientation (NSM_ORIENTATION_0, NSM_ORIENTATION_ANY, etc).
NST_ORIENTATION_MODE As Long	Set/get: Part orientation mode (NSM_ORIENTATION_MODE_FREE, NSM_ORIENTATION_MODE_SINGLE, etc).
NST_PICKING_CLUSTER As Boolean	Set/get: Part should be nested in a picking cluster.
NST_PRIORITY As Long	Set/get: Part priority (1 - highest, 9 - lowest).
NST_SHEET_UNITS As String	Set/get: Sheet size units, mm or in.
NST_SHEET_X As Double	Set/get: Sheet size X.
NST_SHEET_Y As Double	Set/get: Sheet size Y.
NST_STRATEGY As String	Set/get: Part strategy.
NST_THICKNESS As Double	Set/get: Part/sheet thickness.
NST_THICK_UNITS As String	Set/get: Part/sheet thickness units (mm, in or swg).

Allowed part orientations

Property	Description
NSM_ORIENTATION_0 As Long	Get: Part nesting: Allowed part orientation: 0 degrees. (Equivalent to orientation 1)
NSM_ORIENTATION_90 As Long	Get: Part nesting: Allowed part orientation: 90 degrees. (Equivalent to orientation 2)
NSM_ORIENTATION_0_90 As Long	Get: Part nesting: Allowed part orientation: 0 or 90 degrees. (Equivalent to orientation 3)
NSM_ORIENTATION_0_180 As	Get: Part nesting: Allowed part orientation: 0 or 180 degrees. (Equivalent to

Long	orientation 5)
NSM_ORIENTATION_90_270 As Long	Get: Part nesting: Allowed part orientation: 90 or 270 degrees. (Equivalent to orientation 6)
NSM_ORIENTATION_0_90_180_270 As Long	Get: Part nesting: Allowed part orientation: 0, 90, 180 or 270 degrees. (Equivalent to orientation 7)
NSM_ORIENTATION_ANY As Long	Get: Part nesting: Allowed part orientation: Any orientation. (Equivalent to orientation 8)

Part orientation modes

Property	Description
NSM_ORIENTATION_MODE_FREE As Long	Get: Part nesting: Part orientation mode: Free mix of allowed orientations permitted.
NSM_ORIENTATION_MODE_SINGLE_OPPOSITE As Long	Get: Part nesting: Part orientation mode: Only a single allowed orientation and its opposite (if allowed) permitted.
NSM_ORIENTATION_MODE_SINGLE As Long	Get: Part nesting: Part orientation mode: Only a single allowed orientation permitted.

Mac object: Nest projects

The Mac object contains various methods for managing Nest Projects. For more details about Nest Projects, see Introduction to nest projects.

Nest Project methods

Method	Description
prj_add_nest(• ByVal *nestName* As String) As Boolean	Add a nest to the open project. The nest name must match the project's nest name template, and the file must already be in the project's nests folder. **Parameters** [in] nestName Name of the nest to add to the project. This should be the nest filename without the full directory path or ".drg" extension. **Return Value** Returns true for success (or if the nest is already present in the project), false for failure.
prj_add_part() As Long	Add a part or kit to the open project using the current PRJ_PART property values (e.g. PRJ_PART_FILENAME, etc). See the table below for details of the PRJ_PART properties which act as parameters to this method. **Return Value** Returns the number of distinct parts added (or 0 for failure).
prj_add_sheet() As Long	Add a sheet to the open project using the current PRJ_SHEET property values (e.g. PRJ_SHEET_MATERIAL, etc). See the table below for details of the PRJ_SHEET properties which act as parameters to this method. **Return Value**

Returns the number of distinct parts added (or 0 for failure).

prj_clear_additional_info() As Boolean

Clear the additional info to be appended when adding a new part.

Return Value

Returns true.

prj_clear_part_data() As Boolean

Clear the data for adding a new part, returning them to default values. This function does not clear the current record of additional info to be added.
After this function has been called, the parameters to **prj_add_part** will represent a part with:

- 1 copy required, and no extras allowed
- Material set to "-", at 1 mm or 0.1 inches thick
- No strategy set
- All orientations allowed
- Mirroring not allowed
- Common cutting off, but limited to 2 parts if set to "same part"
- No picking cluster required
- Nesting priority 5
- Bin number set to 0
- Not excluded from automatic nesting

Return Value

Returns true.

prj_clear_sheet_data() As Boolean

Clear the data for adding a new sheet, returning them to default values. After this function has been called, the parameters to **prj_add_sheet** will represent a sheet with:

- 1 copy available
- Material set to "", at 1 mm or 0.1 inches thick
- Size set to 1000 mm x 1000 mm or 40 in x 40 in
- Blank stock ID
- Nesting priority 5

Return Value

Returns true.

prj_close() As Boolean

Close the currently open Nest Project.

Return Value

Returns true for success, false for failure.

prj_delete_nest(

- ByVal *nestName* As String

) As Boolean

Delete a nest from the open project. The nest file will also be deleted from the project's nests folder.

Parameters

[in] nestName
: Name of the nest to delete from the project. This should be the nest filename without the full directory path or ".drg" extension.

Return Value

Returns true for success, false for failure.

Return the full path of the currently open Nest Project file, or an empty string if no

project is open.

prj_get_file_path() As String

Return Value

Returns the full path of the currently open Nest Project file, or an empty string if no project is open.

Return true if the currently open Nest Project has been edited since it was last saved.

prj_is_edited() As Boolean

Return Value

Returns true if the currently open Nest Project has been edited since it was last saved.

Return true if a Nest Project is currently open, otherwise false.

prj_is_open() As Boolean

Return Value

Returns true if a Nest Project is currently open, otherwise false.

Start a new Nest Project, closing the currently open project if required. The nests and remnants folders are always created as subfolders of the project folder. If you want to use different nests or remnants folders, use **prj_new_project()**.

Parameters

prj_new(

- ByVal *name* As String
- ByVal *location* As String
- ByVal *remnantsSheets* As String
- ByVal *useCurrent* As Boolean
- ByVal *resetNestNumber* As Boolean

) As Boolean

[in] name
: Name of new project.

[in] location
: Folder to contain project folder.

[in] remnantsSheets
: Folder of remnant sheets.

[in] useCurrent
: Initialise from the currently open project (if one is open), or from the current schedule.

[in] resetNestNumber
: When initialising from the current project/schedule, reset next nest number to 1, otherwise use the next nest number from the current project/schedule.

Return Value

Returns true for success, false for failure.

Create (and open) a new nest project using the current PRJ_PROJECT property values (e.g. PRJ_PROJECT_NAME, etc).

See the table below for details of the PRJ_PROJECT properties which act as parameters to this method.

prj_new_project() As Boolean

Return Value

Returns true for success, false for failure.

Open an existing Nest Project.

Parameters

prj_open(

- ByVal *file* As String

[in] file
: Full path of project file.

) As Boolean

Return Value

Returns true for success, false for failure.

Generate and output a nest project report for the open project.

Parameters

prj_output_report(

- ByVal *reportName* As String
- ByVal *filePath* As String
- ByVal *fileType* As Long
- ByRef *errorMessage* As String

) As Boolean

[in] reportName
Name of the report to output.
[in] filePath
Path and filename to output the report to.
[in] fileType
File type to use for report output. (REPORT_TYPE_PDF, REPORT_TYPE_CSV, etc.: see Report types in Finish nesting
[in] errorMessage
Error message, set on failure to output the report.

Return Value

Returns true for success; otherwise returns false and sets errorMessage.

Remove a piece of additional info to be appended when adding a new part, by label.

Parameters

prj_remove_additional_info(

- ByVal *label* As String

) As Boolean

[in] label
Label for the additional into entry to remove.

Return Value

Returns true if the label was present, false if not. The label will be absent after return either way.

Remove all parts from the open project.

prj_remove_all_parts() As Long

Return Value

Returns the number of distinct parts removed.

Remove all sheets from the open project.

prj_remove_all_sheets() As Long

Return Value

Returns the number of distinct sheets removed.

Save the currently open Nest Project to its current file path.

prj_save() As Boolean

Return Value

Returns true for success, false for failure.

Set a (label, value) pair of additional info to be appended when adding a new part.

Parameters

prj_set_additional_info(

[in] label

- ByVal *label* As String
- ByVal *value* As String

) As Boolean

Label for the additional info entry.

[in] value
Value for the additional info entry.

Return Value

Returns true.

Set the value of one of the project's numeric properties (e.g. PRJ_PROPERTY_MATCH_MATERIAL).

prj_set_numeric_property(

- ByVal *projectProperty* As Long
- ByVal *value* As Double

) As Boolean

Parameters

[in] projectProperty
Property type (e.g. PRJ_PROPERTY_MATCH_MATERIAL, PRJ_PROPERTY_FIRST_NEST_NUMBER).

[in] value
Numeric property value.

Return Value

Returns true for success, false for failure.

Numeric property types

Property	Description
PRJ_PROPERTY_FIRST_NEST_NUMBER As Long	Get: Reset first nest number Nest Project property type.
PRJ_PROPERTY_MATCH_MATERIAL As Long	Get: Match material Nest Project property type.

Function parameter variables - Parameters to prj_new_project

Variable	Description
PRJ_PROJECT_CREATE_SUBFOLDER As Boolean	Set: Create a folder of LOCATION with the project's name. If true, the project's nests and remnants will be saved to subfolders of the project folder.
PRJ_PROJECT_LOCATION As String	Set: Folder to contain project folder. If left blank, defaults to the Installed Data folder.
PRJ_PROJECT_NAME As String	Set: Name to give to the new project.
PRJ_PROJECT_RESET_NEST_NUM As Boolean	Set: When initialising from the current project/schedule, reset next nest number to 1, otherwise use the next nest number from the current project/schedule.
PRJ_PROJECT_SAVE_NESTS_FOLDER As String	Set: If CREATE_SUBFOLDER is false, the folder where the project's nests should be saved. If left blank, defaults to a subfolder of LOCATION.
PRJ_PROJECT_SAVE_REMNANTS_FOLDER As String	Set: If CREATE_SUBFOLDER is false, the folder where remnants produced by the project should be saved. If left blank, remnants will not be saved.
PRJ_PROJECT_USE_CURRENT As Boolean	Set: Whether to initialise from the currently open project (if one is open), or from the current schedule.
PRJ_PROJECT_USE_REMNANTS_FOLDER As String	Set: Folder containing remnant sheets for the project to use. If left blank, remnants will not be used.

Function parameter variables - Parameters to prj_add_part

Variable	Description

PRJ_PART_BIN As Long	Set: Bin number for the added part. (0-500)
PRJ_PART_COMMON_CUT As Long	Set: Common cutting method for the added part. • 0: No common cutting • 1: Unrestricted common cutting • 2: Same part common cutting
PRJ_PART_CUSTOM_COLOUR_BLUE As Long	Set: Blue component of the user-defined fill colour for the added part (0 - 255, or negative for none).
PRJ_PART_CUSTOM_COLOUR_GREEN As Long	Set: Green component of the user-defined fill colour for the added part (0 - 255, or negative for none).
PRJ_PART_CUSTOM_COLOUR_RED As Long	Set: Red component of the user-defined fill colour for the added part (0 - 255, or negative for none).
PRJ_PART_EXCLUDE As Boolean	Set: Whether to exclude the added part from the automatic nester.
PRJ_PART_EXTRA_ALLOWED As Long	Set: Allowed extra quantity of the added part. (0-99999)
PRJ_PART_FILENAME As String	Set: Full path to the symbol file containing the single part to be added.
PRJ_PART_KIT_FILENAME As String	Set: If FILENAME is blank, full path to the symbol file containing the kit of parts to be added.
PRJ_PART_MATERIAL As String	Set: Material of the added part.
PRJ_PART_MAX_COMMON_CUT As Long	Set: Maximum common cut group size for "same part" common cutting. (0-999999)
PRJ_PART_MIRROR As Boolean	Set: Whether the added part can be mirrored.
PRJ_PART_NEST_MODE As Long	Set: Nest mode for the added part. (PRJ_PART_MULTI, PRJ_PART_PRE_CUT or PRJ_PART_SINGLE)
PRJ_PART_NUMBER_REQUIRED As Long	Set: Required quantity of the added part. (0-999999)
PRJ_PART_ORIENT As Long	Set: Allowed orientations of the added part. (NSM_ORIENTATION_0, NSM_ORIENTATION_ANY, etc)
PRJ_PART_ORIENTATION_MODE As Long	Set: Orientation mode of the added part. (NSM_ORIENTATION_MODE_FREE, NSM_ORIENTATION_MODE_SINGLE, etc)
PRJ_PART_PICKING_CLUSTER As Boolean	Set: Whether the added part should be put in a picking cluster.
PRJ_PART_PRIORITY As Long	Set: Priority level for the added part. (1 [high] - 9 [low])
PRJ_PART_STRATEGY As String	Set: Strategy for the added part.
PRJ_PART_THICKNESS As Double	Set: Thickness of the added part. (0-999.99)
PRJ_PART_THICK_UNITS As String	Set: Units for the added part's thickness. ('mm', 'in' or 'swg')

Function parameter constants - Parameters to prj_add_part

Constant	Description
PRJ_PART_MULTI As Long	Constant value for PRJ_PART_NEST_MODE, indicating that the part is to be nested using the multi-part nester.
PRJ_PART_PRE_CUT As Long	Constant value for PRJ_PART_NEST_MODE, indicating that the part is to be nested as a pre-cut blank.
PRJ_PART_SINGLE As Long	Constant value for PRJ_PART_NEST_MODE, indicating that the part is to be nested using the single-part nester.

Function parameter variables - Parameters to prj_add_sheet

Variable	Description
PRJ_SHEET_MATERIAL As String	Set: Material of the added sheet.
PRJ_SHEET_NUM_AVAILABLE As Long	Set: Number available of the added sheet. (PRJ_SHEET_UNLIMITED for no limit)
PRJ_SHEET_PRIORITY As Long	Set: Priority level for the added sheet. (1 [high] - 9 [low])
PRJ_SHEET_SIZE_UNITS As	

String	Set: Units for the added sheet's size. ('mm' or 'in')
PRJ_SHEET_STOCK_ID As String	Set: Stock ID for the added sheet.
PRJ_SHEET_THICKNESS As Double	Set: Thickness of the added sheet. (0-999.99)
PRJ_SHEET_THICK_UNITS As String	Set: Units for the added sheet's thickness. ('mm', 'in' or 'swg')
PRJ_SHEET_X_SIZE As Double	Set: Added sheet's size in X.
PRJ_SHEET_Y_SIZE As Double	Set: Added sheet's size in Y.

Function parameter constants - Parameters to prj_add_sheet

Constant	Description
PRJ_SHEET_UNLIMITED As Long	Constant value for PRJ_SHEET_NUM_AVAILABLE, representing an unlimited number of available sheets.

Mac object: Finish nesting

The Mac object contains various methods and properties that are related to the finish nesting process (e.g. adding annotation, saving remnants, etc). For more details about the finish nesting process, see Finishing manual nesting, The Finalise Nest tab of the Finish Manual Nesting dialog and The Remnants tab of the Finish Manual Nesting dialog.

Finish nesting related methods

Method	Description
pfl_finish_nesting(• ByVal *updateAnnotation* As Boolean • ByVal *updateSchedule* As Boolean • ByVal *reserved* As Double) As Boolean	Update the annotation and parts list in the same way as the Finish Manual Nesting dialog without showing the dialog. **Parameters** [in] updateAnnotation Update annotation. [in] updateSchedule Update current nest schedule (ignored if a Nest Project is open). [in] reserved Reserved - set to 0. **Return Value** Return true for success, false for failure.
pfl_save_remnants(• ByVal *saveFolder* As String • ByVal *areaWorthSaving* As Double • ByVal *saveRectRemnantsOnly* As Boolean • ByVal *alignRectsWithClampStrip* As Boolean) As Boolean	Save remnant drawings for the current drawing in the same way as the Finish Manual Nesting dialog. Sets the number of remnants attribute (which can be queried to find out how many drawings were saved). Return true for success, false for failure. **Parameters** [in] saveFolder Folder for saving remnants. [in] areaWorthSaving Minimum remnant area in sq.m or sq.ft. [in] saveRectRemnantsOnly Save rectangular remnants only. [in] alignRectsWithClampStrip Align rectangular remnants with clamp strip.

Return Value

Return true for success, false for failure.

Nest utilities

Method	Description
	Get the position of the sheet and nest in the drawing (as NEST_SHEET_BL_X, NEST_BL_X, etc).

nst_calculate_extents(

- ByVal *considerTooling* As Boolean

) As Boolean

Parameters

[in] considerTooling
Consider tooling as well as geometry.

Return Value

Return true for success, false for failure.

Reporting

Method	Description
	Generate and output a setup sheet report for the open nest drawing.

stp_output_report(

- ByVal *reportName* As String
- ByVal *filePath* As String
- ByVal *fileType* As Long
- ByRef *errorMessage* As String

) As Boolean

Parameters

[in] reportName
Name of the report to output.
[in] filePath
Path and filename to output the report to.
[in] fileType
File type to use for report output. (REPORT_TYPE_PDF, REPORT_TYPE_CSV, etc.: see Report types
[in] errorMessage
Error message, set on failure to output the report.

Return Value

Returns true for success; otherwise returns false and sets errorMessage.

Nest extents results

Property	Description
NEST_BL_X As Double	Get: nst_calculate_extents: Bottom left X of nest.
NEST_BL_Y As Double	Get: nst_calculate_extents: Bottom left Y of nest.
NEST_SHEET_BL_X As Double	Get: nst_calculate_extents: Bottom left X of sheet.
NEST_SHEET_BL_Y As Double	Get: nst_calculate_extents: Bottom left Y of sheet.
NEST_SHEET_TR_X As Double	Get: nst_calculate_extents: Top right X of sheet.
NEST_SHEET_TR_Y As Double	Get: nst_calculate_extents: Top right Y of sheet.
NEST_TR_X As Double	Get: nst_calculate_extents: Top right X of nest.
NEST_TR_Y As Double	Get: nst_calculate_extents: Top right Y of nest.

Report types

Property	Description
REPORT_TYPE_CSV As Long	Get: Reporting: Output report as a CSV (Comma Separated Values) file.
REPORT_TYPE_DOCX As Long	Get: Reporting: Output report as a DOCX (Microsoft Word Document) file.
REPORT_TYPE_HTML As Long	Get: Reporting: Output report as an HTML (HyperText Markup Language) file.
REPORT_TYPE_MHT As Long	Get: Reporting: Output report as an MHTML (MIME HTML) file.
REPORT_TYPE_PDF As Long	Get: Reporting: Output report as a PDF (Adobe Acrobat Document) file.
REPORT_TYPE_RTF As Long	Get: Reporting: Output report as an RTF (Rich Text Format) file.
REPORT_TYPE_TXT As Long	Get: Reporting: Output report as a TXT (Text) file.
REPORT_TYPE_XLS As Long	Get: Reporting: Output report as an XLS (Microsoft Excel 97-2003 Worksheet) file.
REPORT_TYPE_XLSX As Long	Get: Reporting: Output report as an XLSX (Microsoft Excel Worksheet) file.
REPORT_TYPE_BMP As Long	Get: Reporting: Output report as a BMP (Bitmap) format image file.
REPORT_TYPE_EMF As Long	Get: Reporting: Output report as an EMF (Enhanced Windows Metafile) format image file.
REPORT_TYPE_GIF As Long	Get: Reporting: Output report as a GIF (Graphics Interchange Format) format image file.
REPORT_TYPE_JPEG As Long	Get: Reporting: Output report as a JPEG (Joint Photographic Experts Group) format image file.
REPORT_TYPE_PNG As Long	Get: Reporting: Output report as a PNG (Portable Network Graphics) format image file.
REPORT_TYPE_TIFF As Long	Get: Reporting: Output report as a TIFF (Tag Image File Format) format image file.
REPORT_TYPE_WMF As Long	Get: Reporting: Output report as a WMF (Windows Metafile) format image file.

Mac object: Sheet scrapping and offcuts

The Mac object contains methods that allow you to automatically add tooling to a nest to scrap the sheet and produce offcuts. For more details about sheet scrapping and offcuts, see Introduction to sheet scrapping and offcuts.

Scrapping and offcut methods

Method	Description
pfl_sheet_cut() As Boolean	Apply sheet scrapping and offcuts to the current nest using settings from the MDB. **Return Value** Return true for success, false for failure.
pfl_sheet_offcut(• ByVal *idealWidths* As String • ByVal *idealHeights* As String • ByVal *reserved* As String • ByVal *tool* As Double • ByVal *pen* As Long • ByVal *cutFromCentre* As Boolean • ByVal *overlap* As Double • ByVal *type* As Long • ByVal *minSize* As Double	Do sheet offcutting by profiling with given parameters. (Type: 0 - vertical, 1 - horizontal, 2 - vertical then horizontal, 3 - horizontal then vertical). **Parameters** [in] idealWidths List of ideal offcut widths. [in] idealHeights List of ideal offcut heights. [in] reserved Reserved - set to empty string. [in] tool Tool to use. [in] pen Pen to use. [in] cutFromCentre Whether to cut in two halves from centre out. [in] overlap Overlap between two halves of lines. [in] type Type of offcut lines to add (0 - vertical, 1 - horizontal, 2 - vertical then horizontal, 3 - horizontal then vertical). [in] minSize

- ByVal *partGap* As Double
- ByVal *leftGap* As Double
- ByVal *rightGap* As Double
- ByVal *topGap* As Double
- ByVal *bottomGap* As Double
- ByVal *edgeCutTool* As Double
- ByVal *edgeCutPen* As Long
- ByVal *edgeCutDist* As Double

) As Boolean

Minimum width or height of offcut to produce.
[in] partGap
Gap to leave at edges of parts.
[in] leftGap
Gap to leave at left of sheet.
[in] rightGap
Gap to leave at right of sheet.
[in] topGap
Gap to leave at top of sheet.
[in] bottomGap
Gap to leave at bottom of sheet.
[in] edgeCutTool
Tool to use for edge cuts.
[in] edgeCutPen
Pen to use for edge cuts.
[in] edgeCutDist
Distance on sheet where edge cuts finish.

Return Value

Return true for success, false for failure.

Do sheet scrapping with the given parameters.

Parameters

sheet_scrap(

- ByVal *xSpacing* As Double
- ByVal *ySpacing* As Double
- ByVal *leftGap* As Double
- ByVal *rightGap* As Double
- ByVal *topGap* As Double
- ByVal *bottomGap* As Double
- ByVal *partGap* As Double
- ByVal *overlap* As Double
- ByVal *tool* As Double
- ByVal *pen* As Long

) As Boolean

[in] xSpacing
Spacing of vertical cuts.
[in] ySpacing
Spacing of horizontal cuts.
[in] leftGap
Gap at left of sheet.
[in] rightGap
Gap at right of sheet.
[in] topGap
Gap at top of sheet.
[in] bottomGap
Gap at bottom of sheet.
[in] partGap
Gap at edge of parts.
[in] overlap
Overlap at slot mid-points.
[in] tool
Tool to use.
[in] pen
Pen to use.

Return Value

Return true for success, false for failure.

Mac object: Order mode

The Mac object contains various Order mode related properties that are continually updated whenever the current state changes in Order mode (e.g. when you move up/down the order text). For more details about Order mode, see Introduction to Order mode and Order mode keystroke reference.

Order mode properties

Property	Description
ORD_CURS_X As Double	Get: Cusor position X.
ORD_CURS_Y As Double	Get: Cusor position Y.
ORD_CUR_LINE As Long	Get: Current order text line.
ORD_CUR_TEXT As String	Get: Text on current order text line.
ORD_MAX_LINE As Long	Get: Number of order text lines.
ORD_REDOS As Long	Get: Number of redos available.
ORD_SHEET_OFFSET_X As Double	Get: Sheet offset in X.
ORD_SHEET_OFFSET_Y As Double	Get: Sheet offset in Y.
ORD_SHEET_TO_WR_X As Double	Get: Sheet to working range in X.
ORD_SHEET_TO_WR_Y As Double	Get: Sheet to working range in Y.
ORD_SHEET_X As Double	Get: Sheet size in X.
ORD_SHEET_Y As Double	Get: Sheet size in Y.
ORD_STATE_ALL_TOOLS As String	Get: Space separated list of all tools.
ORD_STATE_AVOID As Long	Get: Avoid area type mask (0x1 - forms, 0x2 - cutouts, 0x4 - profiles).
ORD_STATE_FEED As Double	Get: Current feed rate.
ORD_STATE_INDEX As String	Get: Space separated list of active angles.
ORD_STATE_LIFT As Boolean	Get: Head in raised position.
ORD_STATE_PMODE As Long	Get: Punch mode type (0 - off, 1 - delay, 2 - foot, 3 - nibble, 4 - high speed nibble, 5 - punch).
ORD_STATE_SUB As Boolean	Get: Subroutine state.
ORD_STATE_TOOL As String	Get: Space separated list of active tools.
ORD_STATE_TURN As String	Get: Turn state ('O' - over, 'R' - round).
ORD_TOOL_OFFSET_X As Double	Get: Tool offset in X.
ORD_TOOL_OFFSET_Y As Double	Get: Tool offset in Y.
ORD_UNDOS As Long	Get: Number of undos available.
ORD_UNL_GTYPE_BIN_INFO As Long	Get: General feature type for unload bin info.
ORD_UNL_GTYPE_PICK_INFO As Long	Get: General feature type for unload pick info.
ORD_UNL_GTYPE_PTN_SCALE As Long	Get: General feature type for unload pattern scale.
ORD_WR_X As Double	Get: Working range in X.
ORD_WR_Y As Double	Get: Working range in Y.

Mac object: Block compiling

The Mac object contains methods that allow you to compile blocks for the current nest. For more details about compiling blocks, see Compiling a program of blocks.

Block compiler methods

Method	Description
ord_run_blockmaker() As Boolean	Run the blockmaker (you must be Order mode).

Return Value

ord_run_blockmaker_silently() As Boolean	Return true for success, false for failure. Run the blockmaker silently (you must be Order mode). **Return Value** Return true for success, false for failure.

Mac object: Program verification

The Mac object contains methods that allow you to run the verifier. For more details about program verification, see Introduction to the program verifier.

Verifier methods

Method	Description
run_verifier() As Long	Run the verifier (you must be in Verifier mode). Return zero for success, non-zero error code for failure. **Return Value** Return zero for success, non-zero error code for failure.
run_verifier_silently() As Long	Run the verifier silently (you must be in Verifier mode). Return zero for success, non-zero error code for failure. **Return Value** Return zero for success, non-zero error code for failure.

Mac object: Machine configuration

The Mac object contains some methods that allow you to query the current machine configuration and to determine if various features are supported by the current block compiler (e.g. strategies). For more details about the configuration of the current machine, see Editing the Radpunch/Radprofile system data.

Note: To change the current machine, you can use the ']' tooling mode keystroke command.

Machine configuration methods

Method	Description
get_num_machines() As Long	Get the number of supported machines. **Return Value** Return the number of supported machines.
machine_type(• ByVal *machineId* As Long) As Long	Get the type (PCC_PUNCHING, PCC_COMBINATION, PCC_LASER, PCC_NONE) of the machine with the given id, or the current machine if the id is 0 or -1. **Parameters** [in] machineId Machine id. **Return Value**

Return the machine type (PCC_PUNCHING, PCC_COMBINATION, PCC_LASER, PCC_NONE).

Get the id of the current machine.

pcc_get_current_mc_id() As Long

Return Value

Return the id of the current machine.

Get the default strategy for the current machine and given material and thickness. Return true if a default strategy value is returned, otherwise false.

pfl_get_default_mdb_strategy(

- ByVal *thickness* As Double
- ByVal *thicknessUnits* As String
- ByVal *material* As String
- ByRef *strategy* As String

) As Boolean

Parameters

[in] thickness
Thickness value.
[in] thicknessUnits
Thickness units ('mm' or 'in').
[in] material
Material.
[out] strategy
Default strategy.

Return Value

Return true if a default strategy value is returned, otherwise false.

Block compiler support methods

Method	Description
bki_bevel_cutting_available() As Boolean	Return true if the blockmaker supports bevel cutting.
bki_flycutting_available() As Boolean	Return true if the blockmaker supports fly cutting.
bki_get_strategy_levels(ByRef *numLevels* As Long, ByRef *levels* As String) As Boolean	Get the strategy levels string from the blockmaker.

bki_bevel_cutting_available() As Boolean

Return Value

Return true if the blockmaker supports bevel cutting.

Return true if the blockmaker supports fly cutting.

bki_flycutting_available() As Boolean

Return Value

Return true if the blockmaker supports fly cutting.

Get the strategy levels string from the blockmaker.

bki_get_strategy_levels(

- ByRef *numLevels* As Long
- ByRef *levels* As String

) As Boolean

Parameters

[out] numLevels
Number of strategy levels (e.g. 1 for 'oxygen', 2 for 'oxygen/lens2', etc).
[out] levels
Strategy levels (e.g. 'oxygen' or 'oxygen/lens2', etc).

Return Value

Return true for success, false for failure.

Mac object: Tool library

The Mac object contains a number of properties and methods that allow you to manage the tools in your Tool Library. For more details about the Tool Library, see Editing the Tool Library.

Tool library methods

Method	Description
mtm_add_ai(• ByVal *mtn* As Double • ByVal *label* As String • ByVal *value* As String) As Boolean	Adds a label/value pair to the additional information for the given tool. **Parameters** [in] mtn Tool number. [in] label Additional info label. [in] value Additional info value. **Return Value** Return true for success, false for failure.
mtm_get(• ByVal *mtn* As Double) As Boolean	Get the details (e.g. MTNUM, MTX, MTY, etc) of the tool or multi-tool with the given tool number. **Parameters** [in] mtn Tool number. **Return Value** Return true for success, false for failure.
mtm_get_ai(• ByVal *mtn* As Double • ByVal *label* As String) As String	Gets the value of the additional information packet with the given label for the given tool. **Parameters** [in] mtn Tool number. [in] label Additional info label. **Return Value** Additional info value.
mtm_get_ai_label(• ByVal *mtn* As Double • ByVal *index* As Long) As String	Gets the label of the additional information packet with the given (1 based) index for the given tool. **Parameters** [in] mtn Tool number. [in] index Zero based index of additional info packet to get.

Return Value

Additional info label.

Gets the value of the additional information packet with the given (1 based) index for the given tool.

mtm_get_ai_value(

- ByVal *mtn* As Double
- ByVal *index* As Long

) As String

Parameters

[in] mtn
Tool number.
[in] index
Zero based index of additional info packet to get.

Return Value

Additional info value.

Gets the number of additional information packets for the given tool.

mtm_get_num_ai(

- ByVal *mtn* As Double

) As Long

Parameters

[in] mtn
Tool number.

Return Value

Gets the number of additional information packets for the given tool.

Gets a format string corresponding to the given tool name format type.

mtm_get_tool_name_format(

- ByVal *formatType* As Long

) As String

Parameters

[in] formatType
Format type.

Return Value

Format string (for use in C and MAC language print format statements).

Extract and return the tool number from the given symbol name or feature ID.

mtm_get_tool_number(

- ByVal *name* As String

) As Double

Parameters

[in] name
Symbol name or feature id.

Return Value

Tool number.

Return true if the given tool number is the tool number for a multi-tool, otherwise false.

mtm_isa_multi_tool_mtn(

Parameters

[in] mtn

- ByVal *mtn* As Double

) As Boolean

Tool number

Return Value

Return true if the given tool number is the tool number for a multi-tool, otherwise false.

Return true if the supplied path is a tool symbol name or a path to a tool symbol instance, otherwise return false. Optionally ignore multi-tools (i.e. return false if the supplied path is for a multi-tool symbol). See also isa_tool_sym().

mtm_isa_tool_sym(

- ByVal *nameOrPath* As String
- ByVal *ignoreMultiTools* As Boolean

) As Boolean

Parameters

[in] nameOrPath
Symbol name or path to test.
[in] ignoreMultiTools
Ignore multi-tools.

Return Value

Return true if the supplied path is a tool symbol name or a path to a tool symbol instance, otherwise return false.

Make the string name of the tool with the given tool number using the given format type.

mtm_make_tool_name(

- ByVal *mtn* As Double
- ByVal *formatType* As Long

) As String

Parameters

[in] mtn
Tool number.
[in] formatType
Format type.

Return Value

Formatted name of tool.

Return the pre-loaded tool loading information for the given station in the given multi-tool. If the station is empty, the returned tool number will be zero. Return true if the returned load data is valid, otherwise false.

mtm_multi_tool_load_info(

- ByVal *mtn* As Double
- ByVal *station* As Double
- ByRef *tool* As Double
- ByRef *angle* As Double
- ByRef *die* As Double

) As Boolean

Parameters

[in] mtn
Tool number of multi-tool.
[in] station
Station number (1-based).
[out] tool
Tool number (or 0 if no tool loaded).
[out] angle
Load angle.
[out] die
Die clearance.

Return Value

Return true for success, false for failure.

Start a scan of the multi-tools in the Tool Library.

mtm_multi_tool_scan() As Boolean

Return Value

Return true for success, false for failure.

Return the number of stations in the given multi-tool.

mtm_multi_tool_stations(

- ByVal *mtn* As Double

) As Long

Parameters

[in] mtn
Tool number of multi-tool.

Return Value

Return the number of stations in the given multi-tool.

Get the details (e.g. MTNUM, MTX, MTY, etc) of the next tool in the current tool scan or the next multi-tool in the current multi-tool scan.

mtm_next() As Boolean

Return Value

Return true for success, false for failure.

Get the details (e.g. MTNUM, MTX, MTY, etc) of the previous tool in the current tool scan or the previous multi-tool in the current multi-tool scan.

mtm_prev() As Boolean

Return Value

Return true for success, false for failure.

Format the given tool number to the desired number of decimal places.

mtm_round_mtn(

- ByVal *mtn* As Double
- ByVal *places* As Double

) As Double

Parameters

[in] mtn
Tool number.
[in] places
Required number of decimal places.

Return Value

Rounded tool number.

Saves the changes to the tool file (e.g. after adding additional information).

mtm_save() As Boolean

Return Value

Return true for success, false for failure.

mtm_scan(

- ByVal *mask* As String

) As Boolean

Start a scan of the tools in the Tool Library given a mask of tools to scan for (mask may contain any of 'ucsrxlwe'). Multi-tools are not scanned. See also mtm_multi_tool_scan().

Parameters

[in] mask
Mask of tool types to scan (mask may contain any of 'ucsrxlwe').

Return Value

Return true for success, false for failure.

Tool properties

Property	Description
MTA0, MTA1, MTA2, MTA3, MTA4 As Double	Get: Tool record: Setting angle 1, 2, 3, 4 and 5.
MTABLOW As Boolean	Get: Tool record: Air blow tool.
MTAVAIL As Long	Get: Tool record: Number available.
MTD0, MTD1, MTD2, MTD3, MTD4 As Double	Get: Tool record: Die clearance 1, 2, 3, 4 and 5.
MTDESIG As String	Get: Tool record: Designation.
MTFEED As Long	Get: Tool record: Feed release lift type.
MTINDX As Long	Get: Tool record: Auto-index support type (0 - 'n', 1 - '-', 2 - 'y').
MTLUBE As Boolean	Get: Tool record: Lubricated tool.
MTMCODE As String	Get: Tool record: M/C code.
MTMULT As Long	Get: Tool record: Multitool support type (0 - 'n', 1 - '-', 2 - 'y').
MTMULTITYPE As String	Get: Tool record: Multi-tool type.
MTNAME As String	Get: Tool record: Tool description.
MTNIB As Boolean	Get: Tool record: Nibbling tool.
MTNUM As Double	Get: Tool record: Tool number.
MTNUMDP As Long	Get: Number of decimal places supported in tool numbers.
MTPREF As Long	Get: Tool record: Autotooling usage type (0 - 'n', 1 - '-', 2 - 'y').
MTRAD As Double	Get: Tool record: Corner radius.
MTSIZ0 As Long	Get: Tool record: Station size range index (without sleeving).
MTSIZ1 As Long	Get: Tool record: Station size range index (with sleeving).
MTSIZ2 As Long	Get: Tool record: Station size range index (with extra sleeving).
MTSTRIP As Long	Get: Tool record: Stripper lift type.
MTSTYPE As Long	Get: Tool record: Supplementary tool type.
MTSYM As Long	Get: Tool record: Symmetry type (mask of: 1 - X symmetry, 2 - Y symmetry, 4 - 90 degree symmetry, 8 - 180 degree symmetry).
MTTYPE As Long	Get: Tool record: Tool type (0 - unspecified, 1 - circular, 2 - square, 3 - rectangular, 4 - special, 5 - laser/plasma, 6 - wheel, 7 - scribe).
MTUNIT As Double	Get: Tool record: Tool size units (1 for mm, 25.4 for inches).
MTX As Double	Get: Tool record: Tool size X.
MTY As Double	Get: Tool record: Tool size Y.

Mac object: Standard tool loading

The Mac object contains a number of methods that allow you manage your Standard Tool Loading. For more details about the Standard Tool Loading, see Editing Standard Tool Loadings.

Standard tool loading methods

Method	Description
	Find tool in the current standard tool loading. Ignore die, if set to 0. Offset is in mm. Return a mask that contains 1 if offset found and 4 if station found.

Parameters

[in] tool

Tool number.
[in] angle
Angle of tool.
[in] die
Die clearance. If die is passed as 0, no die matching is done.
[in] reserved1
Reserved - set to 0.
[in] reserved2
Reserved - set to 0.
[out] station
Station. The station is returned as it would appear in a load statement i.e. 101 is first station in first multi-tool. If no matching station is found, then station is zero on exit.
[out] offsetX
Turret offset X. The offset is returned in MILLIMETRES, and WORLD coordinates, and is directed from the turret centre to tool centre. The turret offset is returned even if the tool loading is unknown, if this is possible.
[out] offsetY
Turret offset Y. The offset is returned in MILLIMETRES, and WORLD coordinates, and is directed from the turret centre to tool centre. The turret offset is returned even if the tool loading is unknown, if this is possible.

stm_find_tool(

- ByVal *tool* As Double
- ByVal *angle* As Double
- ByVal *die* As Double
- ByVal *reserved1* As Double
- ByVal *reserved2* As Double
- ByRef *station* As Long
- ByRef *offsetX* As Double
- ByRef *offsetY* As Double

) As Long

Return Value

Return a mask that contains 1 if offset found and 4 if station found.

Returns the current STL id.

stm_get_stl_id() As Long

Return Value

Returns the current STL id.

Get the name of the current STL file.

Parameters

[in] reserved
Reserved - set to 0.
[out] name
Name of current STL file.

stm_get_stl_name(

- ByVal *reserved* As Double
- ByRef *name* As String

) As Boolean

Return Value

Return true for success, false for failure.

Get a list of the STL ids for the current machine (as a space separated list).

Parameters

[in] reserved
Reserved - set to 0.
[out] ids
STL ids for the current machine (as a space separated list).

stm_get_valid_stl_ids(

- ByVal *reserved* As Double
- ByRef *ids* As String

) As Boolean

Return Value

Return true for success, false for failure.

Get a list of the STL names for the current machine (as a space separated list).

stm_get_valid_stl_names(

- ByVal *reserved* As Double
- ByRef *names* As String

) As Boolean

Parameters

[in] reserved
Reserved - set to 0.
[out] names
STL names for the current machine (as a space separated list).

Return Value

Return true for success, false for failure.

Returns information about the supplied station.

stm_load_info(

- ByVal *station* As Double
- ByRef *tool* As Double
- ByRef *angle* As Double
- ByRef *die* As Double

) As Boolean

Parameters

[in] station
Station.
[out] tool
Tool number in station.
[out] angle
Angle.
[out] die
Die clearance.

Return Value

Return true for success, false for failure.
Sets the STL id to the passed id.

stm_set_stl_id(

- ByVal *id* As Long

) As Boolean

Parameters

[in] id
STL id.

Return Value

Return true for success, false for failure.

Mac object: Standard sheets

The Mac object contains a number of methods that allow you manage your standard sheets. For more details about standard sheets, see Editing standard sheets.

Standard sheets methods

Method	Description

ssm_get_density(

- ByVal *material* As String
- ByVal *units* As String

Returns the density for the specified material. If the density is not found, a value of 0.0 is returned.

Parameters

[in] material
Name of material to consider.
[in] units

) As Double

Units required ('mm' or 'in').

Return Value

Returns the density for the specified material. If the density is not found, a value of 0.0 is returned.

Returns the (1-based) index of the standard material with given material name in the current machine tool's sheet stock file. 0 is returned if there is no matching material.

ssm_get_index(

- ByVal *material* As String

) As Long

Parameters

[in] material
Name of material to consider.

Return Value

Returns the (1-based) index of the standard material with given material name in the current machine tool's sheet stock file.

Returns the material name of the material with given (1-based) index in the current machine tool's sheet stock file. Considers only materials in use by the current machine tool.

ssm_get_mat(

- ByVal *num* As Long

) As String

Parameters

[in] num
One-based index of material to get.

Return Value

Returns the material name of the material with given (1-based) index in the current machine tool's sheet stock file.

Returns the priority (1 high -> 9 low) of the standard sheet with given index in the current machine tool's sheet stock file. Consider only standard sheets in use by the current machine tool. Return 0 on failure.

ssm_get_sheet_priority(

- ByVal *material* As String
- ByVal *thickness* As Double
- ByVal *units* As String
- ByVal *num* As Long

) As Long

Parameters

[in] material
Name of material to consider.
[in] thickness
Thickness to consider.
[in] units
Thickness units to consider.
[in] num
One-based index of sheet.

Return Value

Return the priority (1 high -> 9 low), or 0 for failure.

Get the size (as SSM_X, SSM_Y, SSM_UNITS) of the standard sheet with given (1-based) index in the current machine tool's sheet stock file. Consider only standard sheets in use by the current machine tool.

ssm_get_size(

- ByVal *material* As String
- ByVal *thickness* As Double
- ByVal *units* As String
- ByVal *num* As Long

) As Boolean

Parameters

[in] material
Name of material to consider.
[in] thickness
Thickness to consider.
[in] units
Thickness units to consider.
[in] num
One-based index of sheet.

Return Value

Return true for success, false for failure.

Return the thickness value and units of the material thickness with given (1-based) index in the current machine tool's sheet stock file. Considers only thicknesses in use by the current machine tool.

ssm_get_thick(

- ByVal *material* As String
- ByVal *num* As Long
- ByRef *units* As String

) As Double

Parameters

[in] material
Name of material to consider.
[in] num
One-based index of thickness to get.
[out] units
Units of thickness value returned.

Return Value

Return the thickness value and units of the material thickness with given (1-based) index in the current machine tool's sheet stock file.

Returns the number of standard materials in use by the current machine tool, in the current machine tool's sheet stock file.

ssm_num_mat() As Long

Return Value

Returns the number of standard materials in use by the current machine tool, in the current machine tool's sheet stock file.

Returns the number of standard sheets for the specified material and thickness in the current machine tool's sheet stock file. Considers only standard sheets in use by the current machine tool.

ssm_num_size(

- ByVal *material* As String
- ByVal *thickness* As Double
- ByVal *units* As String

) As Long

Parameters

[in] material
Name of material to consider.
[in] thickness
Thickness to consider.
[in] units
Thickness units to consider.

Return Value

Returns the number of standard sheets for the specified material and thickness in the

current machine tool's sheet stock file.

Returns the number of thicknesses for the specified material in the current machine tool's sheet stock file. Considers only material thicknesses in use by the current machine tool.

ssm_num_thick(

- ByVal *material* As String

) As Long

Parameters

[in] material
Material to consider.

Return Value

Returns the number of thicknesses for the specified material in the current machine tool's sheet stock file.

Sheet properties

Property	Description
SSM_UNITS As String	Get: Sheet size units.
SSM_X As Double	Get: Sheet size X.
SSM_Y As Double	Get: Sheet size Y.

Mac object: Utility methods

The Mac object contains a small number of utility methods.

Utility methods

Method	Description
fla_create_zip_embed_silent(• ByVal *folder* As String • ByVal *allMachines* As Boolean • ByVal *includeProject* As Boolean) As Boolean	Create a embed ZIP file to send to Support.

Parameters

[in] folder
Folder (existing) where embed ZIP file will be created. The filename is automatically generated from system information.
[in] allMachines
Include information about all machines (true) or only the current machine (false).
[in] includeProject
If a nest project is open and this parameter is true, information from the currently open nest project will be included.

Return Value

Return true for success, false for failure.

Mac object: MAC language access

The Mac object supports the following methods that provide some basic capabilities for running MAC language code and for setting and getting the values of MAC variables.

You will need to use the set and get number/string methods if you want to use temporary MAC variables with the methods that run keystroke commands (mac2(), fmac2(), etc). Otherwise, you should only use these methods if you need to access a MAC function or variable that is not directly supported by the Mac object. Using directly supported methods and properties is more efficient and robust.

Note: The MAC language stores all numeric variables (i.e. reals, integers and Booleans) as real numbers. Therefore, to set or get

any type of numeric variable, use SetNumber() and GetNumber() respectively.

An alternative way of executing MAC language functions is to write some MAC language code in a separate file and use the MacFiles, MacCommands and other related objects to load the file and execute the code.

MAC language methods

Method	Description
Execute(• ByVal *line* As String) As Boolean	Execute a line of MAC language code. Note: Normally Execute() would be used to execute a single MAC statement. However, it is possible to execute multiple MAC statements with one call to Execute(). To do this, separate each MAC statement with a semi-colon and surround all of the statements with braces (i.e. {}). Alternatively, simply call Execute() once for each MAC statement. # Parameters [in] line The line of MAC language code to execute. # Return Value True if the line of MAC code was successfully parsed, otherwise false.
GetNumber(• ByVal *name* As String) As Double	Get the value of the named MAC language numeric variable (note: numeric MAC variables are used for real, integer and Boolean MAC values). # Parameters [in] name Name of MAC variable. # Return Value Returns the value of the named MAC language numeric variable.
GetString(• ByVal *name* As String) As String	Get the value of the named MAC language string variable. # Parameters [in] name Name of MAC variable. # Return Value Returns the value of the named MAC language string variable.
SetNumber(• ByVal *name* As String) As Double	Set the value of the named MAC language numeric variable (note: numeric MAC variables are used for real, integer and Boolean MAC values). # Parameters [in] name Name of MAC variable. [in] value Value of MAC variable.

Return Value

Returns true for success, false for failure.
Set the value of the named MAC language string variable.

SetString(

- ByVal *name* As String

) As Double

Parameters

[in] name
Name of MAC variable.
[in] value
Value of MAC variable.

Return Value

Returns true for success, false for failure.

MacArgument object

The MacArgument object represents an environment within which an argument to a MAC procedure can be defined. It exists as an item in the MacArguments collection.

The MacArgument object has the following properties:

Properties

Property	Description
Application As Application	Gets the Application object.
Parent As MacArguments	Gets the parent object from which this object can be logically reached.
Type As RadMacArgumentType	Gets the MAC argument type.
Value As Variant	Gets/sets the MAC argument value. Note that there is a limit of 10240 bytes for the Value of a MacArgument of Type radMacArgumentTypeString.

Example

MacArguments collection

The MacArguments object is a collection object. It provides access to existing MacArgument objects and allows addition of new MacArgument objects. It exists as a member of the MacCommand object.

This collection is **ordered**. Each MacArgument object should be appended in the correct order of use for the associated MAC procedure.

The MacArguments collection object has the following methods and properties:

Methods

Method	Description
Append(• *ArgumentValue* As Variant) As MacArgument	Creates a new MacArgument object (based on the given value) and appends it onto the ordered collection.

Parameters

ArgumentValue
The value of the new MacArgument object.

Return Value

Returns the new MacArgument object.

Removes all items from the collection.

Clear As Boolean

Return Value

Returns **True** if all items were successfully removed from the collection. If the method fails, it causes an error.

Gets a specific MacArgument object by index from the collection.

Item(

- *ItemIndex* As Long

) As MacArgument

Parameters

ItemIndex
The index of the MacArgument to get from the collection.

Return Value

Returns the requested MacArgument object.

Properties

Property	Description
Application As Application	Gets the Application object.
Count As Long	Gets the number of MacArgument items in this collection.
Parent As MacCommand	Gets the parent object from which this object can be logically reached.

Example

MacCommand object

The MacCommand object represents an environment within which a MAC command can be defined and executed. It exists as an item in the MacCommands collection.

The MacCommand object has the following methods and properties:

Methods

Method	Description
Execute As Boolean	Executes the MAC procedure as specified by the ProcedureName and MacArguments members.

Return Value

Returns **True** if the MAC procedure was executed successfully. If the method fails, it causes an error.

Properties

Property	Description
Application As Application	Gets the Application object.
MacArguments As MacArguments	Gets the MacArguments collection object.
Parent As MacCommands	Gets the parent object from which this object can be logically reached.

ProcedureName As String	Gets/sets the MAC procedure name.

Example

MacCommands collection

The MacCommands object is a collection object. It provides access to existing MacCommand objects and allows addition of new MacCommand objects. It exists as a member of the MacFile object.

The MacCommands collection object has the following methods and properties:

Methods

Method	Description
Add As MacCommand	Creates a new MacCommand object and appends it onto the collection. **Return Value** Returns the new MacCommand object.
Clear As Boolean	Removes all items from the collection. **Return Value** Returns **True** if all items were successfully removed from the collection. If the method fails, it causes an error.
Item(• *ItemIndex* As Long) As MacCommand	Gets a specific MacCommand object by index from the collection. **Parameters** ItemIndex The index of the MacCommand to get from the collection. **Return Value** Returns the requested MacCommand object.

Properties

Property	Description
Application As Application	Gets the Application object.
Count As Long	Gets the number of MacCommand items in this collection.
Parent As MacFile	Gets the parent object from which this object can be logically reached.

Example

MacFile object

The MacFile object represents an environment within which a MAC file can be loaded. It exists as an item in the MacFiles collection.

The MacFile object has the following methods and properties:

Methods

Method	Description

EnquireGlobalValue(

- *Name* As String

) As Variant

Gets the value of the given global MAC variable.

Parameters

Name
The name of the global MAC variable to get the value of.

Return Value

Returns the value of the requested global MAC variable.

Load(

- *File* As String

) As Boolean

Loads the given MAC file (e.g. C:\mac\text.mymacros).

Parameters

File
The MAC file to load, which can be specified in any of the following formats:
- the full path of the MAC file (e.g. C:\mac\text.mymacros),
- the macro filename (e.g. text.mymacros) of a MAC file within the dat folder,
- just the macro name (e.g. mymacros) of a MAC file within the dat folder.

Return Value

Returns **True** if the MAC file was loaded successfully. If the method fails, it causes an error.

Unload As Boolean

Unloads the previously-loaded MAC file. Note that this method is optional: the MAC file will be automatically unloaded when the MacFile object is destroyed.

Return Value

Returns **True** if the MAC file was unloaded successfully. If the method fails, it causes an error.

Properties

Property	Description
Application As Application	Gets the Application object.
FilePath As String	Gets the file path of the loaded MAC file.
MacCommands As MacCommands	Gets the MacCommands collection object.
Parent As MacFiles	Gets the parent object from which this object can be logically reached.

Example

MacFiles collection

The MacFiles object is a collection object. It provides access to existing MacFile objects and allows addition of new MacFile objects. It exists as a member of the Application object.

The MacFiles collection object has the following methods and properties:

Methods

Method	Description

Creates a new MacFile object and appends it onto the collection.

Add As MacFile

Return Value

Returns the new MacFile object.

Gets a specific MacFile object by index from the collection.

Item(

- *ItemIndex* As Long

) As MacFile

Parameters

ItemIndex
The index of the MacFile to get from the collection.

Return Value

Returns the requested MacFile object.

Properties

Property	Description
Application As Application	Gets the Application object.
Count As Long	Gets the number of MacFile items in this collection.
Parent As Application	Gets the parent object from which this object can be logically reached.

Example (from within this software program)

This simple example demonstrates use of the MacFiles object and objects below it in the object hierarchy (MacFile, MacCommands, MacCommand, MacArguments and MacArgument).

Two MAC files are loaded and MAC procedures from each executed. The MAC files are then unloaded.

```
Public Sub sample()
    On Error Resume Next

    Dim files As Radraft.MacFiles
    Dim file1 As Radraft.MacFile
    Dim file2 As Radraft.MacFile

    Dim cmds1 As Radraft.MacCommands
    Dim cmds2 As Radraft.MacCommands

    Dim cmdA As Radraft.MacCommand
    Dim cmdB As Radraft.MacCommand
    Dim cmdC As Radraft.MacCommand

    Dim arg As Radraft.MacArgument
    Dim resultA As Variant
    Dim resultB As Variant
    Dim resultC As Variant

                    ' Get the MacFiles collection object
    Set files = Application.MacFiles

                    ' Get two items from the collection
                    ' (add them if necessary)
    Set file1 = files.Item(1)
    If (Err) Then
        Err.Clear
        Set file1 = files.Add
    End If

    Set file2 = files.Item(2)
    If (Err) Then
        Err.Clear
        Set file2 = files.Add
    End If
```

```
                ' Load MAC files into the MacFile objects
If (file1.Load ("C:\mac\text.mymacros") = False) Then
    MsgBox "Load of C:\mac\text.mymacros failed"
End If

If (file2.Load ("C:\mac\text.yourmacros") = False) Then
    MsgBox "Load of C:\mac\text.yourmacros failed"
End If

                ' Get the MacCommands collection objects
Set cmds1 = file1.MacCommands
Set cmds2 = file2.MacCommands

                ' Get two MacCommand items from the first MacCommands
                ' collection (add them if necessary)
Set cmdA = cmds1.Item(1)
If (Err) Then
    Err.Clear
    Set cmdA = cmds1.Add
End If

Set cmdB = cmds1.Item(2)
If (Err) Then
    Err.Clear
    Set cmdB = cmds1.Add
End If

                ' Get one MacCommand item from the second MacCommands
                ' collection (add it if necessary)
Set cmdC = cmds2.Item(1)
If (Err) Then
    Err.Clear
    Set cmdC = cmds2.Add
End If

                ' Set the members for the first MacCommand
cmdA.ProcedureName = "mymacro"
                ' Arguments must be added in the correct order
cmdA.MacArguments.Append ("C:\drawings")
cmdA.MacArguments.Append ("test.drg")

                ' Set the members for the second MacCommand
                ' This MAC procedure does not take any arguments
cmdB.ProcedureName = "test"

                ' Set the members for the third MacCommand
cmdC.ProcedureName = "yourmacro"
Set arg = cmdC.MacArguments.Append(20)

                ' Execute the MAC procedures in turn and
                ' retrieve global variables that were set in them
If (cmdA.Execute = True) Then
    resultA = file1.EnquireGlobalValue("MYMACRORESULT")
Else
    MsgBox "Failed to execute cmdA"
End If

If (cmdB.Execute = True) Then
    resultB = file1.EnquireGlobalValue("TESTVALUE")
Else
    MsgBox "Failed to execute cmdB"
End If

If (cmdC.Execute = True) Then
    resultC = file2.EnquireGlobalValue("YOURMACRORESULT")
Else
    MsgBox "Failed to execute cmdC"
End If

                ' Modify an argument and execute one of the
                ' MAC procedures again
arg.Value = 15.5
If (cmdC.Execute = True) Then
```

```
        resultC = file2.EnquireGlobalValue("YOURMACRORESULT")
    Else
        MsgBox "Failed to execute cmdC"
    End If

    file1.Unload
    file2.Unload
End Sub
```

Note that this example is written for use from within this software program. It uses the top-level Application object, which is automatically available. To achieve the same result from an external client, the example requires additional preparatory work. The top-level Application object is not automatically available, and you should therefore obtain a reference to the Application object via the GetObject() and CreateObject() functions. All uses of the top-level Application object should then be replaced with the variable into which your reference is set.

PluginManager object

The PluginManager object provides the means of controlling the appearance of **plug-ins** on the menus of this software program. It exists as a member of the Application object.

.NET plug-ins can contain a procedure called **OnUpdateGUI()**. This special OnUpdateGUI() procedure, if it exists, is run by the software whenever it changes its GUI state. The code in the OnUpdateGUI() procedure can be used to add command items and submenus to the Plug-Ins menu, add new drop-down menus to the main menubar, and add command items and submenus to them. It is the PluginManager object that contains the methods necessary to modify the software's menus in this manner.

The PluginManager object has the following methods and properties:

Methods

Method	Description
AddMenu(• *MenuName* As String • *MenuText* As String) As Boolean	Adds a drop-down menu or submenu to the named menu. **NOTE:** Must only be called from within the OnUpdateGUI() procedure. **Parameters** MenuName The name of the menu to add the new drop-down menu or submenu to. Specify the null string, "", to add a new drop-down menu to the main menubar. Use the "|" character to separate the names of menus and submenus when adding to submenus, e.g. "Plug-Ins|MySubMenu". MenuText The name of the new drop-down menu or submenu. **Return Value** Returns **True** if the new drop-down menu or submenu was added successfully. If the method fails it causes an error.
AddMenuItem(• *MenuName* As String • *MenuText* As String	Adds a command or separator item to the named menu. To add a separator item, both the menu text string and procedure name string should be empty. **NOTE:** Must only be called from within the OnUpdateGUI() procedure. **Parameters** MenuName The name of the menu to add the command item to. Use the "|" character to separate the names of menus and submenus when adding to submenus, e.g. "Plug-Ins|MySubMenu". MenuText The name of the new command item. To add a separator item, the menu text string should be empty.

- *MenuHandlerProcedure* As String

) As Boolean

MenuHandlerProcedure
The name of the procedure to run when the new command item is selected. For .NET plug-ins, this procedure must be a member of the same RadraftInteropPlugIn class as the OnUpdateGUI() method from which AddMenuItem() is being called, and it must be a public or private void function that does not take any arguments.
To add a separator item, the procedure name string should be empty.

Return Value

Returns **True** if the new command item was added successfully. If the method fails it causes an error.

Properties

Property	Description
Application As Application	Gets the Application object.
Parent As Application	Gets the parent object from which this object can be logically reached.

SystemDataFile object

The SystemDataFile object represents an environment within which a System Data File file can be loaded. It allows a SystemDataItem to be retrieved from the file.

The SystemDataFile object has the following methods and properties:

Methods

Method	Description
Item (	Gets a SystemDataItem for the specified key. If the key is not found in the system data file, then it is created.

Parameters

- *Key* As String
- [optional *DefaultValue* As String]

) As SystemDataItem

Key
The key of the item to get, or add.
DefaultValue
The default value of the item if the key is missing ("" if not specified)

Return Value

Returns the SystemDataItem

Close ()

Closes the file object and removes it from the SystemDataFiles collection.

Return Value

Nothing.

Save ()

Overwrites the file opened with any changes to the SystemData. The Method returns True if the file was successfully saved.

Return Value

Nothing - if the file can't be saved, it will cause an error.

Saves the file contained in the object with the name specified. The Method returns True if the file was successfully saved.

SaveAs (

- *Filename* As String

)

Parameters

Filename
The name to save the file as.

Return Value

Nothing - if the file can't be saved, it will cause an error.

Saves the current MDB settings.

SaveMDB ()

Return Value

Nothing - if the file can't be saved, it will cause an error.

Properties

Property	Description
Application As Application	Gets the Application object.
Parent As SystemDataFiles	Gets the parent object from which this object can be logically reached.
Filename As String	Gets the the file name of the System Data file contained in this object.
FullName As String	Gets the full file name of the System Data contained in this object, including the file path and any file extension.
Edited As Boolean	This is True if the file has changed since it was opened or last saved.

Example

SystemDataFiles collection

The SystemDataFiles object is a collection of SystemDataFile objects. It provides access to existing SystemDataFile objects and allows addition of new SystemDataFile objects. It exists as a member of the Application object.

The SystemDataFiles collection object has the following methods and properties:

Public Methods

Method	Description
	Opens a new SystemDataFile object and adds it to the collection. After you have finished with the file, you must call the Close() method to free the data.

Open (

- *Filename* As String
- *DataFileType* As RadDataFileType

) As SystemDataFile

Parameters

Filename
The name of the data file to open. The absolute path is used if it is specified, otherwise the file will be opened from the dat directory. If a numeric value is given, then the data file for the machine with that number will be opened.
A number of 0 will open the psys_conf or psys_user.xxx files.
A number of -1 will open the system data file for the current machine.
If the file does not exist, a new one will be created when you save any changes.
DataFileType
The type of data file to open, either radPersonalFile for data personal to each user or radCommonFile for data shared by all users.

Return Value

Returns the opened SystemDataFile

Opens a new SystemDataFile object and adds it to the collection (as above). However, the specified MDB data is also read and overrides the data in the original file. And values set on the system data are applied to the MDB data.

OpenMdb (

- *Filename* As String
- *DataFileType* As RadDataFileType
- *Machine* As Long
- *Material* As String
- *Thickness* As Double
- *ThicknessUnits* As String
- *Strategy* As String
- *Create* As Boolean

) As SystemDataFile

Parameters

Filename
: The name of the data file to open.

DataFileType
: The type of data file to open, either radPersonalFile for data personal to each user or radCommonFile for data shared by all users.

Machine
: The number of the machine to open the MDB for.

Material
: The material name in the MDB.

Thickness
: The material thickness in the MDB.

ThicknessUnits
: The units for the material thickness (normally 'mm' or 'in')

Strategy
: The Strategy path in the MDB. It may be a name, or several names delimited by / characters. Strategy may be "" if it is not relevant for the current machine.

Create
: Create a new mdb folder if it doesn't exist.

Return Value

Returns the opened SystemDataFile.

As OpenMdb, but opens the MDB data associated with the current material and thickness as well.

OpenCurrentMdb(

- Filename As String
- *DataFileType* As RadDataFileType

) As SystemDataFile

Parameters

Filename
: The name of the data file to open.

DataFileType
: The type of data file to open, either radPersonalFile for data personal to each user or radCommonFile for data shared by all users.

Return Value

Returns the opened SystemDataFile

Gets the path to the MDB folder for the specified material, etc - also gets whether the folder exists.

MdbFolder (

- *Machine* As Long
- *Material* As String
- *Thickness* As Double
- *ThicknessUnits* As String
- *Strategy* As String
- *Create* As Boolean

Parameters

Machine
: The number of the machine to open the MDB for.

Material
: The material name in the MDB.

Thickness
: The material thickness in the MDB.

ThicknessUnits
: The units for the material thickness (normally 'mm' or 'in')

Strategy

- *FolderExists* As Boolean

) As String

The Strategy path in the MDB (if relevant)
Create
Create a new mdb folder if it doesn't exist.
FolderExists
This is set by the function if the folder already exists.

Return Value

Returns the path to the MDB Folder.

Gets the Psys file name for the specified machine number.

PsysFile (

- *Machine* As Long

) As String

Parameters

Machine
Number of the machine to get the psys file name for.

Return Value

Returns the name of the psys file.

Gets the indicated item in the collection. The value passed in is between 1 and count.

Index (

- *FileIndex* As Long

) As SystemDataFile

Parameters

FileIndex
The index to file in the collection. Note: this can change as other files are opened and closed.

Return Value

Returns the specified SystemDataFile

Closes all open SystemDataFile objects. After this has been called it will not be possible to access any data until the file is opened again.

CloseAll()

Return Value

Nothing.

Properties

Property	Description
Application As Application	Gets the Application object.
Parent As Application	Gets the parent object from which this object can be logically reached.
Count As Long	Gets the number of SystemDataFile items in this collection.

Example (from within this software program)

Two System Data files are loaded and items from each file are read, set and added to.

```
Public Sub sample()
    On Error Resume Next

    Dim files As Radraft.SystemDataFiles
    Dim machineFile As Radraft.SystemDataFile
    Dim userFile As Radraft.SystemDataFile
```

```
    Dim itemA As Radraft.SystemDataItem
    Dim itemB As Radraft.SystemDataItem

    Dim textValue  As String
    Dim numValue  As Double

                    ' If required, we may need to check if the system data file
                    ' exists before we try to open it. Note: If the file passed
                    ' to SystemDataFiles.Open does not exist on disk, a new
                    ' empty file will be created in memory.
    If Dir("\\RADAN_DAT\psys_TEST_1") = "" Then
        MsgBox "\\RADAN_DAT\psys_TEST_1 not found"
    End If

                    ' Get the SystemDataFiles collection object
    Set files = Application.SystemDataFiles

                    ' opens the psys_TEST_1 system data file
    Set machineFile = files.Open ("\\RADAN_DAT\psys_TEST_1", radCommonFile)
                    ' opens the psys_user file for the current user
    Set userFile = files.Open (0, radPersonalFile)

                    ' Check they opened correctly
    If (machineFile is Nothing) Then
        MsgBox "Load of \\RADAN_DAT\psys_TEST_1 failed"
    End If

    If (userFile is Nothing) Then
        MsgBox "Load of psys_user.<user_name> failed"
    End If

                    ' Get an item of system data
    Set itemA = machineFile.Item("m1")
                    ' Get an item of system data, with 9999 as the default
    Set itemB = machineFile.Item("m2", "9999")

                    ' Get the value contained in itemA (i.e. the value of m1)
    textValue = itemA.Value()
                    ' value is the default property so doesn't have to be specified
    textValue = itemB

                    ' Similarly Item is the default method on the SystemDataFile
    Set itemA = machineFile("m1")
                    ' Or even
    textValue = machineFile("m1")
    numValue = machineFile("m2")

                    ' To set a value, the following can be used:
    itemA.Value = "1000"
    machineFile.item("m1").Value = "1000"

                    ' and using default properties:
    itemA = "1000"
    machineFile("m1") = "1000"
    machineFile("m1") = 1000

                    ' Finally close the files when they are no longer used:
    machineFile.Save
    If (Err) Then
        MsgBox "Failed to save machine file: " & machineFile.Filename
        Err.Clear
    End If
                    ' Finally close the files when they are no longer used:
    machineFile.Close
    userFile.Close
End Sub
```

Note that this example is written for use from within this software program. It uses the top-level Application object, which is automatically available. To achieve the same result from an external client, the example requires additional preparatory work. The top-level Application object is not automatically available, and you should therefore obtain a reference to the Application object via the GetObject() and CreateObject() functions. All uses of the top-level Application object should then be replaced with the variable into which your reference is set.

SystemDataItem object

TheSystemDataItem object represents a single item of system data contained within a SystemDataFile. A SystemDataItem is obtained from the SystemDataFile using the Item method.

TheSystemDataItem object has the following methods and properties:

Public Methods

None

Properties

Property	Description
Application As Application	Gets the Application object.
Parent As SystemDataFile	Gets the parent object from which this object can be logically reached.
Value As String	Gets and sets the value of the system data item.
Key As String	Gets the system data key for the data item.
Comment As String	Gets and sets the comment in the system data file associated with the data item.

Example

Enumerations

RadDataFileType enumeration

RadDocumentTypeEnum enumeration

RadGUIStateEnum enumeration

RadGUISubStateEnum enumeration

RadLanguageEnum enumeration

RadMacArgumentTypeEnum enumeration

RadDataFileType enumeration

The RadDataFileType enumeration defines constants to identify the types of System Data Files to open.

Members

Member	Description
radPersonalFile	The system data file is specific to each user.
radCommonFile	The system data file is common to all users.

RadDocumentType enumeration

The RadDocumentType enumeration defines constants to identify the types of documents.

Members

Member	Description
radDrawingDocument	Represents a drawing document.
radSymbolDocument	Represents a symbol document.

RadGUIState enumeration

The RadGUIState enumeration defines constants to identify the software's current GUI state.

Members

Member	Description
rad3D	Represents 3D mode.
radDrawing	Represents 2D CAD (drawing editor) mode.
radNest	Represents nest editor mode.
radPart	Represents part editor mode.
radTopLevel	Represents top-level mode.

RadGUISubState enumeration

The RadGUISubState enumeration defines constants to identify the software's current GUI substate.

Members

Member	Description
radAssembly	Represents assembly mode within the rad3D GUI state.
radDrafting	Represents drafting mode within the radDrawing or radPart GUI state.
radModifyNest	Represents modify nest mode within the radNest GUI state.
radOrder	Represents order mode within the radNest GUI state.
radRadbend	Represents 3D modelling mode within the rad3D GUI state.
radRadbendCNC	Represents Radbend mode within the rad3D GUI state.
radScene	Represents scene mode within the rad3D GUI state.
radTooling	Represents tooling mode within the radPart or radNest GUI state.
radVerifier	Represents verifier mode within the radNest GUI state.

RadLanguage enumeration

The RadLanguage enumeration defines constants to identify the software language.

Members

Member	Description
radCzech	Represents Czech.
radDanish	Represents Danish.
radDutch	Represents Dutch.
radEnglish	Represents British English.
radFinnish	Represents Finnish.
radFrench	Represents French.
radGerman	Represents German.
radGreek	Represents Greek.
radHungarian	Represents Hungarian.
radItalian	Represents Italian.
radJapanese	Represents Japanese.
radKorean	Represents Korean.
radLithuanian	Represents Lithuanian.
radPolish	Represents Polish.
radPortuguese	Represents Portuguese.
radPortugueseBrazil	Represents Brazilian Portuguese.
radRussian	Represents Russian.
radSimplifiedChinese	Represents Simplified Chinese.
radSpanish	Represents Spanish.

radSpanishArgentina	Represents Argentinian Spanish.
radSwedish	Represents Swedish.
radTraditionalChinese	Represents Traditional Chinese.
radTurkish	Represents Turkish.
radUSEnglish	Represents American English.

RadMacArgumentType enumeration

The RadMacArgumentType enumeration defines constants to identify the types of arguments to MAC procedures.

Members

Member	Description
radMacArgumentTypeNone	Represents an unspecified argument.
radMacArgumentTypeNumeric	Represents a numeric argument.
radMacArgumentTypeString	Represents a string argument.